Primary School

Lesson 1

Lesson 1

How does a computer think? The PB&J sandwich lesson

Age group: Groep 5–8 (ages 8–12)

Duration: 45–60 minutes

Devices required: None — fully unplugged

SLO alignment: Domain A (21A), Domain B (22B)

Lesson overview

Learning goal: Students can explain in their own words that a computer follows instructions exactly, one step at a time, and that the order of those steps determines the outcome.

What you need: Bread, peanut butter, and jam, or printed picture cards of these items if allergies are a concern. Whiteboard and marker. Paper and pencils for the exercise.

Key vocabulary to write on the board and leave up all lesson

- Algorithm: a set of step-by-step instructions
- Input: what goes in (ingredients, data, commands)
- Output: what comes out (the sandwich, the search result, the route)
- Sequence: the order of steps

Role model of the day: Ada Lovelace (1815–1852) was a mathematician who wrote the world's first algorithm designed to be processed by a machine, over 100 years before computers were invented. She is considered the world's first computer programmer. Today students will write their own algorithms, just like she did.

Phase 1. Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

"What do you think a robot could do better than a human, and what could it never do?"

Ask students to turn to the person next to them and share one idea each. Take 3–4 answers from the class. Write the two most interesting ones on the board — you will come back to them at the end of the lesson. Do not correct or guide their answers. Accept everything.

Teacher tip: Common answers include "a robot could never be kind" or "a robot would always be faster." Both are great. Neither is wrong. The nuance will emerge through the activity itself — you don't need to explain it yet.

Real world connection: GPS apps sometimes give directions that technically work but make no sense — like taking you through a tiny side street when the main road is right there. That happens because the algorithm doesn't know everything a human knows. Today students will experience exactly this problem from the inside.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: Robot Teacher

Tell the class: *"I am going to be a robot for the next 2 minutes. You need to give me instructions to walk from here to the door. But I only understand four commands: STEP FORWARD, TURN LEFT, TURN RIGHT, and STOP. Nothing else."*

Stand still and wait for instructions. Follow them exactly and literally. If a student says "walk to the door" — do nothing. That is not one of the four commands. If they say "turn left" when you are facing the wrong direction, turn left anyway.

After 2 minutes, stop and ask: *"What was hard about giving me instructions? What did you have to think about that you don't normally think about when you walk somewhere?"*

Teacher tip: The more literally you follow — and fail to follow — instructions, the better the learning moment. Students find it funny when you stand still because they said "go to the door." That laugh is the moment the concept locks in. If you bump into a desk because they didn't tell you to turn, even better.

If students are shy: Have students write instructions on small cards first, then pass them to you one at a time. This gives quieter students a way to participate without speaking in front of the class.

Phase 3 — Concept introduction

Time: 10–20 minutes

Write the word **ALGORITHM** on the board. Tell students: *"This is a word that sounds complicated but describes something you already do every day."*

An algorithm is a set of step-by-step instructions that solves a problem or completes a task. The instructions must be in the right order, and precise enough that anyone — or any machine — could follow them without guessing.

Give these three real-world examples and write them on the board:

- **A recipe** — if you add the eggs before you mix the flour, the cake comes out wrong. Order matters.
- **Your morning routine** — you always put on socks before shoes. That is an algorithm.
- **LEGO instructions** — if you skip step 14, the model falls apart in step 20. Each step depends on the one before it.

Then say: *"Computers run on algorithms. Every single thing a computer does — playing a video, sending a message, showing you a search result — is the result of someone writing very precise instructions. Today we are going to write one ourselves, and find out what happens when the instructions aren't precise enough."*

Real world connection — bring this in now: Ask the class, *"Has a GPS ever given you directions that seemed wrong? Or has a streaming service recommended something you hate?"* Those are algorithm mistakes — the instructions were precise, but they didn't account for everything a human would notice. This is exactly the problem students are about to experience.

Teacher tip — vocabulary: Remind students of the three words on the board: Algorithm, Input, Output. They will use them in the discussion and the exercise.

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The PB&J sandwich robot

This is the core activity of the lesson. It is one of the most powerful and well-tested ways to teach algorithms to young learners — used across Dream Space, CS Unplugged, and coding education programs worldwide.

Setup (2 minutes) Place on the table at the front: a loaf of bread still in the bag, a closed jar of peanut butter, a closed jar of jam, a knife, and a plate. Tell the class:

"You are going to give me instructions to make a peanut butter and jam sandwich. I am the robot. I will follow your instructions exactly. I cannot guess what you mean. I cannot use common sense. If you don't tell me to do it, I won't do it."

Round 1: The class gives instructions (5–7 minutes) Ask for the first instruction. A student will probably say something like "put the peanut butter on the bread." Pick up the closed jar and place it on top of the closed bag of bread. Look confused. *"Is that right? I put the peanut butter on the bread. The jar is on the bag."*

Students will protest — they didn't mean that. Say: *"I know what you meant. But I'm a robot. I can only do exactly what you say."*

Continue following instructions extremely literally. Do not open the jar until told to open the jar. Do not take bread out of the bag until told. Spread peanut butter on the outside of the bread if that is technically what the instruction says. The more ridiculous the result, the better the learning.

Round 2: Pairs write better instructions (10 minutes) Stop the demonstration. Tell students: *"Now you are going to fix this. In pairs, you have 8 minutes to write a perfect algorithm. It must be so precise that even a robot that cannot guess could follow it."*

Give each pair paper. Steps must be numbered. Walk around and push them to be more precise: *"What if I don't know what 'spread' means? What if I don't know which side of the knife to use?"*

Round 3: Test one pair's algorithm (5 minutes) A volunteer pair reads their algorithm aloud while you follow it literally. This time the class can spot mistakes and suggest fixes in real time. This is the debugging moment — finding errors and correcting them.

Teacher tip — making it funny: The more you commit to being a literal robot, the more students laugh and the more they learn. Putting the closed jar on the closed bag is the right move. Every failed instruction is a perfect teaching moment. The embarrassment of the algorithm failing is exactly what makes the concept unforgettable.

Teacher tip — allergy alternative: If allergies are a concern, use printed picture cards of the ingredients. You can also substitute any simple physical task: making a paper aeroplane, stacking blocks in a specific order, or drawing a shape.

If no props are available — fully paper version: Give each pair a printed sheet with pictures of the ingredients. They write instructions for assembling the images. A partner acts as robot and follows the instructions using cut-out pieces of paper. Works exactly as well.

Phase 5 — Discussion

Time: 42–47 minutes

Use these three questions. You don't have to use all three — one good discussion is better than three rushed ones. Pick the one that feels most alive based on how the class went.

Question 1: *"A robot followed your instructions exactly and got the wrong result. Whose fault is that — the robot's or the programmer's?"* Follow-up: *"Does this change how you think about technology making mistakes? When a GPS takes you the wrong way, is that the GPS's fault?"*

Question 2: *"Go back to the question from the start of the lesson: what can a robot do better than a human, and what can it never do? Has your answer changed?"* This brings the lesson full circle. Point to the answers on the board from the start. Students will often notice they were partially right and partially wrong, that is the perfect conclusion.

Exercises

Time: 47–57 minutes

Exercise 1, Write the perfect algorithm *Written, no devices needed · 10 minutes · Core exercise*

Working individually, students write a step-by-step algorithm for one of the following tasks they choose:

- Getting ready for school in the morning
- Making a glass of chocolate milk
- Feeding a pet (or an imaginary pet)

Rules for the algorithm:

- Each step must be numbered
- Each step must be one action only, not two actions combined
- The algorithm must have at least 6 steps
- A robot that cannot guess must be able to follow it

What the teacher looks for: You are not marking this for correctness — you are watching for understanding. Good signs: students who catch themselves writing vague steps and make them more precise unprompted. Students who argue with their partner about whether a step is specific enough. Students who add steps they initially skipped, like "open the fridge door" before "take out the milk."

Partner test: When students finish, they swap with a partner. The partner reads the algorithm and marks any step that is not precise enough with a question mark. The original writer must then improve that step. This is the debugging phase.

No devices needed. Paper and pencil only. If time is short, skip the partner test — even an incomplete algorithm shows whether the concept has landed.

Exercise 2 — What order do these go in? *Sequence sorting game · 8 minutes · Whole class or pairs*

Print and cut out the following 6 instruction cards, or write them on the board. Students put them in the correct order to make a cup of tea:

1. Pour water into the cup
2. Remove the teabag from the cup
3. Wait 3 minutes
4. Boil the kettle
5. Put the teabag in the cup
6. Add milk if desired

Students write numbers 1–6 next to each step in the correct order, then compare with a partner. If they disagree on any step, can they argue their case?

Discussion at the end: Ask, *"Is there only one correct order? Or could some steps swap?"* Yes — "add milk" could come before or after removing the teabag depending on preference. This introduces the idea that some algorithms have flexibility built in, and some have none at all.

No devices needed. Works with physical cards, written on whiteboards, or projected on a smartboard.

Extension — Algorithm for a robot chef *For fast finishers · 10 minutes*

A robot chef has been hired at the school canteen. It can only follow written instructions. It has no common sense — if something is not in the instructions, it will not do it. It also cannot taste food, so it cannot adjust.

Task: write the algorithm for one item on today's school lunch menu. It must be detailed enough that the robot could follow it without any help.

Challenge questions:

- What would happen if a student had a nut allergy and the robot followed your algorithm — would they be safe?
- What if the robot ran out of one ingredient halfway through — what should it do? How would you write that into the algorithm?
- Can you write an algorithm that works for making ANY sandwich, not just one specific type?

These questions introduce input validation and generalisation without naming them. Students will encounter these ideas again in later weeks.

Share and reflect

Time: 52–57 minutes

Ask 2–3 students to read one step from their algorithm aloud. The class listens and raises a hand if they spot a step that is not precise enough. Keep it short and warm — this is not a presentation, just a light public share moment.

Then ask every student — verbally or on a sticky note — to complete this sentence:

"An algorithm needs to be _____ because computers cannot _____."

Collect sticky notes or hear 3–4 answers. Write the best one on the board and leave it there. Close with: *"Next week we will look at what computers are actually very good at — spotting patterns — and how that connects to something you use every day."*

Teacher dashboard note — what to observe after the lesson: Did students catch themselves writing vague instructions and self-correct? Did they debate whether a step was precise enough? Did the exit reflection sentences show understanding beyond "computers follow instructions"? If yes to most — the class is ready for Week 2. If not — the warm-up in Week 2 should briefly revisit the sandwich example before moving on.

Lesson 2

Lesson 2

Don't repeat yourself — loops and efficiency

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain B (22B) **Builds on:** Lesson 1 — students already understand algorithms and sequences

Lesson overview

Learning goal Students can explain what a loop is, why programmers use them, and identify where repetition exists in both code and everyday life. They can spot a "bug" in a loop and suggest a fix.

What you need Whiteboard and marker. Paper and pencils for every student. Optional: sticky notes or index cards for the loop cards activity. A clear space at the front of the room for the physical activity.

Key vocabulary to write on the board and leave up all lesson

- Loop — an instruction that repeats a set of steps a certain number of times
- Iteration — one single run through the loop
- Bug — an error in the code that causes the wrong output
- Debug — finding and fixing a bug
- Efficient — doing the most with the least effort

Role model of the day Grace Hopper (1906–1992) was a computer scientist and US Navy admiral who helped invent one of the first programming languages. She is also the person who found the very first real computer bug — a moth stuck inside a computer causing errors. She taped it into her notebook and wrote "First actual case of bug being found." The words "bug" and "debug" come directly from her. Today students will debug their first loop, just like she did.

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"If I asked you to write down every single step of brushing your teeth for an entire year — every stroke, every rinse, every spit — how many lines would you need to write?"*

Ask students to turn to the person next to them and make a guess. Take 3–4 answers. Write the biggest and smallest guesses on the board.

Tell them: *"A dentist recommends 2 minutes of brushing twice a day. That is roughly 2 strokes per second. That is 240 strokes per session, 480 per day, 175,200 per year. Imagine writing 175,200 lines of code just for brushing teeth. There has to be a better way. Today we are going to find it."*

Teacher tip: Don't calculate this precisely in class — the point is the scale, not the exact number. Students finding it absurd is exactly the right reaction. That absurdity is the motivation for loops.

Real world connection: Every time your phone plays music, it does not have a separate instruction for each beat. Every time a game character runs, it does not have a separate instruction for each footstep. Repetition is everywhere in technology, and loops are how programmers handle it without losing their minds.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: The human loop

Ask the whole class to stand up. Tell them: *"I am going to give you a set of instructions. Listen carefully."*

Write this on the board:

1. Clap once
2. Stamp your foot
3. Say "beep"

Tell students to run through the three steps once. They do it. Then say: *"Now I want you to do it again. And again. And again."* Let them run it a few times and then suddenly shout *"STOP."*

Ask: *"How many times did you do those three steps?"* They probably lost count. Then say: *"Now imagine I told you at the start: do those three steps exactly five times. You would have known when to stop and you would have counted automatically. That instruction — do this set of steps X number of times — is a loop."*

Rewrite the instructions on the board:

LOOP 5 TIMES:

1. Clap once
2. Stamp your foot
3. Say "beep"

Run it again. Students count the iterations themselves and stop at 5.

Teacher tip: The physical version makes the concept completely concrete. Students are literally executing a loop with their bodies. The moment they count iterations and stop at

exactly 5 is the moment the concept lands. Don't skip this — it takes only 3 minutes and it makes the rest of the lesson much easier.

Follow-up question: *"What would happen if I wrote LOOP FOREVER instead of LOOP 5 TIMES?"* Students will immediately say they would never stop. Tell them that is exactly what happens when a program has a bug in its loop — it runs forever and crashes the computer. That is called an infinite loop.

Phase 3 — Concept introduction

Time: 10–20 minutes

Refer back to Lesson 1. Say: *"Last week you wrote a sandwich algorithm. Some of you had steps like 'spread the peanut butter across the bread, left to right, covering the whole surface.' How many spreading strokes would that actually be?"* Students will estimate — maybe 8 or 10.

Tell them: *"A programmer would not write Stroke 1, Stroke 2, Stroke 3... all the way to Stroke 10. They would write:"*

Write on the board:

LOOP 10 TIMES:

- Move knife one stroke across the bread

"Same result. Ten times fewer lines of code. That is what loops do — they make code efficient."

Now introduce the three types of loop in plain language. Write each on the board as you explain it:

Count loop — repeat this exactly X number of times. Example: do 10 jumping jacks. You know when it ends.

Until loop — repeat this until something changes. Example: keep stirring until the sugar dissolves. You don't know exactly how many times — you stop when the condition is met.

Forever loop — repeat this indefinitely. Example: a clock ticking, a heartbeat, a game running in the background waiting for you to press a button. These loops only stop when the whole program stops.

Give a real-world example for each:

- Count loop: a washing machine spin cycle — spins exactly 800 times
- Until loop: a self-driving car braking — keeps applying brakes until speed reaches zero
- Forever loop: your phone screen checking every second whether you have a new notification

Teacher tip — keep it simple: You do not need to go deep on until loops today. The count loop and the forever loop are enough for this age group. The until loop is introduced here so students have the vocabulary, but the challenge focuses on count loops and debugging a forever loop.

Real world connection: Ask students: *"When does a video game loop? When does it stop?"* They will usually say the game loops forever while you play, but stops when you turn it off or lose all your lives. That is exactly right — games are forever loops with a stop condition buried inside them.

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The Fortnite dance bug

This activity comes directly from the Dream Space Unplugged Explorers curriculum and has been tested with this age group. It is physical, funny, and makes the debugging concept impossible to forget.

Setup (1 minute) Clear a small space at the front or ask students to push their chairs back slightly. You need enough room for students to stand and move.

Round 1: Write and run the loop (8 minutes)

Tell students: *"Imagine a game programmer has written a dance emote for a Fortnite character. Here is their code:"*

Write on the board:

LOOP 3 TIMES:

1. Dab
2. Spin
3. Jump

Have the whole class stand up and run the code. Count the iterations out loud together. After three complete loops, everyone sits down.

Ask: *"Did it work? Did you do exactly what the code said?"* Yes. *"That is a working loop."*

Round 2: Introduce the bug (7 minutes)

Tell students: *"The programmer updated the dance, but accidentally introduced a bug. Here is the new code:"*

Erase the board and write:

LOOP FOREVER:

1. Dab
2. Spin
3. Touch your toes

Have students stand and begin running the code. After the first iteration, they will be bent over touching their toes. Now try to spin from that position. They cannot — they are stuck.

Let the chaos run for about 30 seconds, then call stop. Ask: *"What went wrong?"*

Students will identify two problems: the loop runs forever with no stop condition, and step 3 leaves them in a position where step 2 in the next iteration is physically impossible.

Round 3: Debug it in teams (7 minutes)

Tell students to work in pairs. Their job is to fix the code. Rules:

- They must keep all three dance moves
- They must fix the loop so it ends
- They must fix the sequence so each step flows into the next

Give them 5 minutes to write their corrected version on paper. Then take 3–4 pairs to share their solution and explain why their fix works.

The simplest correct answer is:

LOOP 3 TIMES:

1. Dab
2. Spin
3. Jump *(not touch your toes — jump brings you back to standing)*

But accept any solution that works. Some students may add a step 4: "Stand up straight" before the loop repeats. That is a perfectly valid debug. Celebrate creative fixes.

Teacher tip: Do not give the answer before students have tried. The 5 minutes of pair work is where the learning happens. Some pairs will struggle — walk around and ask: *"After step 3, what position are you in? Can you do step 2 from that position?"* That question is usually enough to unlock the problem without giving it away.

Teacher tip — managing the physical activity: If the class is very energetic and the physical version risks becoming chaotic, have students mime the moves at their seats rather than standing and moving around the room. The concept lands just as well when seated.

No props needed — fully paper version: Students draw stick figures in each position and annotate them with the step number. They then draw arrows between figures to show the flow. The problem of "stuck bent over" is just as visible in a drawing as it is in person.

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In Round 1 the loop worked perfectly. In Round 2 it broke. What was the difference between those two programs?"* Follow-up: *"It wasn't just the forever loop that broke it — the sequence inside the loop was also the problem. What does that tell us about how loops and sequences work together?"*

Question 2: *"Where in real life does something loop forever — and what stops it?"* Follow-up: *"A heartbeat loops forever — what stops it? A washing machine loops — what stops it? A YouTube video autoplays forever — what stops it? Are all forever loops bad, or are some of them useful?"*

Question 3: *"Last week we said computers are precise. Today we said they are also efficient. Are those two things in tension — can you be both precise AND efficient at the same time?"* This is the hardest question and is designed for Groep 7–8. For Groep 5–6, replace it with: *"If loops make code shorter, why don't programmers just use loops for everything?"*

Teacher tip: Question 2 is the most accessible and usually generates the best discussion across all ages. The connection between forever loops and everyday cycles — heartbeats, seasons, school timetables — surprises students and makes the concept feel much larger than coding.

Exercises

Time: 47–57 minutes

Exercise 1 — Rewrite it with a loop *Written, no devices needed · 8 minutes · Core exercise*

Last week students wrote algorithms with every step listed out. Today they are going to make those algorithms more efficient using loops.

Present this algorithm on the board — tell students it is a robot brushing someone's hair:

1. Move brush from top of head to bottom
2. Move brush from top of head to bottom
3. Move brush from top of head to bottom
4. Move brush from top of head to bottom
5. Move brush from top of head to bottom
6. Move brush from top of head to bottom
7. Move brush from top of head to bottom
8. Move brush from top of head to bottom
9. Move brush from top of head to bottom
10. Move brush from top of head to bottom

Working individually, students rewrite this as a loop. They should also write a second, more complex version that adds: brush left side, brush right side, brush back — and loops through all three sections three times each.

What the teacher looks for: Students who correctly identify the repeated unit (the single brushstroke) and wrap it in a count loop. Students who attempt the complex version and correctly nest the section changes inside the count are showing strong understanding — that is the beginning of nested loops, which appears in later weeks.

Partner check: Students swap papers and check whether their partner's loop would produce the same output as the original long version. If not, they suggest a fix.

Exercise 2 — Spot the bug *Whole class or pairs · 7 minutes*

Present these three buggy loops on the board one at a time. Students work in pairs to find the bug and write the fix.

Buggy loop 1:

LOOP 5 TIMES:

1. Stand up
2. Sit down
3. Lie on the floor

Bug: After step 3, the next iteration starts with "Stand up" — which is fine. But if this is a classroom robot, lying on the floor five times in a row is probably not the intended behaviour. Is this a logic bug or a sequence bug? Students should argue their case.

Buggy loop 2:

LOOP FOREVER:

1. Add one sugar to the coffee
2. Stir

Bug: This loop never stops. The coffee will have infinite sugar. Students must add a stop condition. Most will suggest: LOOP UNTIL coffee tastes sweet enough — which is an until loop. Good. Ask: how does a machine know when coffee tastes sweet enough? That is the hard question — machines cannot taste.

Buggy loop 3:

LOOP 3 TIMES:

1. Open fridge
2. Take out juice
3. Pour glass

Bug: there is no "close the fridge" step. After 3 iterations the fridge has been opened 3 times and never closed. This is a resource leak — a real concept in programming where loops use up a resource and never release it. Students don't need the term, but they should notice the fridge is left open.

Teacher tip: Buggy loop 2 is the hardest and the most interesting. The question of "how does a machine know when something is good enough" is genuinely difficult and connects directly to AI and sensor-based computing in later weeks. Don't resolve it for students — leave the question open and write it on the board as something the class will come back to.

Extension — Write a loop for something in your life *For fast finishers · 10 minutes*

Students choose any repeated activity from their own life and write it as a loop. They must:

- Name the activity and the type of loop (count, until, or forever)
- Write out the steps inside the loop
- Describe what stops the loop (or explain why it never stops)
- Find one possible bug and write the fix

Examples to suggest if students are stuck: a dog wagging its tail when it hears a noise, a traffic light changing colours, a vending machine dispensing a snack, a teacher marking homework.

Challenge question for fast finishers: *"Can you write a loop that has another loop inside it? Think about a weekly school timetable — each week repeats, and inside each week, each day repeats."* This is an introduction to nested loops that will be explored more deeply in the extended 12-week program.

Share and reflect

Time: 52–57 minutes

Ask 2–3 pairs to read their corrected loop from Exercise 1 or their bug fix from Exercise 2. After each one, ask the class: *"Would that work? Is it efficient? Is there still a bug hiding in there?"*

Then ask every student — verbally or on a sticky note — to complete this sentence:

> *"A loop is useful when _____ and dangerous when _____."*

Collect sticky notes or hear 3–4 answers. Write the most interesting one on the board. Close with: *"Next week we look at how computers make decisions — not just following a sequence and not just repeating, but actually choosing what to do next based on what is happening. And the tool they use for that is something you already understand: if this, then that."*

Teacher dashboard note — what to observe after the lesson: Did students identify the bug in Round 2 without being told what to look for? Did they connect the physical impossibility (stuck bent over) to the concept of a loop error? In Exercise 2, did students engage with the sugar loop question — how does a machine know when to stop? Students who wrestle genuinely with that question are showing exactly the kind of critical thinking this program is designed to build. Note which students push back on the idea that machines can sense quality — they are the ones to stretch in Week 3.

Lesson 3

Lesson 3

If this, then that — how computers make decisions

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain A (21A), Domain B (22B) **Builds on:** Lesson 1 (algorithms, sequences) and Lesson 2 (loops, bugs)

Lesson overview

Learning goal Students can explain what a conditional is, write a basic IF/THEN/ELSE rule in plain language, and identify where conditional logic is already at work in technology they use every day.

What you need Three sheets of coloured paper — one red, one blue, one green — placed on the floor at the front of the room. Whiteboard and marker. Paper and pencils. Optional: sticky notes for the exit reflection.

Key vocabulary to write on the board and leave up all lesson

- Conditional — a rule that only runs if a certain condition is true
- IF — the condition being checked
- THEN — what happens if the condition is true
- ELSE — what happens if the condition is not true
- Boolean — a value that is either true or false, nothing in between

Role model of the day Katherine Johnson (1918–2020) was a mathematician at NASA whose conditional calculations determined whether Apollo 11 would safely reach the Moon and return. She wrote the equations that told the spacecraft: IF you are on this trajectory AND within this speed range, THEN fire the engine for exactly this many seconds, ELSE abort. Without her conditionals, the Moon landing does not happen. Today students will write their own conditional logic — the same kind of thinking that got humans to the Moon.

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"Have you ever noticed that your phone screen turns off by itself? Or that a game character takes damage when they walk into fire? How does the device know to do that — nobody pressed a button?"*

Ask students to turn to the person next to them and share one idea. Take 3–4 answers from the class. Write the most interesting ones on the board.

Tell them: *"Those things happen because someone wrote a rule. Not a step-by-step sequence and not a loop — a different kind of instruction entirely. A rule that says: IF something specific happens, THEN do this. Today we are going to learn how to write those rules, and we are going to find out they are behind almost everything a computer does."*

Teacher tip: Students often guess "sensors" or "the camera" — those are partly right, and worth acknowledging. The key point to land is that the sensor just provides data. It is the conditional rule written by a programmer that decides what to do with that data. A sensor alone does nothing.

Real world connection: Every time you walk through automatic doors at a supermarket, a sensor detects your presence — but it is a conditional rule that decides what to do: IF something is within 1 metre of the sensor, THEN open the doors, ELSE keep them closed. The sensor and the conditional work together. One provides the data, the other makes the decision.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: Human conditionals

Tell students: *"I am going to read out some IF/THEN rules. If the rule applies to you, do the action. If it does not apply to you, do nothing. This is exactly what a computer does when it runs a conditional."*

Read these rules one at a time, pausing after each one:

- IF you are wearing something blue, THEN stand up.
- IF you had breakfast this morning, THEN clap twice.
- IF you have a pet at home, THEN wave your hand.
- IF you walked to school today, THEN tap your desk.

After the fourth rule, ask: *"Did everyone do every action?"* No. *"Why not?"* Because the condition was not true for everyone. *"What did you do when the condition wasn't true for you?"* Nothing. *"In coding, that 'nothing' is the ELSE — the default action when the condition is false."*

Reread the first rule with an ELSE added: *"IF you are wearing something blue, THEN stand up. ELSE stay seated."*

Ask: *"Is that any different from what you actually did?"* No — because doing nothing IS the ELSE. The computer always has to know what to do in both cases.

Teacher tip: Some students will stand up when they shouldn't, or miss their cue. That is fine — use it. If a student who has no pet waves anyway, say: *"The condition was not true for you — that is a bug. You executed the wrong branch."* Students find this funny and it reinforces the precision concept from Lesson 1.

Follow-up question: *"What if I changed the rule to: IF you are wearing blue OR you are wearing red, THEN stand up? Is that different?"* This plants the seed for AND/OR logic which appears in the exercise later.

Phase 3 — Concept introduction

Time: 10–20 minutes

Write the basic syntax on the board and leave it up for the entire lesson:

IF (condition is true)
THEN (do this)
ELSE (do this instead)

Tell students: *"This is one of the most important structures in all of computing. Almost every interesting thing a computer does is built on this pattern. Let's break it down."*

Explain each part:

IF is the question the computer is asking. It is always a yes or no question — there is no maybe. The computer checks whether something is true or false. In coding this is called a **Boolean** — a value that can only ever be true or false, nothing in between.

THEN is what happens if the answer is yes. The computer runs this branch.

ELSE is what happens if the answer is no. The computer runs this branch instead. If you do not write an ELSE, the computer does nothing when the condition is false — which is sometimes fine, but sometimes a bug.

Give three real-world examples. Write each one on the board in the IF/THEN/ELSE format:

Example 1 — automatic street lights: IF the sky is dark THEN turn the street lights on ELSE turn the street lights off

Example 2 — a game character: IF the character touches water THEN reduce health by 10 ELSE keep health the same

Example 3 — a phone notification: IF a new message arrives THEN vibrate the phone ELSE do nothing

Ask students: *"In example 3 — is 'do nothing' a good ELSE? What if you are in a meeting and your phone is already vibrating from a previous notification?"* This introduces the idea that ELSE matters just as much as THEN — a missing or wrong ELSE is one of the most common sources of bugs in real software.

Real world connection: Ask students: *"When does your school's heating turn on in the morning?"* They probably don't know exactly, but they know it is not a person switching it on. It is a conditional: IF the temperature inside the building drops below 18 degrees, THEN turn on the heating. ELSE leave it off. The same logic runs in fridges, traffic lights, fire alarms,

and aircraft autopilot systems. Conditionals are not a coding concept — they are how the entire built world around us works.

Teacher tip — connecting to previous lessons: Take a moment to show how all three lessons connect on the board. Write: Sequence (Lesson 1) → Loop (Lesson 2) → Conditional (Lesson 3). Tell students: *"A real program uses all three. A game loops forever, it sequences your character's movements, and it uses conditionals to decide what happens when you hit something. Next week we will start putting all three together."*

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The floor is lava

Place a red piece of paper, a blue piece of paper, and a green piece of paper on the floor at the front of the room, spaced slightly apart. These are the "sensors" in the classroom floor.

Setup (1 minute) Tell students: *"This classroom is now a video game level. The floor has sensors built into it. Each colour triggers a different rule. You are going to write the conditional logic that runs this game — and then we are going to test whether your code actually works."*

Round 1: Write the rules as a class (8 minutes)

Together, decide what each colour does. Write suggestions on the board and vote. A typical result:

- Red paper: lose 10 health points
- Blue paper: gain a shield (immune to damage for 5 seconds)
- Green paper: speed drops to zero — frozen

Once the three rules are agreed, ask students to write them on their paper in the IF/THEN/ELSE format. They should write all three rules as a connected block — what happens for each colour, and what happens if the teacher steps on normal floor (no colour).

A correct version looks like:

```
IF foot touches RED
THEN lose 10 health

ELSE IF foot touches BLUE
THEN gain a shield

ELSE IF foot touches GREEN
THEN freeze — speed = 0

ELSE
continue normally
```

Teacher tip: Write the term ELSE IF on the board and explain it: *"When you have more than two options — not just true or false, but three or four possibilities — you chain your conditionals using ELSE IF. The computer checks each condition in order and runs the first one that is true. After that, it skips all the others."* This is a significant conceptual step and worth spending two minutes on clearly.

Round 2: Execute the code (8 minutes)

The teacher walks around the front of the room while a student narrator reads the active conditional rules aloud. The teacher deliberately steps on each colour one at a time. The class must respond correctly — shouting out what the rule triggers — and must stay silent when the teacher steps on plain floor (the ELSE condition).

Make deliberate mistakes: step on blue immediately after stepping on red. Ask: *"I just lost 10 health and now I gained a shield. Do I still have the shield? Does the shield cancel the damage I already took?"* Students will disagree. This is intentional — it introduces the concept of rule order and state, which is genuinely complex in real programming.

Round 3: Pairs extend the rules (7 minutes)

Tell students: *"The game designer wants to make the level harder. Add two new rules to your conditional block:"*

- IF foot touches RED and the player is already at zero health, THEN the game ends
- IF foot touches BLUE while already having a shield, THEN nothing happens — the shield does not stack

Students work in pairs to add these rules to their existing conditional block. They must decide where in the order these new rules go — before or after the existing ones — and why order matters.

Walk around and ask: *"If you check the 'already at zero health' rule after the 'lose 10 health' rule, is that too late?"* Yes — if health is already zero, losing 10 more should end the game, not reduce health to negative 10. The order of conditionals matters, just as the order of steps in an algorithm matters.

Teacher tip — managing the energy: The floor activity generates a lot of energy. If the class becomes too loud, ask students to respond by holding up fingers instead of shouting — one finger for red triggered, two for blue, three for green, flat hand for plain floor. This keeps the physical engagement while making it manageable.

No props available — fully paper version: Draw a simple 4x4 grid on the board and colour some squares red, blue, and green. Draw a character at a start position. Students write the path the character takes and annotate each step with the conditional that fires. Exactly the same logic, no floor space needed.

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In Round 2, when I stepped on blue right after red, we disagreed about whether the shield cancelled the damage. In real game programming, who decides the answer to that question?"* Follow-up: *"Is it the computer that decides? Or the programmer? And what happens if the programmer forgets to decide — what does the computer do?"* The answer is: the computer does whatever the code says, even if that produces a nonsensical result. A missing rule is a bug.

Question 2: *"We wrote ELSE IF three times for three different colours. What if there were 100 colours? Would you write 100 ELSE IFs?"* Follow-up: *"Is there a smarter way to handle a large number of conditions?"* Students will often suggest something like a lookup table or a list — they won't have the terminology, but the instinct is right. Tell them there is a structure called a SWITCH statement that handles exactly this problem, and that they will encounter it when they start writing real code.

Question 3: *"Think about the automatic doors example from the start of the lesson. What could go wrong with the IF/THEN/ELSE rule? What edge cases did the programmer probably not think about?"* Follow-up: *"What if two people approach from opposite sides at the same time? What if the sensor detects a shopping trolley but no person? What if the doors open and someone is standing exactly in the middle?"* This introduces the concept of edge cases — situations that the programmer did not anticipate when writing the conditional. Edge cases are one of the most important concepts in software engineering and connect directly to Lesson 5 on AI and bias.

Teacher tip: Question 3 is the best discussion question for this lesson. Edge cases resonate with students because they immediately start generating examples from games they play — *"In Minecraft, what happens if you place a block inside another block?"* or *"In FIFA, what if the ball hits the post at exactly the same time as the final whistle?"* Those are both edge cases in real games. Let the discussion run — it is valuable.

Exercises

Time: 47–57 minutes

Exercise 1 — Write the conditional *Written, no devices needed · 8 minutes · Core exercise*

Students choose one of the following three scenarios and write the full conditional logic in IF/THEN/ELSE format. They should write at least three connected conditions (IF, ELSE IF, ELSE).

Option A — the smart alarm clock: The alarm clock app needs to decide what sound to play in the morning. The user has set three preferences: gentle music on weekdays, a loud buzzer on Monday only, and silence on weekends.

Option B — the school canteen queue: The canteen system needs to decide who gets served first. The rules are: students with a pre-ordered meal go first, then students paying by card, then students paying with cash. If the canteen is out of a meal, the student goes to the back of the queue.

Option C — the game power-up: A game character runs through a level collecting items. Red items add 10 points, gold items double the current score, grey items do nothing, and black items end the game immediately regardless of score.

What the teacher looks for: Students who correctly chain ELSE IF statements rather than writing three separate IF statements. The difference matters: three separate IFs all run independently, meaning multiple conditions could fire at once. A properly chained ELSE IF block stops after the first true condition. Students who identify this distinction without being told are showing strong understanding.

Partner check: Students swap papers and try to find one scenario their partner's conditional does not handle — an edge case. The original writer must then add a rule to cover it.

Exercise 2 — Fix the broken smart home *Whole class or pairs · 7 minutes*

Present this scenario: a smart home system has been installed, but it is behaving strangely. The homeowner reports three problems. Students must read each problem, identify the bug in the conditional logic, and write the fix.

Problem 1: The heating turns on when the temperature drops below 18 degrees — but it never turns off.

The bug: there is no ELSE or ELSE IF to turn the heating off when the temperature rises back above 18 degrees. The fix: add ELSE IF temperature is above 20 degrees THEN turn heating off.

Problem 2: The security light is supposed to turn on when someone approaches the front door after dark. It turns on perfectly during the day, but not at night.

The bug: the conditional checks IF motion detected THEN turn on light — but it does not check whether it is dark. It fires in daylight too. The needed condition is actually: IF motion detected AND it is dark THEN turn on light. The AND operator is required here.

Problem 3: The fridge sends a notification when it is running low on milk. The homeowner is receiving 47 notifications per minute.

The bug: the conditional is inside a forever loop with no delay and no condition to stop sending after the first notification. The fix: add a state variable — once the notification has been sent, set notified = true, and add a condition: IF milk is low AND notified = false THEN send notification AND set notified = true.

Teacher tip: Problem 3 is the hardest and connects back directly to Lesson 2 — a loop without a proper stop condition causes the bug. Students who make this connection independently have understood how conditionals and loops interact, which is a significant milestone. Acknowledge it explicitly if a student spots it.

Extension — Build a smarter traffic light *For fast finishers · 10 minutes*

A standard traffic light uses a simple loop: green for 30 seconds, amber for 5, red for 30, repeat forever. But a smart traffic light uses sensor data and conditionals to decide how long each light stays on.

Task: design the conditional logic for a smart traffic light at a busy junction. The traffic light has four sensors: one counting cars on each of the four roads leading to the junction.

Students must write conditional rules for:

- How long the green light stays on depending on how many cars are waiting
- What happens if an ambulance is detected approaching on any road
- What happens at 3am when there are no cars on any road

Challenge questions:

- *"Can a traffic light be so smart that it causes a traffic jam on a different road while solving one here?"*
- *"If your traffic light talks to the next traffic light down the road, what new rules become possible?"*
- *"Amsterdam has 2,400 traffic lights. If they all ran your algorithm, would the city flow better or worse?"*

These questions introduce the concept of systems thinking — how individual algorithms interact when many of them run together. This connects directly to Lesson 5 on data and Lesson 6 on designing technology for real problems.

Share and reflect

Time: 52–57 minutes

Ask 2–3 students to read one of their conditional rules from Exercise 1 aloud — just one rule, not the whole block. After each one, ask the class: *"Can you think of an edge case that rule doesn't handle?"*

Then ask every student — verbally or on a sticky note — to complete this sentence:

> *"A conditional is like a _____ because _____."*

The metaphor they choose reveals how they have understood the concept. Common good answers: a referee (it checks the rules and makes a call), a traffic light (it controls what happens based on what it detects), a bouncer (it decides who gets in and who doesn't). All of these are correct and worth sharing.

Write the most interesting metaphor on the board next to the exit sentences from Lessons 1 and 2. By Week 6 there will be a small gallery of the class's own definitions of core computing concepts — in their own words.

Close with: *"You now know three of the most fundamental ideas in all of computing — sequences, loops, and conditionals. Almost every program ever written uses all three. Next week we are going to look at something that confused the entire world when it first*

appeared, but that you are now almost ready to understand: what AI actually is, and why it is not as clever as it looks."

Teacher dashboard note — what to observe after the lesson: Did students identify that three separate IFs behave differently from a chained ELSE IF block? Did anyone independently connect Problem 3 in Exercise 2 to the loop bug from last week? Did the traffic light extension prompt students to think beyond the single algorithm to the system it operates in? Those three observations together are a strong signal that the class is building genuine computational thinking — not just following steps, but reasoning about systems. Note it on the dashboard and use it to calibrate the depth of Lesson 4.

Lesson 4

Lesson 4

Memory boxes — how computers remember things

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain A (21A), Domain B (22B) **Builds on:** Lessons 1–3 — students understand sequences, loops, and conditionals

Lesson overview

Learning goal Students can explain what a variable is, why computers need them, and write simple examples showing how a variable's value changes as a program runs. They can connect variables to the conditional and loop concepts from previous lessons.

What you need One small cardboard box or envelope per group — or just draw a labelled box on the board. Sticky notes or small pieces of paper to represent the values inside the box. Whiteboard and marker. Paper and pencils for the exercise.

Key vocabulary to write on the board and leave up all lesson

- Variable — a named container that stores a value which can change
- Value — the thing stored inside the variable at any moment
- Assign — to put a value into a variable
- Update — to change the value stored in a variable
- State — the current snapshot of all variables in a program at one moment

Role model of the day Radia Perlman (born 1951) is a computer scientist and engineer who invented the algorithm that makes the internet stable — called the Spanning Tree Protocol. Her work means that when data travels across the internet, it does not get lost in infinite loops. Her algorithm uses variables to track which paths are open and which are blocked, updating them in real time as the network changes. She is sometimes called "the mother of the internet" — though she finds that title too simple for what she actually did. Today students will use variables the same way she did: to keep track of changing information.

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"When you play a video game and turn it off, how does it remember your score, your level, and where you were when you come back?"*

Ask students to turn to the person next to them and share one idea. Take 3–4 answers. Write the most interesting ones on the board.

Tell them: *"The answer is that every piece of information the game needs to remember is stored in something called a variable — a labelled container that holds a value. Today we are going to open those containers and find out how they work."*

Teacher tip: Students often say "it saves to the cloud" or "it goes to the memory card." Both are correct at a hardware level — acknowledge them. Then redirect: *"But before it saves anywhere, the game needs somewhere to hold that information while it is running. That is what a variable is. It is the scratchpad the program uses while it is thinking."*

Real world connection: When a doctor measures your temperature, they note it down on a chart. That note is a variable — it has a label ("temperature") and a value ("38.2 degrees"). If they measure again an hour later and it has changed, they update the value. The label stays the same. The value changes. That is exactly how a variable works in code.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: The box game

If you have small boxes or envelopes, use them. If not, draw three large labelled boxes on the board and use sticky notes as the values inside.

Label three boxes:

- **SCORE**
- **LIVES**
- **LEVEL**

Write the starting values on sticky notes and place them inside (or write them inside the drawn boxes):

- SCORE = 0
- LIVES = 3
- LEVEL = 1

Tell students: *"These are the variables for a simple game. I am going to describe what happens during the game. Every time I describe an event, you need to tell me which variable changes, what it changes to, and what all three variables look like after the change."*

Read these events one at a time, pausing after each:

1. *"The player collects a coin worth 10 points."* → SCORE changes from 0 to 10. LIVES and LEVEL stay the same.
2. *"The player collects another coin worth 10 points."* → SCORE changes from 10 to 20.
3. *"The player falls into a pit."* → LIVES changes from 3 to 2. SCORE stays at 20.
4. *"The player reaches the end of the level."* → LEVEL changes from 1 to 2. What happens to SCORE and LIVES? Students will disagree — does the score reset? Do

lives reset? Tell them: *"In this game, the programmer decided that score carries over but lives reset to 3 at each new level. How would you write that?"*

5. *"The player falls into another pit."* → LIVES changes from 3 to 2 again.
6. *"The player falls into a pit two more times."* → LIVES goes to 1, then 0. Ask: *"What happens when LIVES reaches 0?"* Students know — game over. *"How would you write that as a conditional using what we learned last week?"*

Teacher tip: The final question — connecting the variable to a conditional — is the key moment of this warm-up. Write the answer on the board together:

```
IF LIVES = 0
THEN game over
ELSE continue
```

This makes the connection between Lesson 3 and Lesson 4 completely explicit. Variables give conditionals something to check. Without variables, most conditionals cannot function.

Phase 3 — Concept introduction

Time: 10–20 minutes

Tell students: *"A variable is like a labelled box with a sticky note inside. The label never changes — that is the variable's name. The sticky note can be replaced at any time — that is the variable's value."*

Draw this on the board:

```
┌──────────────┐
│  SCORE       │ ← the name (never changes)
├──────────────┤
│   20         │ ← the value (can change any time)
└──────────────┘
```

Explain three things variables can hold, using examples the class will recognise:

Numbers — score, health, speed, time remaining, coins collected. These are the most common. You can do maths on them: add 10, subtract 1, multiply by 2.

Text — a player's name, a username, the name of the level. These are called strings. You cannot do maths on them — you cannot multiply "Angela" by 3.

True or False — whether the shield is active, whether the door is locked, whether the player has collected the key. These are Boolean variables, the same Boolean from Lesson 3. The value is always either true or false, nothing else.

Write one example of each on the board:

```
SCORE = 20          (number)
PLAYER_NAME = "Noel"    (text)
SHIELD_ACTIVE = true    (true/false)
```

Now show how a variable connects to a loop from Lesson 2. Write on the board:

```
SCORE = 0

LOOP 5 TIMES:
   add 10 to SCORE
```

What is SCORE after the loop?

Ask students to work it out. The answer is 50. Walk through each iteration: after loop 1 SCORE is 10, after loop 2 it is 20, and so on. This shows that variables change inside loops — which is how games track scores in real time.

Then show how a variable connects to a conditional from Lesson 3:

```
IF SCORE > 100
THEN unlock the bonus level
ELSE keep playing the normal level
```

Tell students: *"This is why variables matter. Without a variable storing the score, the conditional has nothing to check. Sequences give computers their steps. Loops give computers their efficiency. Conditionals give computers their decisions. Variables give computers their memory. All four work together."*

Teacher tip — keep it concrete: For age 8–10, spend more time on the number variable and the box metaphor. For age 10–12, introduce the true/false variable and ask: *"Can you think of five true/false variables a game might need?"* Common strong answers: HAS_KEY, DOOR_LOCKED, GAME_OVER, SOUND_ON, LEVEL_COMPLETE. Every one of those is a real variable in a real game.

Real world connection: Ask students: *"When you use a streaming app, how does it know you have already watched something?"* There is a variable — probably called something like WATCHED — that is set to true the moment you finish watching. The little tick that appears on a completed show is just the program reading that variable and displaying it. Every tick on every show you have ever watched is a true/false variable stored somewhere.

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The arcade scoreboard

This activity is physical, fast-paced, and ties variables directly to something students care about — keeping score.

Setup (2 minutes) Place a wastepaper bin at the front of the room. Scrunch up 10 pieces of paper into balls — these are the "shots." If you have no bin, mark an X on the floor with tape as the target zone.

Each group draws three boxes on their paper and labels them:

SCORE = 0
SHOTS_LEFT = 10
MULTIPLIER = 1

Tell students: *"I am going to take 10 shots at the bin. Your job is to keep track of all three variables after every single shot. You need to update them correctly and instantly. At the end, every group should have the same final values — if they don't, someone made a variable error."*

The rules:

- Every shot that goes in: add 10 to SCORE, then multiply by MULTIPLIER
- Every shot that misses: SCORE does not change
- After every 3 successful shots in a row: MULTIPLIER increases by 1 (so after 3 in a row, MULTIPLIER = 2, meaning each basket is now worth 20)
- SHOTS_LEFT decreases by 1 after every shot, whether it goes in or not

Take the shots at a steady pace. Make some on purpose and miss some on purpose. Increase speed after the fifth shot. Students who are keeping accurate variables will start to fall behind — that is intentional. It shows that computers need to update variables instantly and accurately, and that a single missed update corrupts all the data that follows.

After the shots: Ask each group to call out their final SCORE. If any groups have different values, work backwards together to find where the divergence happened. This is a live debugging session — finding the exact moment a variable was updated incorrectly.

Round 2: Add a conditional (7 minutes) Tell students: *"Now I want to add a rule. Before each shot, check this conditional:"*

IF SHOTS_LEFT is an even number
THEN the shot is worth double points if it goes in
ELSE the shot is worth normal points

Groups must now track an extra condition per shot. Run 5 more shots. Walk around and check whether groups are evaluating the conditional correctly before updating the score.

Teacher tip: The multiplier rule deliberately makes the maths slightly complex. For age 8–10, simplify: remove the multiplier and just track SCORE and SHOTS_LEFT. For age 10–12, keep the multiplier and add the Round 2 conditional. The same physical activity works for both groups — just adjust which variables they are tracking.

No props version: Replace the shooting game with a dice-roll activity. Roll a dice (or simulate one by asking a student to call out a number between 1 and 6). Each number maps to a different event: 1 = lose a life, 2 = gain 10 points, 3 = level up, 4 = nothing, 5 = double the score, 6 = gain an extra life. Students track all variables as the dice is "rolled" 10 times.

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In the game from the warm-up, the programmer decided that LIVES resets to 3 when you reach a new level, but SCORE carries over. What would change about the game experience if it was the other way around — SCORE resets but LIVES carries over?"* Follow-up: *"Which version is more fun? Which is fairer? Who gets to decide — the programmer or the player?"*

Question 2: *"We had three variables: SCORE, SHOTS_LEFT, and MULTIPLIER. Which one was the most important? What would break if you removed each one?"* Follow-up: *"Is a variable that is always 1 and never changes still a variable? What would you call it instead?"* (The answer is a constant — a named value that never changes. Students don't need the term, but the concept is worth planting.)

Question 3: *"Think about a variable called MOOD in a social media app. What value would it have? Who would update it? Who would read it?"* Follow-up: *"Do you think apps that track how you are feeling — what you click, how long you pause on something, whether you scroll past quickly — are storing variables about you without telling you? What should the rules be about that?"*

Teacher tip: Question 3 is designed for Groep 7–8 and is the first moment in the program where students encounter the ethical dimension of data. It connects directly to Lesson 5 on data and Lesson 6 on designing technology responsibly. For Groep 5–6, replace it with: *"If a robot pet had variables for HUNGER, HAPPINESS, and ENERGY — what rules would you write to make it feel real?"*

Exercises

Time: 47–57 minutes

Exercise 1 — Track the variables *Written, no devices needed · 8 minutes · Core exercise*

Present this short program on the board. Students must read through it step by step and write down the value of each variable after every line.

```
HEALTH = 100
SHIELD = false
POTIONS = 3

Step 1: the character walks into fire
→ subtract 20 from HEALTH

Step 2: the character picks up a shield
→ set SHIELD to true

Step 3: the character walks into fire again
→ IF SHIELD = true THEN subtract 0 from HEALTH
  ELSE subtract 20 from HEALTH

Step 4: the character uses a potion
→ add 30 to HEALTH
```

→ subtract 1 from POTIONS

Step 5: the character walks into fire again
→ IF SHIELD = true THEN subtract 0 from HEALTH
  ELSE subtract 20 from HEALTH

Students write the values of HEALTH, SHIELD, and POTIONS after each step. The correct final state is:

- After Step 1: HEALTH = 80, SHIELD = false, POTIONS = 3
- After Step 2: HEALTH = 80, SHIELD = true, POTIONS = 3
- After Step 3: HEALTH = 80 (shield blocked damage), SHIELD = true, POTIONS = 3
- After Step 4: HEALTH = 110, SHIELD = true, POTIONS = 2
- After Step 5: HEALTH = 110 (shield still active), SHIELD = true, POTIONS = 2

What the teacher looks for: Students who correctly apply the conditional in Steps 3 and 5 — recognising that the shield variable makes the THEN branch fire, not the ELSE branch. Students who make an error in Step 4 (forgetting to subtract from POTIONS, or adding 30 to the wrong starting value) have a variable tracking error — the most common real programming mistake.

Common mistake to watch for: Some students will reduce HEALTH in Steps 3 and 5 despite the shield being active. If this happens, ask: *"What is the value of SHIELD at the moment you reach Step 3? If SHIELD is true, which branch of the conditional fires?"* Usually this resolves it immediately.

Partner check: Students swap papers and check every single value. If they find a disagreement, they must agree on the correct answer before moving on — not just mark it wrong.

Exercise 2 — Design the virtual pet *Pairs · 7 minutes*

Working in pairs, students design the variable system for a simple virtual pet. They must:

Define exactly three variables with starting values. For example:

- HUNGER = 5 (scale of 0–10, where 10 is very hungry)
- HAPPINESS = 8 (scale of 0–10)
- ENERGY = 10 (scale of 0–10)

Write three events that change the variables. For example:

- The pet eats a meal → HUNGER decreases by 3
- The pet plays a game → HAPPINESS increases by 2, ENERGY decreases by 2
- The pet sleeps → ENERGY resets to 10, HAPPINESS decreases by 1 (it is bored while sleeping)

Write two conditionals that use the variables. For example:

- IF HUNGER > 8 THEN the pet cries

- IF ENERGY = 0 THEN the pet falls asleep and cannot play until ENERGY is restored

What the teacher looks for: Students who make the variables interact — where one event changes multiple variables, or where one variable affects a conditional that triggers a change in another variable. This interconnection is what makes virtual pet systems feel alive. A pet where every variable is independent feels flat. A pet where feeding affects both HUNGER and HAPPINESS, and playing affects both HAPPINESS and ENERGY, starts to feel like a real system.

Extension — The broken vending machine *For fast finishers · 10 minutes*

A vending machine has these variables:

STOCK_CRISPS = 5
STOCK_WATER = 0
STOCK_CHOCOLATE = 3
COINS_INSERTED = 0
CHANGE_DUE = 0

A customer walks up and inserts €2.00. A bag of crisps costs €1.50, water costs €1.00, chocolate costs €1.80.

The customer selects crisps. Write out every variable update that should happen, step by step. Then answer these questions:

- What should the machine do if STOCK_WATER = 0 and the customer selects water?
- What if the customer inserts €2.00 but the machine has no change — no coins to give back?
- What if the customer selects chocolate but has only inserted €1.50?
- After three customers, what do all five variables look like?

This exercise combines variables, conditionals, and edge case thinking. It is a scaled-down version of real transaction software. Every payment terminal, every online shop, every ticket machine runs a version of this logic.

Challenge question: *"What would happen if the COINS_INSERTED variable was never reset to 0 after a purchase? Walk through what the next customer's experience would be."* This is a real class of software bug called a state persistence error — data from one session bleeding into the next. Students who identify it and describe the consequences are thinking at a genuinely high level.

Share and reflect

Time: 52–57 minutes

Ask 2–3 pairs to share one variable from their virtual pet and one conditional they wrote for it. After each, ask the class: *"Can you find a situation that conditional doesn't handle?"*

Then ask every student — verbally or on a sticky note — to complete this sentence:

"A variable is like a _____ because _____."

Expect answers like: a scoreboard, a name tag, a locker, a sticky note on the fridge. All of these are good. Write the most interesting one on the board next to the exit sentences from the previous three lessons. The wall is now building into a class-written glossary of computing in their own words.

Close with: *"You now have four building blocks: sequences, loops, conditionals, and variables. Next week we use all four of them to understand something that confuses most adults: what AI actually is, how it learns, and why it makes mistakes. It is not magic. It is patterns — and you are already very good at spotting those."*

Teacher dashboard note — what to observe after the lesson: Did students correctly track variables through the conditional branches in Exercise 1 — especially in Steps 3 and 5 where the shield is active? Did any pairs in Exercise 2 create variables that interact with each other rather than operating independently? Did any students in the extension identify the state persistence bug without being told what to look for? These three signals together indicate students are developing genuine systems thinking — understanding not just individual concepts but how they combine. This is the foundation the AI lesson builds directly on.

Lessons 5

Lesson 5

Teaching the machine — what AI actually is

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain A (21A, 21D), Domain B (22B), Domain C (23A) **Builds on:** All previous lessons — sequences, loops, conditionals, variables all appear in this lesson

Lesson overview

Learning goal Students can explain in their own words that AI learns from examples rather than following fixed instructions, identify what training data is and why it matters, and describe at least one way AI can make a mistake and why that happens.

What you need A set of picture cards — print or hand-draw 12–16 small pictures of mixed objects, animals, and people. Suggestions: dog, cat, horse, bird, fish, apple, banana, chair, car, bicycle, person running, person sitting, tree, cloud. Whiteboard and marker. Paper and pencils. Sticky notes for the exit reflection.

Key vocabulary to write on the board and leave up all lesson

- Artificial Intelligence (AI) — a program that learns patterns from examples rather than following fixed instructions
- Training data — the examples the AI learns from
- Pattern — a rule or regularity that the AI identifies across many examples
- Bias — when the training data is incomplete or unbalanced, causing the AI to make unfair or wrong decisions
- Prediction — the AI's best guess based on patterns it has seen before

Role model of the day Fei-Fei Li (born 1976) is a computer scientist and professor at Stanford University who built ImageNet — a database of over 14 million labelled images that became the foundation for most modern AI image recognition. Before ImageNet, AI could not reliably identify a cat in a photo. After training on Fei-Fei Li's data, AI could. She showed that the secret to better AI was not a smarter algorithm — it was better, more diverse training data. Today students will experience exactly what she discovered: the data you train on determines what the AI can and cannot do.

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"Your phone can unlock by recognising your face. It has never taken a class in faces. Nobody programmed it with a list of rules about what your nose looks like. So how does it know it is you?"*

Ask students to turn to the person next to them and share one idea. Take 3–4 answers. Write the most interesting ones on the board.

Tell them: *"The answer is not that it is clever. The answer is that it has seen thousands of photos of your face from different angles, in different lighting, with different expressions — and it found the pattern. Today we are going to find out exactly how that works, and more importantly, what happens when the pattern is wrong."*

Teacher tip: Students often say "it memorised your face" or "it scanned you." Both point in the right direction — acknowledge them and then redirect toward the word pattern. The distinction that matters is: a scanner follows rules (measure this, check that), while AI finds its own rules from examples. That difference is the conceptual core of this lesson.

Real world connection: Ask students: *"Has an app ever recommended something to you that was surprisingly right — like it somehow knew exactly what you wanted?"* And then: *"Has it ever recommended something completely wrong — like it clearly did not understand you at all?"* Both happen because of patterns in training data. The good recommendations come from patterns the AI found correctly. The bad ones come from patterns that were incomplete, outdated, or just wrong. Today students will experience both.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: You are the AI

This activity is drawn directly from the Dream Space Unplugged Explorers document and from the human AI sorting game discussed in the earlier curriculum planning. It is the single most effective way to make training data concrete for this age group.

Tell students: *"I have a secret rule in my head. I am going to show you some cards one at a time and say either YES or NO. Your job is to figure out my rule — just like an AI figuring out a pattern from training data. Do not shout out your guess. Write it down when you think you know."*

Hold up picture cards one at a time and say YES or NO according to your secret rule. Choose a rule students can actually figure out within 8–10 cards. Good rules for this age group:

- YES to all animals, NO to all objects
- YES to things that can fly, NO to things that cannot
- YES to things with four legs, NO to things without four legs
- YES to things found outside, NO to things found inside

After 8–10 cards, ask: *"How many of you think you know the rule? Don't say it yet — write it down."* Show 2–3 more cards to confirm or challenge their hypothesis. Then ask for the rule.

Take 3–4 guesses. Reveal the actual rule. Then ask: *"How many examples did you need before you were confident? Did anyone change their guess after seeing a new card that didn't fit?"*

Teacher tip: Choose a rule that has at least one surprising card — for example, if your rule is "things with four legs," include a table or a chair in the set. When students see a chair get a YES, their existing hypothesis (animals) breaks and they have to revise it. That moment of revision is exactly what AI does when it encounters new training data that challenges its current model. Name that moment explicitly: *"What just happened to your theory when you saw the chair? That is called updating the model — and AI does exactly that every time it sees a new example."*

Follow-up question: *"What if I had only shown you the first three cards before asking for the rule? Would you have been as confident?"* No — and that is the point. More training data means more confidence. But it also means: what if those first three cards were all very similar? What patterns might you miss?

Phase 3 — Concept introduction

Time: 10–20 minutes

Tell students: *"For the last four weeks, every program we wrote followed fixed instructions. A sequence runs in order. A loop repeats. A conditional checks a rule. A variable stores a value. All of that is traditional programming — a human writes the rules and the computer follows them."*

Write on the board:

TRADITIONAL PROGRAMMING:
Human writes rules → Computer follows rules → Output

"AI is different. In AI, the human does not write the rules. Instead, the human gives the computer thousands of examples — and the computer figures out the rules itself."

Write underneath:

MACHINE LEARNING (a type of AI):
Human gives examples → Computer finds rules → Output

Tell students: *"The examples are called training data. The rules the computer finds are called the model. And once the model is trained, it can make predictions about things it has never seen before."*

Give this concrete example: *"Imagine you want to teach an AI to recognise cats in photos. You do not write: 'a cat has pointy ears, whiskers, and fur.' Instead, you show it one million photos of cats labelled 'cat' and one million photos of things that are not cats labelled 'not*

cat.' The AI looks at all of those examples and finds the patterns — pointy ears, whiskers, fur — itself. Then when you show it a new photo it has never seen, it predicts: cat or not cat."

Ask: *"What is the AI actually doing when it makes that prediction?"* It is checking whether the new photo matches the patterns it found in the training data. It is running a very complex conditional — but one it wrote itself, not one a human wrote for it.

Now introduce bias with a simple example. Tell students: *"Imagine you trained the same cat AI, but all one million training photos were taken in sunny California in the daytime. Then someone shows it a photo of a cat sitting in the dark, or a cat in the snow, or a cat photographed from above. What do you think happens?"*

Students will usually guess: it gets confused or gets it wrong. Tell them: *"That is called AI bias. The AI did nothing wrong — it learned exactly what it was shown. But what it was shown was not the full picture. And now it makes mistakes in situations it was never trained for."*

Write on the board:

BETTER TRAINING DATA → BETTER AI
INCOMPLETE TRAINING DATA → BIASED AI

Teacher tip — connecting to the warm-up: Point back to the card activity. *"When you were guessing my rule, you were acting like an AI being trained. When a new card changed your guess, you were updating your model. And if I had only shown you cards from one category — say, only outdoor animals — you might have guessed the wrong rule entirely. That is training data bias happening in real time."*

Real world connection: Fei-Fei Li's insight was that AI image recognition was bad not because the algorithms were weak, but because the training data was too small and too narrow. When she built ImageNet with 14 million diverse images, AI accuracy jumped from roughly 70% to over 95% in just a few years. The data was the breakthrough, not the algorithm. This is one of the most important ideas in all of modern technology.

Phase 4 — The challenge

Time: 20–42 minutes

Activity: Train the class AI

This activity combines the YouTube Recommender and self-driving car activities from the Unplugged Explorers document into a single cohesive challenge suited to the 45-minute format and age range.

Round 1: Training the recommender (10 minutes)

Tell students: *"You are going to train a class AI to predict what kind of content a student likes to watch. I am the student. You are the AI — and you start knowing nothing about me."*

Draw two columns on the board:

WATCH ✓ | SKIP ✗

Read out 12 content titles one at a time and sort them yourself into WATCH or SKIP without explaining why. Use a mix that has a hidden pattern students can find:

- Minecraft Let's Play — WATCH
- A cooking tutorial — SKIP
- Dog learns to skateboard — WATCH
- Football match highlights — SKIP
- Cat knocks things off a table — WATCH
- How to bake a cake — SKIP
- Goat climbs a tree — WATCH
- Basketball trick shots — SKIP
- Parrot sings a song — WATCH
- Travel vlog in Japan — SKIP
- Hamster runs a maze — WATCH
- Science experiment explosion — SKIP

After all 12 are sorted, ask pairs to study the pattern and write their prediction of the rule. Take answers — the correct rule is "animals doing funny or unusual things."

Then say: *"Now I show you three new titles you have never seen before. Predict where they go."*

- Horse learns to open a gate
- How to make pizza
- Penguin steals a fish from a market

Students predict: WATCH, SKIP, WATCH. They are correct. Tell them: *"You just did what a recommendation AI does. You found the pattern in the training data and used it to make predictions about new content."*

Round 2: Find the bias (8 minutes)

Now complicate it. Tell students: *"Here is a new title: 'Man runs a marathon in world record time.' WATCH or SKIP?"*

Students will probably say SKIP — no animal. Confirm: SKIP.

Then say: *"A year later, the student gets a dog and starts running with it. Now 'Man runs a marathon' videos are suddenly interesting. But the AI still predicts SKIP — because the AI's training data is from a year ago."*

Ask: *"What is the problem here?"* The training data is outdated. The student changed, but the AI's model did not update.

Then introduce a second bias scenario. Tell students: *"The same AI is used for all students in a school. But the training data came only from 11-year-old boys. A 9-year-old girl starts using the app. What might go wrong?"*

Students will suggest: the recommendations might not match what she likes. Tell them: *"That is demographic bias — the training data did not represent all the different types of people who would use the system."*

Ask pairs to write down two things that would make this AI better. Most will suggest: more diverse training data, more recent data, letting users correct wrong recommendations. All of those are real techniques used by actual recommendation systems.

Round 3: The hard question (5 minutes)

Tell students: *"The AI on a social media app has been trained to maximise watch time — to show you whatever keeps you watching the longest. It has found a pattern: angry or shocking content keeps people watching longer than calm or happy content. So it recommends more angry and shocking content. Is that a bias? Is it a bug? Or is it working exactly as intended?"*

Do not answer this. Write the responses on the board. Tell students this is one of the most important debates happening in technology right now — and they have just understood it well enough to have an opinion.

Teacher tip: Round 3 is optional for Groep 5–6 and recommended for Groep 7–8. If the class is young or time is tight, skip it and spend the extra five minutes on the discussion phase. The first two rounds are sufficient to meet the learning goal.

No props version: Replace the picture cards in the warm-up with titles written on the board. Replace the content sorting in Round 1 with a simple list projected or written up. The entire activity works without any printed materials.

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In Round 1 you figured out my watching pattern from 12 examples. How many examples do you think a real YouTube AI uses?"* Follow-up: *"If it uses billions of examples, does that mean it is always right? What could it still get wrong even with billions of examples?"*

Question 2: *"We said AI finds the rules itself from the data. Does that mean nobody is responsible when the AI makes a mistake? Or is someone still responsible — and if so, who?"* Follow-up: *"The company that built it? The people who collected the training data? The people who decided what the AI was trying to predict? All three?"* This is a genuine open question in technology ethics and law. Students should argue their own position.

Question 3: *"Think about an AI that helps decide whether a student should get extra support in school — based on their past test scores, attendance, and behaviour. What could go wrong with the training data for that AI?"* Follow-up: *"If the training data came mostly from schools in wealthy neighbourhoods, what patterns might it learn that don't apply to students in other schools? What would that mean for a student it gets wrong?"*

Teacher tip: Question 3 is the most important and the most challenging. For Groep 5–6, replace it with: *"If an AI was trained only on photos of cats from Europe, would it recognise a cat in Brazil? Why or why not?"* The concept is identical but the stakes feel lower and the example is more concrete. For Groep 7–8, the school support AI example connects directly to their own lives and tends to generate strong opinions.

Exercises

Time: 47–57 minutes

Exercise 1 — Spot the bias *Written, no devices needed · 8 minutes · Core exercise*

Present three AI systems and their training data descriptions. Students must identify the bias in each one and suggest how to fix it.

AI system 1 — a face recognition system for a phone unlock feature. Training data: 500,000 photos of faces, collected from one university in the Netherlands. 80% of the photos are of people aged 18–25.

- What is the bias? The training data does not represent older people, children, or people from diverse backgrounds. The AI will likely perform worse for anyone who does not look like a Dutch university student.
- How would you fix it? Collect training data from more diverse sources — different ages, ethnicities, locations, and lighting conditions.

AI system 2 — a spelling correction AI for a school writing app. Training data: 10 million sentences from Dutch newspapers published between 1990 and 2005.

- What is the bias? The training data is old — language changes, new words appear, and the way people write informally (in messages, on social media) is very different from newspaper writing. The AI will probably flag new words and informal language as errors when they are not.
- How would you fix it? Update the training data to include recent text, including informal writing styles appropriate for the age group using the app.

AI system 3 — an AI that predicts which students are likely to struggle in secondary school. Training data: five years of data from 10 primary schools in Amsterdam, all in the same wealthy neighbourhood.

- What is the bias? The training data comes from a specific demographic. Patterns that predict struggle in that neighbourhood may be completely different from patterns in other neighbourhoods. The AI may make systematically wrong predictions for students from different backgrounds.
- How would you fix it? Collect training data from schools across many different neighbourhoods, regions, and demographic groups. Consider whether this kind of AI should exist at all — and who benefits and who is harmed if it gets predictions wrong.

What the teacher looks for: Students who can name the specific problem in each case — not just "the data is bad" but which dimension is wrong (age, time, location, demographic).

Students who engage with the third example and question whether the AI should exist at all are showing the ethical reasoning the program is designed to build.

Exercise 2 — Design the training data *Pairs · 7 minutes*

Tell students: *"You are going to build a simple AI that can tell the difference between a sunny day and a rainy day based on what students wear to school."*

Working in pairs, students must answer these questions in writing:

What training data would you collect? Name at least five things you would look for and record about what students are wearing.

How many examples would you need to be confident the AI works? Give a number and explain why.

Name two types of student the training data might not represent well — and what the AI might get wrong as a result.

Write one rule the AI might learn from the data. Write it in IF/THEN format using what students learned in Lesson 3.

What the teacher looks for: Students who identify specific gaps in their own training data proposal — for example, realising that a student who wears a coat every day regardless of weather would confuse the AI. Students who connect their IF/THEN rule back to Lesson 3 vocabulary are demonstrating that the four concepts are integrating in their thinking.

Extension — The fairness test *For fast finishers · 10 minutes*

Present this scenario: a hospital uses an AI to help decide which patients should be prioritised for a medical appointment. The AI was trained on 10 years of hospital records.

Students must answer these questions:

- What patterns might the AI have learned that are useful — things that genuinely predict which patients need help most urgently?
- What patterns might it have learned that are unfair — things that predict who got prioritised in the past, not who most needs help?
- If the hospital's records from 10 years ago show that certain groups of patients were seen faster than others, what will the AI learn from that?
- Should this AI be used? If yes, what safeguards would you put in place? If no, what would you use instead?

There is no correct answer to the final question. Students who argue both sides with reasons are demonstrating exactly the kind of ethical reasoning the Dream Space 21CLD framework identifies as a core future-ready skill. If a student writes "it depends" and then explains what it depends on, that is a strong answer.

Share and reflect

Time: 52–57 minutes

Ask 2–3 pairs to share one bias they identified in Exercise 1 and their suggested fix. After each, ask: *"Is the fix enough? Could the fixed version still have a bias?"*

Then ask every student — verbally or on a sticky note — to complete this sentence:

> *"AI is not magic — it is just _____."*

Collect the responses. The best answers will include words like: patterns, data, examples, guessing, trained, learning from mistakes. Write the most precise and interesting one on the board next to the previous exit sentences.

The board now has five statements — one from each lesson — written entirely in the students' own words. Together they form a complete, student-authored definition of the foundations of computing.

Close with: *"You now understand five of the most important ideas in technology: how computers follow instructions, how they repeat efficiently, how they make decisions, how they remember things, and how AI learns from examples. Next lesson we are going to use all of that — because you are going to design something real."*

Teacher dashboard note — what to observe after the lesson: Did students identify specific dimensions of bias in Exercise 1 — age, time, location, demographic — rather than just saying "the data is bad"? Did any pairs in Exercise 2 identify a gap in their own training data proposal without being prompted? Did any students in the extension question whether the AI should exist at all, rather than just trying to fix it? That last signal is the most significant — it indicates a student who is thinking about technology as a social and ethical choice, not just a technical problem. Note those students by name on the dashboard. They are the ones to give the most open-ended extension tasks in Lesson 6.

Lesson 6

Lesson 6

Secrets and codes — how computers keep information safe

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain A (21A), Domain C (23A) **Builds on:** Lesson 1 (algorithms), Lesson 3 (conditionals), Lesson 4 (variables)

Lesson overview

Learning goal Students can explain why encryption exists, apply a simple cipher to encode and decode a message, and describe in their own words what makes a password strong versus weak.

What you need Paper and pencils for every student. Whiteboard and marker. Optional: one envelope per group for the message-passing activity.

Key vocabulary to write on the board and leave up all lesson

- Encryption — scrambling information so only the intended person can read it
- Cipher — the rule or algorithm used to encrypt a message
- Key — the piece of information needed to decrypt a message
- Decrypt — unscramble an encrypted message back to its original form
- Hacker — someone who tries to access information they are not supposed to see

Role model of the day Hedy Lamarr (1914–2000) was a film actress and inventor who co-invented a radio communication system during World War II that changed signals rapidly so enemies could not intercept them. Her invention — called frequency hopping — was ignored during her lifetime but became the foundation for modern Wi-Fi, Bluetooth, and GPS encryption. She received no credit for decades. Today every phone call, every Wi-Fi connection, and every Bluetooth device uses a descendant of her idea. Today students will encrypt their own messages using a cipher — the same basic principle she applied to radio signals eighty years ago.

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"When you send a message to a friend on WhatsApp, it travels through dozens of computers before it arrives. Any of those computers could read it. So how does WhatsApp make sure nobody except your friend can understand it?"*

Ask students to turn to the person next to them and share one idea. Take 3–4 answers. Write the most interesting ones on the board.

Tell them: *"The answer is encryption — scrambling the message in a way that only the person with the right key can unscramble it. Today we are going to encrypt messages ourselves, try to hack each other's messages, and find out why some codes are much harder to break than others."*

Teacher tip: Some students will know the term end-to-end encryption — acknowledge it and tell them that is exactly what they are learning about today. The fact that they have heard the term but may not know what it means is a perfect entry point: today they will understand it properly.

Real world connection: Every time you see a small padlock in your browser's address bar, that means the website is using encryption. Every message you send on WhatsApp or iMessage is encrypted before it leaves your phone and only decrypted when it arrives on your friend's device. Even the people who run WhatsApp cannot read your messages. That padlock and that privacy are both made possible by the same mathematical idea students will work with today.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: Can you crack it?

Write these three encrypted words on the board one at a time. Tell students they are all encrypted using the same rule — and their job is to figure out the rule before working out what the words say.

KHOOR
ZRUOG
FRGH

Give students 2 minutes in pairs to work out the rule. Most will not crack it in time — and that is fine. After 2 minutes, give a hint: *"The first word is a greeting."*

Most pairs will now get it: KHOOR = HELLO. The rule is a Caesar cipher — each letter has been shifted 3 positions forward in the alphabet. H→K, E→H, L→O, L→O, O→R.

Confirm: ZRUOG = WORLD, FRGH = CODE.

Ask: *"How did knowing the first word was a greeting help you crack the code faster?"* That is called a known-plaintext attack — one of the most common ways real hackers crack encryption. If you know what the message is supposed to say, you can work backwards to find the rule.

Write the alphabet shifted by 3 on the board:

Plain: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Encrypted: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Teacher tip: Some students will crack it immediately — give them a harder challenge: *"The same rule was used, but the shift is not 3. Can you crack AOPZ?"* (Answer: shift of 7, the word is THIS.) This keeps fast finishers engaged while the rest of the class works on the main activity.

Phase 3 — Concept introduction

Time: 10–20 minutes

Tell students: *"The Caesar cipher is one of the oldest encryption algorithms in the world — Julius Caesar used it to send military messages over 2,000 years ago. The shift amount is the key. Without the key, you have to try every possible shift — that is 25 possibilities for a basic alphabet. A human would take a few minutes. A computer could try all 25 in less than a millisecond."*

Write on the board:

MESSAGE + KEY = ENCRYPTED MESSAGE
ENCRYPTED MESSAGE + KEY = ORIGINAL MESSAGE

Explain that this is the same structure as an algorithm from Lesson 1 — input, process, output. The input is the original message. The process is the cipher rule. The output is the encrypted message. To reverse it, you run the algorithm backwards using the same key.

Ask: *"If the Caesar cipher is so easy to crack, why do we still use encryption today?"* Because modern encryption uses mathematical problems that are much harder than shifting letters — problems so hard that even the fastest computer in the world would take longer than the age of the universe to crack them by brute force. The principle is the same. The maths is much harder.

Now introduce passwords. Write on the board:

"ninja123" → a computer can crack this in 0.5 seconds
"Tr0ub4dor&3" → a computer would take centuries

Ask: *"What makes the second one harder to crack?"* Students will say: it has symbols, numbers, and mixed capital letters. Tell them: *"All of those things increase the number of possible combinations the computer has to try. The longer and more varied the password, the longer it takes to crack — which is why length is actually more important than complexity."*

Introduce the passphrase method. Tell students: *"The easiest way to create a strong password is to take a sentence you can remember and use the first letter of each word, mixing in the numbers and symbols naturally:"*

Write on the board:

"I love playing football on Tuesdays with 5 friends!"

→ IlpfoTw5f!

Tell students: *"That password has 10 characters, a mix of upper and lower case, a number, and a symbol — and it is easy to remember because the sentence is meaningful to you. A computer would need hundreds of years to crack it by brute force."*

Teacher tip — connecting to variables: Ask: *"If a computer is trying to crack a password by trying every possibility, what variable is it updating with every attempt?"* Students will suggest something like ATTEMPTS or CURRENT_GUESS. Tell them: *"Exactly. Brute-force hacking is just a loop — try one password, check if it works, if not try the next one. The only thing stopping it is how many possible passwords there are. More length and variety means more possible combinations means more iterations of the loop before it finds the right one."* This connects directly to Lessons 2 and 4.

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The hacker intercept game

This is the core activity — a physical message-passing and code-cracking game. It is drawn from the Unplugged Explorers cybersecurity session and adapted for the 45-minute single-session format.

Setup (2 minutes) Organise the class into groups of three. Each group is a team of secret agents. Tell them: *"Your team needs to send a secret 3-word message to another team across the room. But I am a hacker sitting in the middle. If I intercept your message and can read it, your mission fails. You need to encrypt your message before you send it — and your contact team needs to be able to decrypt it when they receive it."*

Round 1: Simple cipher (8 minutes)

All teams use the Caesar cipher with a shift of 4. Write the shifted alphabet on the board:

Plain: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Encrypted: E F G H I J K L M N O P Q R S T U V W X Y Z A B C D

Each team:

1. Chooses a secret 3-word message (can be anything — "meet at lunch," "the code is blue," "pizza for dinner")
2. Encrypts it using the Caesar +4 shift
3. Writes the encrypted message on a piece of paper and folds it
4. Passes it physically to the teacher (the hacker) who then passes it to another team

The teacher reads the message aloud before passing it. Since everyone is using the same cipher and the alphabet is on the board, the teacher can read every message — and so can every team. The mission fails for everyone.

Ask: *"Why did this fail?"* Because everyone was using the same key and it was written on the board. Encryption only works if the key is secret.

Round 2: Secret key (10 minutes)

This time, each team chooses their own shift number (between 1 and 25) and shares it only with the team they are sending to — by whispering it, not writing it. The teacher does not know any team's key.

Teams encrypt a new 3-word message using their chosen shift. Pass messages via the teacher as before. The teacher tries to crack each message — but without knowing the shift, has to try multiple options. With only 3 words and no hint about the content, this is genuinely difficult in 30 seconds.

After all messages have been passed, receiving teams decrypt their message and confirm whether it arrived correctly.

Ask: *"What made Round 2 harder to hack than Round 1?"* The key was secret and different for every team. *"What would make it even harder?"* Longer messages, more complex cipher rules, changing the key for every message.

Round 3: The known-plaintext attack (7 minutes)

Tell students: *"In Round 2, the hacker (me) knew that all messages were 3 words. I also know that common 3-word English messages often start with 'meet' or 'the.' If I assume the first encrypted word decrypts to 'meet,' I can work out the shift and then decrypt the whole message. That is a known-plaintext attack — and it is exactly how many real ciphers have been cracked throughout history."*

Show this live: take one team's encrypted message from Round 2. Tell the class the first word of the original message (ask the sending team to confirm). Work backwards to find the shift, then decrypt the whole message.

Ask: *"How could you protect against a known-plaintext attack?"* Students will suggest: do not start messages with predictable words, use random padding (extra meaningless characters), or use a more complex cipher. All three are real techniques used in modern cryptography.

Teacher tip: Round 3 is conceptually the most interesting but also the most teacher-led. If time is short, skip it and use the time for the discussion. Rounds 1 and 2 are sufficient to meet the learning goal. Round 3 is the extension for classes that are moving quickly.

No props version: All three rounds work equally well with messages written on the board rather than passed physically. The teacher writes the encrypted message publicly and tries to crack it in real time while the class watches. The physical passing adds energy but is not essential to the learning.

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In Round 1, the encryption failed because the key was public. In Round 2, it worked better because the key was secret. But how did the two teams agree on the key without the hacker finding out? In real life, how do two computers agree on an encryption key when they have never communicated before and the hacker might be listening to everything?"* Follow-up: *"This is one of the hardest problems in all of cryptography. It was only solved in the 1970s — and the solution changed the entire internet. You do not need to know the solution today, but the problem is worth thinking about."* (The solution is public-key cryptography — worth naming without explaining, as a hook for curious students to explore independently.)

Question 2: *"The Caesar cipher has 25 possible keys. A modern password with 10 characters using letters, numbers, and symbols has over 59 trillion possible combinations. Is 59 trillion enough? How fast do you think modern computers can try passwords?"* Follow-up: *"A standard laptop can try about 1 billion passwords per second. At that rate, how long would it take to crack 59 trillion combinations?"* Students should work this out: 59 trillion divided by 1 billion = 59,000 seconds = about 16 hours. *"Is that long enough? What if the hacker used 1,000 computers at once?"* This makes the abstract idea of password strength completely concrete.

Question 3: *"Should the police be allowed to break into encrypted messages if they think someone is planning something dangerous? Or should everyone's messages be private, no exceptions?"* Follow-up: *"Is there a difference between being able to break encryption and choosing to? Who should make that decision?"* There is no correct answer. This is one of the most active debates in technology law and policy. Students who can articulate both sides are thinking exactly the way the Dream Space 21CLD framework asks them to.

Teacher tip: Question 3 generates strong opinions quickly — especially in Groep 7–8. Give every student a moment to write their initial position before the discussion starts, so quieter students have something to say rather than waiting for louder voices. The written note also lets you see where every student stands, not just the ones who speak.

Exercises

Time: 47–57 minutes

Exercise 1 — Encrypt and decrypt *Written, no devices needed · 8 minutes · Core exercise*

Students work individually on three tasks.

Task A — Encrypt this message using a Caesar cipher with a shift of 6: *DREAM SPACE IS FUN*

Answer: JXKGS YVEGI OY LAT

Students check their work by decrypting it back. If the decryption does not produce the original message, they find their error.

Task B — Crack this message. The shift is unknown but between 1 and 25. The original message is a common English greeting:

YJQQT BTWQI

Students must work out the shift and decrypt it. Hint: the message is two words. Answer: shift of 17, message is HELLO WORLD.

Task C — Design a better cipher. The Caesar cipher only shifts letters. Design a cipher that also changes the order of the words or adds fake letters between real ones. Write the rule clearly enough that a partner could use it to encrypt a message.

What the teacher looks for: Students who complete Task A correctly by working systematically through the alphabet rather than guessing. Students who identify the greeting in Task B using the known-plaintext approach from the discussion — trying common greetings first rather than brute-forcing all 25 shifts. Students who design a Task C cipher that is genuinely harder to crack than Caesar's — for example, adding a null letter every 3 characters, or reversing the word order after encryption.

Exercise 2 — Password audit *Pairs · 5 minutes*

Present these six passwords. Pairs must rank them from weakest to strongest and explain why for each one.

1. password
2. 12345678
3. Angela2014
4. Tr0ub4dor!
5. correct horse battery staple
6. Xk#9mP$2qL

Expected ranking from weakest to strongest: 1, 2, 3, 4, 6, 5 — though students may argue about the order of 4, 5, and 6 and that argument is valuable. Password 5 is a passphrase — four random common words — and despite looking simple is actually very strong because of its length. Password 6 looks strong but is shorter and harder to remember. The debate about whether length or complexity matters more is exactly the right conversation.

What the teacher looks for: Students who understand that length matters as much as complexity — not just "it has symbols so it is strong." Students who identify that password 3 (Angela2014) is weak because it contains a real name and a real year, both of which a hacker would try early. Students who question why password 1 is weak not because it is short but because it is the most commonly used password in the world — meaning any hacker tries it first.

Extension — Build an unbreakable system *For fast finishers · 10 minutes*

Tell students: *"The Caesar cipher has one fatal weakness — if the hacker knows which cipher you are using, they only need to try 25 keys. Design a two-layer encryption system that is harder to crack."*

Students must design a system with two steps — for example:

- Step 1: Reverse the message

- Step 2: Apply a Caesar shift of their choice

They encrypt a short message using their system, write down the rule clearly, and pass both the encrypted message and the rule to the teacher. The teacher passes the encrypted message (not the rule) to another student who must try to crack it without knowing the rule.

Challenge questions:

- *"How many possible combinations does your two-step system have compared to a single Caesar cipher?"*
- *"If a hacker knows you are using a two-step system but not which two steps, does that help them? Why or why not?"*
- *"Modern encryption uses mathematical operations that are easy to do one way and nearly impossible to reverse — like multiplying two huge prime numbers together. The product is public, but nobody can factor it back into the original numbers fast enough. How is that different from what you designed?"*

This introduces the concept of one-way functions — the mathematical foundation of modern encryption — without requiring any maths beyond what age 8–12 students have. The intuition is enough.

Share and reflect

Time: 52–57 minutes

Ask 2–3 students to share their Task C cipher design from Exercise 1. After each, the class has 60 seconds to try to spot the rule. If they cannot, the designer explains it.

Then ask every student — verbally or on a sticky note — to complete this sentence:

> *"Encryption works because _____."*

The best answers will include ideas like: only the person with the key can decrypt it, it turns information into something unreadable without the rule, it makes brute force take too long to be worth it. Write the clearest one on the board.

Close with: *"Next lesson you are going to use everything you have learned — algorithms, loops, conditionals, variables, AI, and encryption — to design something real. You are going to identify a problem that technology could solve, design the solution, and present it. You are going to be the programmer, the designer, and the engineer all at once."*

Teacher dashboard note — what to observe after the lesson: Did students connect the brute-force cracking explanation back to loops from Lesson 2 — recognising that trying every possible password is just a loop? Did any students in Exercise 1 use the known-plaintext approach for Task B, trying common greetings first rather than working through all 25 shifts? Did the password audit in Exercise 2 produce genuine disagreement about whether length or complexity matters more — and did students argue their position with reasons? These signals indicate students are applying and connecting concepts across lessons, which is the core goal of the program at this stage.

Lesson 7

Lesson 7

Build something real — design thinking and your first prototype

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain B (22A, 22B), Domain C (23A) **Builds on:** All previous lessons — this is the integration session

Lesson overview

Learning goal Students can identify a real problem worth solving, apply the first three stages of design thinking to frame a solution, and produce a paper prototype of a digital tool with at least three connected screens and a clear logic flow that uses concepts from the program.

What you need Paper — at least 6 sheets per group. Pencils, pens, and coloured markers if available. Scissors optional. Whiteboard and marker. Sticky notes for the empathy mapping activity.

Key vocabulary to write on the board and leave up all lesson

- Design thinking — a structured way of solving problems by starting with the people who have the problem, not the solution
- Empathy — understanding a problem from another person's point of view, not your own
- Prototype — a rough early version of a product used to test ideas before building the real thing
- User — the person who will actually use the product
- Iteration — improving a design based on feedback, then testing again

Role model of the day Reshma Saujani (born 1975) is the founder of Girls Who Code, an organisation that has taught over 500,000 girls to code across the United States. She started it after noticing that while boys were encouraged to take risks and fail, girls were taught to be perfect — which made them afraid to try coding because they feared making mistakes. Her insight was not technical. It was human: she understood the problem from the user's point of view before designing the solution. That is exactly what design thinking is. Today students will do the same — start with a person and a problem, not a product.f

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"Think about something that genuinely annoys you or wastes your time at school or at home. Not something you want — something that is actually a problem for you or someone you know. What is it?"*

Ask students to write one problem on a sticky note silently — no discussion yet. After 90 seconds, ask 5–6 students to read theirs aloud. Write them on the board without comment.

Tell them: *"Every piece of technology that exists today started with someone noticing a problem exactly like the ones you just wrote. The people who built Google were annoyed that searching the internet was slow and disorganised. The people who built Spotify were annoyed that buying music was expensive and complicated. Today you are going to start in the same place they did — with a real problem."*

Teacher tip: The silent sticky note step is important. It gives every student — including quieter ones — a chance to contribute before the discussion starts. Collect the sticky notes and put them on the board. The class will use them in Phase 4 to choose their design challenge.

Real world connection: The most successful technology products in history were not invented by people who thought "I want to build an app." They were invented by people who thought "this thing is broken and I am going to fix it." WhatsApp was invented because the founders were annoyed by expensive international phone calls. Instagram was invented because sharing photos between phones was slow and low quality. Minecraft was invented because one person wanted to build things in a virtual world with no rules. The problem always comes before the solution.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: The broken school day

Tell students: *"I am going to describe a school day that has several small problems in it. Your job is to spot every problem — not to suggest a solution yet. Just the problem."*

Read this scenario aloud slowly:

"Maya arrives at school and cannot remember which classroom her first lesson is in because the schedule changed this week. She checks the noticeboard but it has not been updated. She is late. In the canteen at lunch, she queues for 10 minutes and when she gets to the front, the meal she wanted is sold out — but nobody told the people in the queue. After lunch she needs to return a library book but the library is closed with no sign saying when it will reopen. On her way home she realises she left her PE kit in her locker but does not know if she will need it tomorrow."

Ask pairs to list every problem they heard. Take answers from the class and write them on the board. You should get at least five distinct problems:

- Schedule changes not communicated clearly

- Canteen does not warn when items run out
- Library opening times not displayed
- Lost property and forgotten kit
- General communication gap between school and students

Now ask: *"Which of these could technology help with? And which of these is not really a technology problem — it is a people or organisation problem?"*

Students will debate this. Some problems (canteen stock) are obviously solvable with technology. Others (forgetting your kit) might be partially solvable but not fully. And some (schedule communication) might already have a solution that just is not being used. That nuance — technology does not solve everything, and sometimes the problem is not what it first appears to be — is exactly what design thinking addresses.

Teacher tip: The goal of this warm-up is not to find solutions — it is to train students to look carefully at problems before jumping to answers. If a student immediately says "just make an app for it," acknowledge the instinct but push back: *"Before we design the app, what exactly is the problem? Who has it? When does it happen? How often? Why does it happen?"* Those questions are the core of the empathy stage of design thinking.

Phase 3 — Concept introduction

Time: 10–20 minutes

Tell students: *"Professional designers and engineers use a process called design thinking to make sure they are solving the right problem in the right way. It has five stages, but today we are going to focus on the first three — because those are the ones that happen before any building starts."*

Write the five stages on the board and draw a simple arrow between each:

EMPATHISE → DEFINE → IDEATE → PROTOTYPE → TEST

Explain each of the first three in plain language:

Empathise means understanding the problem from the user's point of view — not your own. A user is the person who will actually use what you build. Empathy means asking: who is affected by this problem? When does it happen to them? How does it make them feel? What have they already tried? You find this out by talking to people, watching them, and listening carefully. You do not assume you already know.

Define means writing a clear, specific problem statement based on what you learned in the empathy stage. A good problem statement names who has the problem, what the problem is, and why it matters. It does not mention a solution yet. A weak problem statement: "Students need an app." A strong problem statement: "Students in Groep 7 often arrive late to lessons on days when the schedule changes because they have no reliable way to check the updated timetable quickly."

Ideate means generating as many possible solutions as you can — without judging any of them yet. The goal is quantity, not quality. Bad ideas are welcome because they sometimes lead to good ones. Write everything down. You can cross things out later.

Tell students: *"Most people skip straight from problem to solution. Design thinking forces you to slow down and make sure you understand the problem deeply before you start building. This is why well-designed technology feels easy to use — and why badly designed technology feels like it was built by someone who never actually talked to the people who would use it."*

Draw a simple phone screen outline on the board. Say: *"This is called a wireframe — a rough sketch of what a screen might look like. It does not need to be beautiful. It just needs to show what is on the screen and what happens when you tap something."*

Draw two boxes on the screen — a button labelled "Check timetable" and a button labelled "Report an issue." Draw an arrow from the "Check timetable" button pointing to a second screen. Tell students: *"That arrow means: when you tap this button, the app takes you to this screen. This is the logic of the app — and it is the same conditional logic from Lesson 3. IF the user taps this button, THEN show this screen."*

Teacher tip — connecting to earlier lessons: Spend two minutes explicitly connecting this lesson to the previous six. Write on the board:

- The app uses **sequences** — it does things in order
- The app uses **loops** — it keeps running until you close it
- The app uses **conditionals** — different buttons take you to different screens
- The app uses **variables** — it remembers your name, your class, your preferences
- The app needs to think about **AI** — could it learn your preferences over time?
- The app needs **encryption** — any personal data it stores must be protected

Tell students: *"You are not just designing a pretty picture. You are designing a system. Every screen you draw has logic behind it — and that logic is built from exactly the concepts you have learned."*

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The paper prototype sprint

This is the core activity. Groups of 3 will design and prototype a digital solution to a real problem in 20 minutes. It is deliberately fast — the goal is to produce something rough and testable, not something perfect.

Step 1: Choose your problem (3 minutes)

Groups look at the sticky notes from Phase 1 and the problems identified in the warm-up. Each group chooses one problem to solve. It must be:

- A real problem that students or teachers in their school actually experience

- Something where technology could genuinely help — not just replace a human conversation
- Specific enough that they can design a solution today

If groups struggle to choose, suggest these starting points from the warm-up: the canteen stock problem, the timetable communication problem, or the lost property problem. Any of these work.

Step 2: Empathy — who has this problem? (4 minutes)

Before drawing anything, each group answers these four questions in writing:

1. Who exactly has this problem? (Give them a name and describe them — "Maya, age 11, Groep 7, always rushing in the morning")
2. When does the problem happen? (Be specific — "every Monday when the timetable changes")
3. What have they already tried that did not work? (This prevents designing a solution that already exists and failed)
4. How does it make them feel? (Frustrated? Embarrassed? Stressed? This is the human dimension of the problem)

Teacher tip: The named person — "Maya" — is called a persona in design thinking. Having a specific imaginary person makes the design decisions much clearer. When a group disagrees about whether to include a feature, they can ask: "Would Maya use this?" That question is easier to answer than "would users use this?"

Step 3: Define — write the problem statement (3 minutes)

Groups write one sentence that completes this template:

> *"[Name] needs a way to [do what] because [why it matters], but currently [what makes it hard]."*

For example: *"Maya needs a way to check her updated timetable quickly in the morning because arriving late to the wrong classroom is stressful, but currently the only option is checking a noticeboard that is often outdated."*

Groups share their problem statement with the teacher before moving on. If it mentions a solution, send them back to rewrite it. The problem statement must describe the problem, not the answer.

Step 4: Ideate — generate solutions (3 minutes)

Groups spend exactly 3 minutes writing down every possible solution they can think of — no judging, no crossing out, just generating. Set a timer. Aim for at least 6 ideas. Encourage wild ideas alongside practical ones.

After 3 minutes, groups circle the one idea they want to prototype. It does not have to be the most exciting — it should be the one they can actually sketch in the time available.

Step 5: Prototype — draw three screens (10 minutes)

Groups draw their app on paper. Rules:

- Exactly three screens — no more, no fewer
- Each screen must have a title, at least two interactive elements (buttons, input fields, lists), and arrows showing where each element takes the user
- One screen must show what happens when something goes wrong — for example, what if the timetable has not been updated yet, or what if the canteen has run out of every option?

Walk around and ask:

- *"What variable does this screen need to remember?"*
- *"What conditional fires when the user taps this button?"*
- *"What happens if the user has no internet connection — does your app handle that?"*

The third question introduces edge cases — the same concept from Lesson 3 and Lesson 5. Students should try to handle at least one edge case in their design, even if just by adding a simple error message screen.

Teacher tip — managing the time: The prototype step will feel rushed — that is intentional. Professional designers use time pressure to stop overthinking and start making. Remind groups that this is a rough draft, not a finished product. Messy and functional is better than beautiful and incomplete. If groups are spending too long on layout and colours, redirect them: *"The drawing does not matter. The logic matters. Draw boxes and arrows."*

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In Step 2, you described a specific person who has the problem. How did thinking about that person change the decisions you made when designing the solution — compared to just thinking about 'students in general'?"* Follow-up: *"What assumptions did you make about Maya that might not be true for everyone with the same problem? Who might your design not work for?"*

Question 2: *"We said the problem statement must not mention a solution. Why is that rule important? What goes wrong when you start with a solution instead of a problem?"* Follow-up: *"Think about an app that exists right now that feels like it was designed by someone who never talked to the people who would use it. What did the designer get wrong?"* Students will have strong opinions — let them.

Question 3: *"Your prototype has three screens. A real app might have 300. At what point does more screens and more features make a product better — and at what point does it make it worse?"* Follow-up: *"The original Instagram app had exactly four screens: feed, camera, notifications, and profile. Nothing else. What would you have cut from your design if you could only keep two screens?"* This is the concept of minimum viable product — the

simplest version of a product that still solves the core problem. Students do not need the term, but the thinking is valuable.

Teacher tip: Question 2 generates the most useful discussion for this age group. Students have strong intuitions about badly designed technology — apps with too many steps, websites that make it hard to find what you need, games with tutorials that go on too long. Giving them the design thinking vocabulary to describe why those things feel wrong is empowering. Let this discussion run if the class is engaged.

Exercises

Time: 47–57 minutes

The primary exercise for this lesson is the prototype itself — groups should use any remaining time to add detail, improve their problem statement, or add the error screen if they have not done so.

Additionally, ask each group to complete these two written tasks on a separate piece of paper to accompany their prototype:

Written task 1 — The logic layer *5 minutes*

For each of your three screens, write:

- One variable the screen uses or updates
- One conditional that determines what happens when the user interacts with it
- One edge case the screen needs to handle

Format:

Screen 1: [Screen name]
Variable: _____
Conditional: IF _____ THEN _____ ELSE _____
Edge case: What if _____?

This task makes the connection between the visual design and the underlying logic explicit. Students who can fill this in for all three screens have genuinely understood how the concepts from the program connect to real software design.

What the teacher looks for: Variables that actually make sense for the screen — not just "SCORE = 0" copied from Lesson 4, but something specific like "TIMETABLE_UPDATED = true/false" or "USER_CLASS = Groep 7." Conditionals that reflect real decisions the app needs to make. Edge cases that show students are thinking about what could go wrong, not just what should go right.

Written task 2 — The pitch sentence *3 minutes*

Each group writes one sentence using this formula, drawn directly from the Dream Space Dragon's Den pitch format:

> *"Our app is called [NAME]. It helps [WHO] solve [PROBLEM] by [FEATURE]. Without it, [CONSEQUENCE]."*

For example: *"Our app is called SchoolCheck. It helps students in Groep 7 solve the problem of not knowing about timetable changes by sending a notification 30 minutes before school starts with the updated schedule. Without it, students arrive at the wrong classroom and miss the start of their lesson."*

This sentence is what they will use to open their presentation in Lesson 8.

Extension — stress test your prototype *For fast finishers · 10 minutes*

Groups swap their prototype with another group. The receiving group has 5 minutes to try to break it — find every situation the design does not handle, every question it leaves unanswered, every user who would find it confusing or unusable.

They write their findings on a sticky note and return it with the prototype. The original group then has 5 minutes to respond — either by improving the design or by explaining why the edge case is acceptable.

This is a simplified version of usability testing — the same process that professional designers use before launching a product. The questions the receiving group asks should follow this format:

- *"What happens if the user does not have an account yet?"*
- *"What happens if the user makes a mistake and wants to go back?"*
- *"What happens if the user is in Groep 5, not Groep 7 — does it still work for them?"*
- *"What happens if the app loses internet connection halfway through?"*

Groups who receive genuinely challenging feedback and use it to improve their design are demonstrating the iteration stage of design thinking — the fourth stage, which the lesson intentionally does not cover in full so that it can be experienced here organically.

Share and reflect

Time: 52–57 minutes

Each group holds up their prototype — just holds it up, no explanation yet. The class has 30 seconds to look at each one silently. Then ask: *"Just from looking — without any explanation — what problem does each one seem to solve?"* If the class can identify the problem from the design alone, the prototype is communicating clearly.

Then ask every student — verbally or on a sticky note — to complete this sentence:

> *"The hardest part of designing something was _____ because _____."*

Collect the responses. Common answers: "not jumping to a solution," "figuring out what the user actually needs," "deciding what to leave out." All of these are things professional designers say. Write the most honest one on the board.

Close with: *"Next lesson is the final one. You are going to present your prototype to the class — and to anyone else we invite. You will explain the problem, show the design, and use everything you have learned over the past seven weeks to back it up. You are not presenting a drawing. You are presenting an argument for why this problem matters and why your solution is the right one."*

Teacher dashboard note — what to observe after the lesson: Did groups write a problem statement that genuinely avoids mentioning a solution — or did they keep rewriting it to include their app idea? Did the logic layer task produce variables and conditionals that are specific to their design, or did groups copy generic examples from earlier lessons? Did any groups in the extension receive challenging feedback and actually use it to change their design rather than defend the original? That last behaviour — genuine willingness to revise based on feedback — is one of the most important signals in the entire program. It indicates a student who has internalised the iterative nature of design, which is a core 21CLD self-regulation skill. Note it on the dashboard and make sure those students are given the opportunity to speak first in Lesson 8.

Lesson 8

Lesson 8

Who is technology for? Ethics, fairness, and responsibility

Age group: Groep 5–8 (ages 8–12) **Duration:** 45–60 minutes **Devices required:** None — fully unplugged **SLO alignment:** Domain C (23A), Domain A (21D) **Builds on:** Lesson 5 (AI bias), Lesson 6 (encryption, privacy), Lesson 7 (design thinking, empathy)

Lesson overview

Learning goal Students can identify an ethical question raised by a piece of technology, argue a position using evidence and reasoning, and apply ethical thinking to their own prototype from Lesson 7 — identifying at least one decision they made that has an ethical dimension.

What you need Whiteboard and marker. Paper and pencils. Sticky notes in two colours — one for "agree," one for "disagree." Space at the front of the room for the walking debate activity. The prototypes from Lesson 7.

Key vocabulary to write on the board and leave up all lesson

- Ethics — the study of what is right, fair, and responsible
- Stakeholder — anyone affected by a decision, not just the person who made it
- Unintended consequence — a result of a decision that nobody planned for
- Transparency — being open and honest about how something works and why
- Consent — agreeing to something after being fully informed about what it means

Role model of the day Timnit Gebru (born 1983) is a computer scientist and AI ethics researcher who co-wrote one of the most important papers in AI ethics — a study showing that facial recognition AI was significantly less accurate for darker-skinned women than for lighter-skinned men, and that large AI language models posed risks that their creators were not acknowledging. She was fired from Google for refusing to withdraw the paper. She then founded her own independent AI ethics research institute. She did not stop building technology — she insisted that building technology responsibly means asking hard questions about who it harms, not just who it helps. Today students will ask exactly those questions.

Phase 1 — Spark question

Time: 0–3 minutes

Show this question on the board or read it aloud:

> *"Can something be useful and harmful at the same time? Can a technology be good for some people and bad for others — and still be worth building?"*

Ask students to turn to the person next to them and share one example. Take 3–4 answers. Write the examples on the board.

Tell students: *"Almost every powerful technology ever built has done both — helped some people and harmed others, solved one problem and created a new one. Today we are going to look at that honestly. Not to say technology is bad. Not to say it is good. But to think carefully about who it is for, who it affects, and who gets to decide."*

Teacher tip: Students often give examples like cars (useful but cause accidents), social media (connects people but causes anxiety), or nuclear power (cheap energy but dangerous waste). All of these are exactly right. Accept them all and write them up — they will be referenced during the discussion. If students struggle to think of examples, prompt with: *"Think about something you use every day that has a downside you have noticed."*

Real world connection: In 2018 Amazon scrapped an AI hiring tool it had been developing for four years after discovering it was systematically ranking women lower than men for technical roles — because it had been trained on 10 years of previous applications, and those applications came mostly from men. The tool was not designed to be unfair. Nobody wrote a rule saying "rank women lower." But the training data reflected a historical imbalance, and the AI learned it as a pattern. Useful for Amazon. Harmful for women applicants. The technology worked exactly as designed — and that was the problem.

Phase 2 — Unplugged warm-up

Time: 3–10 minutes

Activity: The walking debate

Clear a space at the front of the room. Mark one side "AGREE" and the other side "DISAGREE" — or write them on the board on either side. Tell students: *"I am going to read a statement. You must physically move to the side that matches your view — agree or disagree. You cannot stand in the middle. You must choose. Then I will ask people on each side to explain their position."*

Read these statements one at a time. After each one, ask one student from each side to give their reason. Keep it fast — 60 seconds per statement.

Statement 1: *"If an app is free to use, the company that makes it has the right to collect data about you and sell it to advertisers."*

Statement 2: *"If a self-driving car must choose between hitting one pedestrian or swerving and hitting five — the programmer who wrote the algorithm should be responsible for the outcome."*

Statement 3: *"Schools should be allowed to use AI to monitor students' messages and social media activity if it helps prevent bullying."*

Statement 4: *"If you design an app that most people love but a small group of people are harmed by, you should be allowed to keep the app running."*

Teacher tip: The walking debate format — borrowed directly from the Dream Space Teaching and Learning Approach — is the most effective way to generate genuine ethical reasoning at this age. The physical commitment to a position forces students to actually have a view rather than hedging. Students who move to the "disagree" side on Statement 3 and the "agree" side on Statement 1 should be asked to explain the apparent contradiction — that tension is exactly where ethical thinking happens.

Teacher tip — on Statement 2: This is the classic trolley problem applied to autonomous vehicles. It will generate strong reactions. The key point to draw out is not what the right answer is — it is who should make the decision and when. Should the programmer decide in advance? Should there be a law? Should the AI decide in the moment? Each answer has different implications. Do not resolve it — leave the tension.

Phase 3 — Concept introduction

Time: 10–20 minutes

Tell students: *"Ethics is not about having the right answer to hard questions. It is about asking the right questions in the first place — and being honest when you cannot answer them."*

Write on the board:

WHEN A TECHNOLOGY IS BUILT, ASK:

1. Who does it help?
2. Who does it harm — including people who did not choose to use it?
3. Who decided to build it — and who did not get a say?
4. What happens if it goes wrong?
5. Who is responsible when it does?

Tell students: *"These five questions apply to every technology ever built — from a traffic light to a social media platform to a nuclear power station. Today we are going to apply them to technologies you already know, and then to the prototype you built last week."*

Introduce three ethical concepts with real examples:

Unintended consequences The people who built the "like" button on Facebook wanted to create a simple way to show positivity. They did not intend to create a system where people's self-worth became tied to how many likes their posts received, or where content designed to provoke outrage spread faster than content designed to inform. Both of those things happened — not because anyone planned them, but because a simple feature interacted with human psychology in ways nobody fully anticipated. Unintended consequences are not excuses. They are reasons to test more carefully, think more broadly about who is affected, and be willing to change things when harm becomes visible.

Stakeholders A stakeholder is anyone affected by a decision — not just the person who made it or the person it was designed for. When a school introduces a new app to track student attendance, the stakeholders include: students (whose location is tracked), parents (who may or may not be informed), teachers (who have to use it), the company that makes the app (who receives the data), and the students who are incorrectly marked absent by a bug. Good ethical design identifies all stakeholders before building — not after.

Consent Consent means agreeing to something after being fully informed about what it means. Most technology platforms ask for consent through terms and conditions that almost nobody reads — and that are often written in language that most adults cannot fully understand. Ask students: *"Have you ever clicked 'I agree' on a terms and conditions page without reading it?"* Most will have. Then ask: *"If you did not read it, did you actually consent to what it said?"* This is a genuine open question in law and ethics, and it is the foundation of debates about children's data, surveillance capitalism, and the right to be forgotten.

Teacher tip — keep it grounded: For Groep 5–6, focus on unintended consequences using examples from games and apps they know. For Groep 7–8, the stakeholder and consent frameworks are accessible and generate strong opinions. Do not try to cover all three concepts in equal depth — pick the one that the class responds to most strongly and spend more time there.

Phase 4 — The challenge

Time: 20–42 minutes

Activity: The ethics audit

This challenge has two parts. The first applies ethical thinking to a case study. The second applies it to students' own prototypes from Lesson 7.

Part 1: The case study — 12 minutes

Tell students: *"A technology company has built an app for primary schools. The app tracks how long each student spends on each exercise, how many mistakes they make, which questions they skip, and how their performance changes over the school year. This data is used to give teachers a dashboard showing which students are struggling and which are excelling. The app is free for schools because the company sells anonymised versions of the data to educational publishers and to universities doing research on learning."*

Groups of three must answer these five questions in writing:

1. Who does this app help, and how?
2. Who might be harmed by it — including people who are not students or teachers?
3. Who decided to build it and who did not get a say — including students themselves?
4. What could go wrong — technically, socially, or ethically?
5. What would you change about it to make it more ethical — or would you not build it at all?

Walk around and listen. Common strong answers: students might be harmed if the data labels them as "struggling" permanently and affects how teachers treat them. Parents did not consent to their child's learning data being shared with publishers. The company has a financial incentive to keep the data — which conflicts with the school's interest in protecting students. Students had no say in whether this data is collected about them.

After 8 minutes, take one answer per question from different groups. Write the most important points on the board. Do not resolve the debate — record the tension.

Part 2: Audit your own prototype — 10 minutes

Tell students: *"Now apply the same five questions to the prototype you designed last week. Every design choice has an ethical dimension — even if it seems small."*

Each group works on their own prototype from Lesson 7. They must answer:

1. Who does your app help? Is it genuinely helpful for all of them equally, or more helpful for some users than others?
2. Does your app collect any information about users? If yes — who sees that information, where is it stored, and could it be used in a way you did not intend?
3. Who did you think about when you designed this? Who did you not think about — and would your app still work for them?
4. What is the worst realistic thing that could happen if your app had a bug, was hacked, or was used in a way you did not plan for?
5. Is there anything about your design you would change now, after thinking about these questions?

Groups write their answers on a sheet of paper and attach it to their prototype. This document is called an ethics audit — a real practice used by responsible technology companies before launching a product.

Teacher tip: Some groups will find this exercise uncomfortable — they may realise their prototype collects data they had not thought carefully about, or that it would not work for students with disabilities, or that a bug could expose personal information. That discomfort is exactly the right feeling. Tell groups: *"Noticing a problem in your own design is not a failure. It is exactly what this stage of the process is for. The failure would be launching without noticing."*

Teacher tip — connecting to Lesson 7: In Lesson 7, groups identified stakeholders as part of the empathy stage. Ask them: *"Look at your stakeholder list from Lesson 7. Did you think about all of those people when you designed your three screens? Which ones got left out?"* This makes the connection between design thinking and ethics explicit — empathy is not just about the primary user, it is about everyone the technology touches.

Phase 5 — Discussion

Time: 42–47 minutes

Question 1: *"In Part 1, the company said the data was anonymised before being sold. Does anonymising data make it ethical to sell? What would you need to know about the anonymisation before you trusted that claim?"* Follow-up: *"In 2019 a major tech company claimed its location data was anonymised — but researchers showed they could identify individuals from the patterns in the data alone, even without names. What does that tell you about the word 'anonymous'?"*

Question 2: *"You applied the ethics audit to your own prototype. Did any of your answers surprise you — did you find a problem you had not noticed before?"* Follow-up: *"If you found a problem — is the right response to fix it, to add a warning, to limit who can use the app, or to not build it? Are those all equally good responses?"*

Question 3: *"We have spent eight weeks learning to build things with technology. Should learning to build things come with a responsibility to think about the consequences? Or is that someone else's job — maybe a lawyer, a regulator, or a politician?"* Follow-up: *"Timnit Gebru was fired for raising ethical concerns about a technology her employer was building. Does that change your answer?"*

Teacher tip: Question 3 is the most important question of this lesson and arguably of the entire program. The answer students give to "whose responsibility is it?" shapes whether they will raise ethical concerns when they encounter them in future workplaces or communities. Students who say "it is everyone's responsibility, including the builder" are articulating the core value of responsible technology design. Students who say "it is someone else's job" are worth engaging further — not to correct them, but to ask: "What if the lawyer and the regulator do not understand the technology well enough to know what questions to ask?"

Exercises

Time: 47–57 minutes

Exercise 1 — The responsible design checklist *Written, no devices needed · 8 minutes · Individual*

Every responsible technology product should be checked against a set of ethical criteria before it is released. Students create their own checklist — not by copying one from the board, but by writing the questions they think matter most based on everything they have discussed today.

Rules:

- Exactly five questions
- Each question must be answerable with yes or no
- At least one question must be about people who are not the primary user
- At least one question must be about what happens if something goes wrong

After writing their checklist, students apply it to their own prototype. For any "no" answer, they write one sentence explaining what they would change to turn it into a "yes."

What the teacher looks for: Students who write questions that are specific rather than vague — "Does the app store personal data about users under 13?" rather than "Is the app safe?" Students whose checklists reflect the specific ethical dimensions discussed in the lesson — consent, stakeholders, unintended consequences — rather than generic safety concerns. Students who identify a genuine "no" in their own prototype and propose a specific, actionable fix rather than a vague improvement.

Exercise 2 — The unintended consequences game *Groups of three · 7 minutes*

Present three technologies. For each one, groups must identify one unintended consequence that the original designers probably did not plan for. They should write their answer as: *"The designers wanted [X], but an unintended consequence was [Y], which affected [Z]."*

Technology 1 — the school canteen app from Lesson 7: Designed to tell students what is on the menu and warn when items run out. Example unintended consequence: students who do not have phones or whose parents do not allow apps are excluded from the information advantage, creating a two-tier system at the canteen.

Technology 2 — autocorrect on a phone keyboard: Designed to fix spelling mistakes automatically and speed up typing. Example unintended consequence: it reduces the frequency with which users notice and correct their own spelling errors, potentially reducing spelling ability over time in populations that rely heavily on it.

Technology 3 — a fitness tracker that counts your daily steps: Designed to encourage people to move more and be healthier. Example unintended consequence: people who use wheelchairs or have mobility conditions cannot use step counting as a meaningful measure, and seeing a zero step count each day may feel exclusionary or demoralising rather than motivating.

Groups share their answers. The best answers will identify consequences that are specific, plausible, and affect a group of people the original designers probably did not think about carefully enough.

Extension — Write the ethical brief *For fast finishers · 10 minutes*

Tell students: *"Before a technology company builds a new product, some companies now require an ethical brief — a short document that identifies the ethical questions the team must answer before launching. Write a one-page ethical brief for one of these three products:"*

Option A: An AI tutor for primary school students that adapts to each student's learning pace and style, and sends weekly reports to parents and teachers.

Option B: A social media platform designed specifically for children aged 8–12, with games, messaging, and the ability to share photos with approved friends.

Option C: A smart school bag that tracks the student's location in real time and sends alerts to parents if the student leaves the school building without permission.

The brief must cover: who the product is designed for, who else it affects, what data it collects and who sees it, three ethical questions the team must answer before launch, and one scenario where the product could cause harm even if it works perfectly as designed.

This exercise mirrors real practice at technology companies that take ethics seriously. Students who complete it are thinking at a level that many professional developers do not reach until years into their careers.

Share and reflect

Time: 52–57 minutes

Ask each group to share one item from their responsible design checklist — just one question, read aloud. After all groups have shared, look at the full set on the board and ask: *"Is there anything important we missed? Any ethical question that nobody wrote?"*

Then ask every student — verbally or on a sticky note — to complete this sentence:

> *"Before I build something, I will always ask _____."*

Collect the responses. Write the most important three on the board alongside the vocabulary wall. These are the ethical commitments the class is making — not to a teacher, but to each other and to the people who will use what they build in the future.

Close with: *"Next lesson is the final one. You are going to present your prototype, your logic layer, and now your ethics audit to the class — and to whoever we invite to listen. You have not just designed an app. You have designed it, thought about who it is for, thought about who it might harm, and asked the hard questions. That is what responsible technology looks like. That is what Dream Space is for."*

Teacher dashboard note — what to observe after the lesson: Did students find genuine ethical problems in their own prototypes during the audit — or did they only find problems in the case study? The willingness to apply critical thinking to your own work is significantly harder than applying it to someone else's. Students who identified a real issue in their own design and proposed a specific fix are demonstrating the most important skill of this lesson. Did any student argue during the walking debate that responsibility lies entirely with regulators or lawyers — and were they able to engage with the counterargument? And did the unintended consequences exercise produce consequences that affected groups the original designers probably overlooked — disabled users, students without devices, younger children? Those observations are the richest data points from this lesson and worth recording individually on the dashboard.

Highschool 15-17

Here is the fully detailed Lesson 1.

Lesson 1

How fast can a computer think? The Sorting Dilemma 📦

Age group: High School (ages 15–17)

Duration: 25–30 minutes

Devices required: Students' smartphones (Dream Space App), Whiteboard, and markers.

SLO alignment: Domain A (21A), Domain B (22B)

Lesson overview

Learning goal: Students can explain in their own words that there are multiple ways to write an algorithm to solve the same problem, but the *efficiency* of the sequence determines how fast a computer processes data.

What you need: Whiteboard and markers. 5 large sticky notes with random numbers (e.g., 4, 87, 12, 99, 3). Students need the Dream Space App installed.

Key vocabulary to write on the board and leave up all lesson

Algorithm: a set of step-by-step instructions

Efficiency: achieving maximum productivity with minimum wasted effort

Optimization: modifying a system to make it work faster

Role model of the day Grace Hopper (1906–1992) was a computer scientist and naval officer who invented the first "compiler," fundamentally changing how efficiently humans could communicate with machines. Today students will look at how efficiency changes everything.

Phase 1. Spark question

Time: 0–2 minutes

Tell the class: "Open your Dream Space app."

The first screen shows a bold question: "Will algorithms completely manage global food distribution in 2050, or will humans always need to intervene?"

Ask students to select their answer in the app. The app saves this baseline for the end of the lesson.

Teacher tip: Accept all answers. Older students might bring up concepts like AI bias, supply chain crashes, or human intuition. These are excellent and will connect to the final discussion.

Real world connection: When you order a package, it doesn't travel in a straight line to your house. It goes through massive sorting facilities where optimization algorithms decide exactly which truck it gets on to save millions in fuel.

Phase 2 — Unplugged warm-up

Time: 2–6 minutes

Activity: App Puzzle & The Human CPU

Tell the class: "Complete the 2-minute logic sequence puzzle on your app screen and hit 'Got it!' when done."

Next, bring 5 students up to the front. Hand each a numbered sticky note to hold up.

Tell the class: "You are the computer processor. Sort these numbers from lowest to highest. Rule: A computer can only compare TWO things at a time. You can only ask two people standing next to each other to swap."

Teacher tip: Count the swaps out loud. Let them feel how slow and tedious it is to move one number across the entire line. This demonstrates "Bubble Sort."

Phase 3 — Concept introduction

Time: 6–10 minutes

Write the word EFFICIENCY on the board. Tell students: "Since you already know what an algorithm is, today we act as systems architects. We aren't just writing instructions—we are writing *efficient* instructions."

Explain Merge Sort (Divide and Conquer): splitting the line in half, sorting small groups simultaneously, and merging them back together. Explain that this is exponentially faster.

Real world connection — bring this in now: Ask the class, "Has a navigation app ever rerouted you while driving to save 2 minutes?" That is an optimization algorithm calculating thousands of possible routes instantly.

Phase 4 — The challenge

Time: 10–22 minutes

Activity: The Logistics Optimization

Setup (2 minutes) Students check the app for their randomly assigned daily team roles (e.g., Captain, Reporter). Draw a map on the whiteboard with 5 "Nodes" (A, B, C, D, E) connected by lines. On each line, write a travel time in minutes.

Round 1: Brute force (3 minutes) Ask the class for a route starting at A, hitting all nodes, and returning to A. Add up the total minutes on the board.

Round 2: Teams write the algorithm (7 minutes) In pairs, guided by the 'Captain', students write a set of rules (an algorithm) that a drone could use to *always* find a fast route, even if the map changed (e.g., "Always pick the lowest number path you haven't visited").

Teacher tip: The algorithm students write in Round 2 is known as a "Greedy Algorithm." It often works, but sometimes traps you in a corner where the only way home is a massive 50-minute path. Let them discover this flaw.

Phase 5 — Discussion

Time: 22–25 minutes

Use these two questions:

Question 1: "What was the flaw in your routing rules? Did picking the shortest path immediately sometimes lead to a longer path overall?"

Question 2: "If you were programming a self-driving ambulance, would you prioritize the absolute shortest distance, or the route with the fewest left turns? Why?"

Teacher tip: Older students appreciate the real-world stakes in Question 2. An algorithm isn't just math; it's a series of priorities chosen by a human.

Exercises & App Extension

Time: 25–28 minutes

Exercise 1 — Optimize the Morning (App integration) · Core exercise

Working individually in the app, students type out their exact morning routine.

Task: They must "optimize" their algorithm in the app to save at least 10 minutes.

Rules for the app input:

They must use "parallel processing" (e.g., making toast while packing a bag).

What the teacher looks for: Students realizing that sequence order matters for efficiency, not just for getting the job done.

Teacher tip: If time is short, assign this app exercise as homework or a fast-finisher task.

Share and reflect

Time: 28–30 minutes

Tell the class: "To earn today's badge, complete your reflection card in the app."

In the app, students complete the 3-tap reflection:

1. What did you make today?
2. What would you change?

3. What do you want to learn next?
   Teacher dashboard note: Did students grasp the difference between "working" code and "efficient" code? Check the dashboard to see if their reflections mention speed or parallel processing.

To make sure the exercises in the remaining 7 lessons are practical for your developers to build into the app, will the Dream Space app support free-text input for the students' algorithms, or should we design the app-based exercises strictly as drag-and-drop/multiple-choice interactions to save time?

HS L1

Lesson 1

Why does your phone freeze? How computers think about efficiency

Age group: 15–17 · **Duration:** 30 minutes · **No prior knowledge assumed SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can explain the difference between a working algorithm and an efficient one, and connect this to technology decisions that affect their daily lives.

What you need: Whiteboard and marker. Five sticky notes with one number written on each: 47, 8, 93, 21, 64. Paper and pencils.

Vocabulary on the board — write before students arrive: Algorithm · Efficiency · Trade-off · Scale

Role model: Grace Hopper (1906–1992) — mathematician, computer scientist, and United States Navy Rear Admiral. She invented the first compiler — a program that automatically translates human-written instructions into machine code. Before her invention every programmer had to write in raw machine language, which was slow, error-prone, and accessible only to specialists. Her famous line: *"The most dangerous phrase in the language is 'we've always done it this way.'"* Today students will apply exactly that thinking — questioning not whether an approach works, but whether it is the best way.

Phase 1 — Spark question · 0–2 min

Write on the board:

> *"Your phone has billions of calculations happening every second. So why does it still freeze, lag, and run out of battery too fast?"*

45 seconds in pairs — one idea each. Then take 2 answers from the class. Students typically say: too many apps open, old hardware, not enough RAM. Acknowledge all of these — they are partially correct at the hardware level. Then land on the key reframe:

"All of those are hardware explanations. But two phones with identical hardware can perform very differently depending on how the software is written. The instructions matter as much as the machine. It is almost never that the hardware is not powerful enough. It is almost always that the instructions are not written well enough. A computer doing the wrong thing very fast is worse than a computer doing the right thing slightly slower. That gap between instructions that work and instructions that work well is what this lesson is about."

Real world hook: Amazon's fulfilment centres sort millions of packages per hour. A one percent improvement in the sorting algorithm saves hundreds of millions of euros annually

and significant fuel costs. A one percent degradation cascades into delays across an entire logistics network. At that scale, the quality of the instructions is everything — not the speed of the machines running them.

Phase 2 — Two ways to sort · 2–12 min

Ask five volunteers to come to the front. Give each a sticky note with one number and ask them to stand in this random order facing the class: 47, 8, 93, 21, 64.

Round 1 — Bubble Sort (4 minutes)

Tell the class: *"Your job is to sort these five people from lowest number on the left to highest on the right. One rule: you can only compare two people standing next to each other, and the only move allowed is a swap. You cannot move someone more than one position at a time."*

Start counting swaps out loud and write the running total on the board. The class directs the sorting — call out which adjacent pair to compare, then whether they should swap. When the line is sorted, stop and write the total number of swaps.

Then write this on the board:

5 items: ~10 swaps
1,000 items: ~500,000 swaps
1,000,000 items: ~500,000,000,000 swaps

Tell students: *"This is called Bubble Sort. It works — every number ends up in the right place. But the number of swaps needed grows roughly with the square of the list size. Double the list, quadruple the work. For a warehouse sorting millions of packages, this is catastrophic. The algorithm is correct. The algorithm is also useless at scale."*

Round 2 — Merge Sort (4 minutes)

Return the five students to their original random order. This time: *"We are going to try something fundamentally different. Split into a group of two on the left and a group of three on the right."*

Walk through it step by step:

- Sort the left group of two — one comparison, done immediately
- Sort the right group of three — two or three comparisons, done
- Now merge: compare only the front person of each group. Take the smaller number. Compare the new front of each group. Repeat until all five are in order.

Count comparisons — it is noticeably fewer than Bubble Sort. Write:

1,000 items: ~10,000 comparisons
1,000,000 items: ~20,000,000 comparisons

Ask: *"Why was that faster?"* Take one answer. Students will usually identify that sorting small groups is easier. Land on: *"The key insight is divide and conquer — break a hard problem*

into smaller problems, solve each one, and combine the results. This principle appears across all of computing, engineering, and management. You just invented it from first principles."

Draw two rough curves on the board — one steep curve for Bubble Sort, one much flatter curve for Merge Sort. Label the x-axis "size of the list" and the y-axis "work required."

Ask: *"Same result — a sorted list. Same computer. Completely different cost. Which line do you want your software on?"*

Teacher tip: Students do not need to understand the mathematics of n^2 versus $n \times \log(n)$. The numbers on the board and the two curves convey the intuition clearly enough. What matters is that students feel the difference between an algorithm that gets unmanageable and one that stays tractable as the problem gets larger. That feeling is the concept.

Phase 3 — Delivery drone challenge · 12–24 min

Draw this map on the board, taking care to label all nodes and write the travel times on each connecting line:

```
      A
      / \
    12   8
   /     \
  B---5---C
   \     /
    9   7
     \ /
      D---11---E
```

Tell students: *"A delivery drone starts at warehouse A and must visit every warehouse exactly once before returning to A. Your group needs to write a set of rules the drone can follow to find a fast route. The rules must be written clearly enough that someone who has never seen this map could follow them without making any decisions of their own — like a computer following an algorithm. You have 5 minutes."*

Groups of three. Give them paper to write on. Walk around while they work — do not help with the content, but do push groups to be more specific: *"What does 'go to the nearest warehouse' mean if two warehouses are the same distance? Your rules need to handle that."*

Testing the algorithm (4 minutes)

Ask one group to read their rules aloud while you trace the path on the board exactly as written — no interpretation, just the rules. Most groups will write something close to: always move to the nearest unvisited warehouse.

Following that rule strictly on this map produces: A → C (8 minutes) → B (5 minutes) → D (9 minutes) → E (11 minutes). At this point the drone is at E with no unvisited warehouses left — but there is no direct route from E back to A on this map. The drone is trapped.

Ask: *"What went wrong? Every single decision looked correct. The drone always chose the nearest option. The algorithm followed the rules perfectly. But the overall result was a failure. Where specifically did the approach break down?"*

Let students diagnose it before you name it. They should identify that the algorithm only looked at the immediate next step — it never considered whether those choices would leave a viable path home. Then name it: *"This is called a greedy algorithm. It always makes the best decision immediately available, without considering future consequences. It is fast, simple, and often works well. But when short-term optimal decisions accumulate into a long-term failure, you need something more sophisticated."*

Ask: *"What single rule would you add to fix this?"* Most groups will suggest something like: before choosing the next warehouse, check whether there is still a path back to A from the remaining unvisited warehouses. That is look-ahead — the algorithm considers future states before committing to a current decision. Tell students: *"That principle — checking the consequences of a decision before making it — appears in GPS routing, chess engines, and AI systems of all kinds. You just derived it from a logistics problem on a whiteboard."*

Teacher tip: If a group's algorithm accidentally avoids the trap — because their rules happened to send the drone in the correct order — use their solution as a positive example. Ask them to explain what their rules did differently, and whether their approach would still work if the travel times on the map changed. Often the algorithm that avoids the trap for this specific map would fail on a different one, which is an equally valuable lesson.

Phase 4 — Discussion · 24–29 min

Pick the question that fits how the lesson went. Do not rush this — a real discussion that develops is worth more than two discussions that are cut short.

If the greedy algorithm failure generated strong reactions:

"Your algorithm made the best local decision at every step and still produced a globally bad result. Can you think of a real-world situation — not in technology — where individually rational decisions accumulate into a collectively bad outcome?"

Push toward specific examples if students are vague: traffic jams (every driver picks the fastest route — the collective result is gridlock that makes everyone slower), climate change (every individual decision to use cheap energy makes sense locally — the global accumulation is catastrophic), social media engagement (every platform optimises for individual engagement — the collective result is a more polarised and misinformed public).

Follow-up: *"In all of those examples, nobody made a wrong decision at the individual level. The harm emerged from the aggregation. Who is responsible for fixing a problem that nobody individually caused — and is there a technical solution, or does it require something else?"*

If the phone framing from Phase 1 resonated more strongly:

"Some apps deliberately use more processing power and battery than they need to — because keeping your screen active longer improves their engagement metrics. From a pure efficiency standpoint, that is a failure. But from a business standpoint, it is working exactly as designed. Is that an efficiency failure or an efficiency success — and does the answer depend on whose perspective you take?"

Follow-up: *"The algorithm is doing its job perfectly. The job itself is the problem. Who should be responsible for changing that — the company that built the app, the platform that hosts it, the regulator, or the user who could choose to delete it? And is 'just delete the app' actually a realistic option for most people?"*

Exit reflection · 29–30 min

Students write one sentence on paper:

"The difference between a working algorithm and a good algorithm is _____."

Take 2–3 answers aloud. Strong answers will include ideas like: how it scales to large inputs, whether it considers future consequences and not just the next step, whose interests it serves, what it costs in time or memory. Write the sharpest answer on the board — it will stay there next lesson alongside the reflection from Lesson 2.

Close: *"Next lesson the question shifts from efficiency to security. How do computers decide what to let in and what to block — and what happens when the rules are too strict, too loose, or simply wrong in ways that only become visible when something goes badly."*

Fast finisher question: *"The route-finding problem from today is a simplified version of a famous unsolved problem called the Travelling Salesman Problem. For 5 warehouses there are 12 possible routes. For 20 warehouses there are over 60 quadrillion. There is a million-dollar prize — the Clay Millennium Prize — for anyone who can prove whether a fast perfect solution to this class of problems is even theoretically possible. The question is called P vs NP. You do not need to understand the mathematics — but think about this: if it turned out that a fast perfect solution did exist, what would change about the world? Which systems that currently rely on the fact that some problems are hard to solve would suddenly be vulnerable?"* No written answer required. Worth discussing briefly if time allows.

Teacher note: Three signals worth watching for. First — did students identify the greedy algorithm failure mode before you named it, or did it only become clear once you pointed it out? Students who diagnose it independently are already thinking about algorithms as systems with consequences rather than just sequences of steps. Second — did the discussion produce genuine disagreement, or did the class converge quickly on one answer? Real disagreement indicates students are engaging with the question rather than performing a correct response. Third — did anyone connect the greedy algorithm failure to a real-world systemic problem — traffic, climate, social media — without being prompted to? That connection, made unprompted, is the clearest signal that a student is thinking about technology as something embedded in the world rather than separate from it. Note those students. They are the ones to stretch hardest in the lessons that follow.

HS L2

Lesson 2

How do computers make secure decisions? The cybersecurity firewall

Age group: 15–17 · **Duration:** 30 minutes · **No prior knowledge assumed beyond Lesson 1 SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can explain how computers use Boolean logic (AND, OR, NOT) to build conditional rules, and apply this to a realistic cybersecurity scenario.

What you need: Whiteboard and marker. 10 index cards prepared before the lesson — each representing a "data packet" trying to enter a hospital server. Write different details on each card. Suggestions:

Card	Origin	File type	Size	Other detail
1	Unknown	.exe	15MB	Sent at 3am
2	Trusted doctor	.pdf	2MB	Normal hours
3	Unknown	.pdf	1MB	Normal hours
4	Trusted device	.exe	8MB	Software update
5	Unknown	.jpg	500KB	Normal hours
6	Trusted doctor	.exe	8MB	New device
7	Unknown	.docx	3MB	Normal hours
8	Hacker IP	.pdf	2MB	Normal hours
9	Trusted device	.pdf	4MB	Normal hours
10	Unknown	.exe	2MB	Normal hours

The point of the cards is that some threats are obvious (unknown origin, .exe file, 3am) and some are ambiguous (trusted doctor sending an .exe — is it a legitimate software update or a hacked account?). That ambiguity is the lesson.

Vocabulary on the board: Boolean logic · Conditional · AND / OR / NOT · Firewall · False positive · False negative

Role model: Claude Shannon (1916–2001) — mathematician who realised that George Boole's true/false logic system could be mapped perfectly onto electrical switches (on/off). He essentially invented the logical foundation of all modern computing. Every conditional statement students write today runs on the architecture he designed.

Phase 1 — Spark question · 0–2 min

Write on the board:

> *"Will passwords still exist in 30 years — or will AI just know whether you are allowed into your own bank account?"*

45 seconds in pairs, then 2 answers from the class. Accept everything — biometrics, device recognition, fear of being locked out by a glitch are all good directions. Do not resolve the question. Tell students: *"Whatever replaces passwords still needs to make a decision: let this person in, or block them. That decision is always built from the same logical building blocks. Today you are going to build them."*

Real world hook: Every time you load a webpage, send a message, or use an app, an invisible system called a firewall checks your data in milliseconds and decides whether to allow it or block it. That decision is made using the exact logic students will write today.

Phase 2 — Human logic gates · 2–8 min

Tell the whole class to stand up. Explain: *"Computers understand only two values: True and False — nothing in between. I am going to read rules out loud. If the rule is true for you, stay standing. If it is false for you, sit down. Do it instantly — a computer does not hesitate."*

Read these rules one at a time. After each one, briefly explain the operator being used before moving to the next:

Rule 1 — simple IF: *"IF you are wearing sneakers, stay standing. Everyone else sit down."* This is a basic conditional — one condition, two outcomes.

Rule 2 — AND: *"IF you are wearing a watch AND you have a dark-coloured top, stay standing."* Explain: AND means both conditions must be true. If either is false, the result is false. Ask a student who sat down: which condition was false for them?

Rule 3 — OR: *"IF you are wearing glasses OR something white, stay standing."* Explain: OR means at least one condition must be true. Only if both are false do you sit down.

Rule 4 — NOT: *"IF you are NOT wearing any jewellery, stay standing."* Explain: NOT flips the result. True becomes false, false becomes true.

After Rule 4 ask: *"What if the rule was: IF you are wearing sneakers AND NOT a watch — how many of you would be standing?"* Give students 10 seconds to work it out mentally, then check by having those students stand.

Teacher tip: Move through these quickly — the goal is to simulate the speed and precision of computer logic, not to dwell on each rule. If a student hesitates or gets it wrong, use it: *"A computer never hesitates and never makes that mistake. That precision is both the power and the limitation of Boolean logic."*

Phase 3 — Concept introduction · 8–13 min

Write on the board:

IF (condition) THEN (action) ELSE (different action)

Tell students: *"This is the core structure behind almost all software decisions. You saw it physically just now — the rule was the condition, standing or sitting was the action. Every app, every website, every security system runs on variations of this."*

Show how AND, OR, and NOT combine with IF/THEN to build more precise rules. Write these three examples on the board and read through each one:

IF (password = correct) AND (device = trusted)
THEN grant access
ELSE block and send alert

IF (post contains banned word) OR (account age < 1 day)
THEN flag for review

IF (user = verified) AND NOT (location = unusual)
THEN allow transaction

For each one, ask: *"What specific problem is this rule solving? What would go wrong if you removed one of the conditions?"*

On example 1: removing AND (device = trusted) means someone who stole your password from another device gets in. On example 3: removing NOT (location = unusual) means your bank never flags when your card is used in a different country.

Tell students: *"The challenge in security is not writing rules that block threats. It is writing rules that block threats without also blocking legitimate users. That balance is what you are about to test."*

Phase 4 — The firewall challenge · 13–24 min

Draw a large box on the board labelled **HOSPITAL SERVER**. Tell students: *"You are the cybersecurity team for a hospital. Malicious software disguised as normal data is trying to get through. You need to write the rules that decide what gets blocked and what gets through. But if you block legitimate data — a doctor's files, a software update for the MRI machine — the hospital stops functioning. Both failures have real consequences."*

Distribute the index cards randomly to students around the class.

Round 1 — Write one rule (3 minutes)

Ask the class to suggest one rule that blocks an obvious threat. Write it on the board together. Most classes will suggest something like:

IF file type = .exe THEN block

Test it immediately. Ask the student holding Card 4 (trusted device, .exe, software update) to stand up. Ask: *"Does your rule block this card?"* Yes. *"Is that a problem?"* Yes — that is a legitimate software update for the MRI machine. The hospital's scanner just went offline.

Name that outcome: *"This is called a false positive — you blocked something safe. In a hospital, a false positive could mean a doctor cannot access patient records during an emergency."*

Ask: *"What about Card 8 — unknown hacker IP, .pdf file? Does your rule block that?"* No. *"That is a false negative — a threat got through."*

Tell students: *"A good firewall minimises both. That requires more precise rules."*

Round 2 — Write a better ruleset (8 minutes)

Groups of three write a set of at least three connected rules using AND, OR, and NOT that are more precise than the single rule from Round 1. Their ruleset must correctly handle all 10 cards — blocking threats and allowing legitimate traffic through.

Walk around and push each group with one question: *"What if a trusted doctor logs in from a new device they have never used before — does your rule block them or let them through? Is that the right call?"* This is the hardest edge case and most groups will not have planned for it.

After 8 minutes, test the best ruleset from one group against all 10 cards. The class calls out BLOCK or ALLOW for each card based on the rules. Count false positives and false negatives.

Teacher tip: Perfect rulesets are rare in 8 minutes and that is fine. A group that writes a thoughtful ruleset with one remaining edge case they can articulate has understood the lesson. The goal is not a correct answer — it is the experience of discovering that making rules precise enough to handle all cases is genuinely difficult.

Phase 5 — Discussion · 24–28 min

Pick one question based on how the challenge went:

If the false positive/negative trade-off came up strongly: *"False positive: you blocked a safe user. False negative: you let a threat through. For a hospital, which is worse — and does the answer change depending on the specific situation? Is blocking a doctor's access to patient records during a routine check worse or better than blocking it during a cardiac emergency?"*

Follow-up: *"Is there a context where a false positive is worse than a false negative? Think about facial recognition used to unlock phones versus facial recognition used to identify suspects in a crowd."*

If the edge case about the doctor on a new device generated strong debate: *"Your rules had to handle a trusted person using an untrusted device. In security, this is called Zero-Trust — the principle that no person or device should be automatically trusted, even if they were trusted before. Is that a good principle? What does it cost in terms of usability — and who bears that cost?"*

Follow-up: *"Apply Zero-Trust to something non-digital. Should a school apply Zero-Trust to students — treating every student as a potential rule-breaker unless they prove otherwise every day? What are the consequences of that approach?"*

Exit reflection · 28–30 min

Students write one sentence:

> *"A good security rule needs to be _____ because _____."*

Take 2–3 answers. The best will include ideas like: specific enough to catch real threats without blocking safe users, tested against edge cases, updated as new threats appear. Write the sharpest one on the board next to Lesson 1's sentence.

Close: *"Next lesson we move from security to something more uncomfortable — what happens when the rules computers follow are not just imprecise, but unfair. Algorithms deciding who gets a loan, who gets a job interview, who gets flagged by police. The logic is the same as today. The stakes are higher."*

Fast finisher question: *"A Zero-Day Exploit is a cyberattack that uses a method nobody has written a rule against yet — it is brand new. Your entire ruleset is built on known threats. What does your firewall do when it encounters something it has never seen before? Is there any rule you could write that would protect against the unknown?"* No written answer needed. Worth discussing if time allows.

Teacher note: Did students independently identify that blocking .exe files catches the obvious threat but also catches the legitimate software update — without being told to look for that conflict? Did any group ask the question "what if a trusted person uses an untrusted device" before you raised it? And in the discussion, did students apply the false positive/false negative trade-off to a non-cybersecurity context — medicine, criminal justice, school discipline — without prompting? Those students are transferring the concept beyond the lesson. That transfer is the goal.

HS L3

Lesson 3

How do computers handle massive data? Loops and the climate crisis

Age group: 15–17 · **Duration:** 30 minutes · **Builds on:** Lessons 1–2 **SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can explain what a loop is, distinguish between a FOR loop and a WHILE loop, and write a plain-language loop that processes a dataset to extract meaningful information.

What you need: Whiteboard and marker. A list of 10 numbers written on the board before students arrive (see Phase 4). Paper and pencils.

Vocabulary on the board: Loop · FOR loop · WHILE loop · Array · Data parsing · Redundancy

Role model: Margaret Hamilton (born 1936) — lead software engineer for NASA's Apollo program. Her team wrote the code that landed humans on the Moon. The software had to process massive streams of real-time flight data flawlessly. A single inefficient loop could have meant mission failure. She is also credited with coining the term "software engineering" — arguing that writing code deserved the same rigour and respect as any other engineering discipline.

Phase 1 — Spark question · 0–2 min

Write on the board:

> *"Climate scientists need to analyse ocean temperature readings from 10,000 sensors, updated every hour, going back 40 years. That is roughly 3.5 billion data points. How does a computer actually do that — and what happens if the instructions are wrong?"*

45 seconds in pairs, then 2 answers from the class. Most students will say something like "it just processes it automatically" or "AI does it." Both point in the right direction. Land on: *"Yes — but that automation is not magic. Someone had to write the instructions that tell the computer to go through every single data point, one by one, and do something with it. Today we are going to write those instructions."*

Real world hook: Predicting hurricanes, heatwaves, and floods depends on processing billions of temperature and pressure readings. No human reads them manually. The loops

that automate this analysis are not just a programming technique — they are infrastructure that climate scientists depend on to give governments the warnings they need.

Phase 2 — The human loop · 2–8 min

Ask one student to come to the front. Hand them the marker and tell them: *"Draw a star on the board."* They draw one. *"Draw another one."* They draw another. *"Draw another one."* They draw another. Keep going until they look mildly annoyed — usually after 4 or 5.

Stop and ask the class: *"If I needed 10,000 stars — one for each temperature sensor in the ocean — how would I write that instruction more efficiently?"*

A student will say something like "just tell them to repeat it 10,000 times." Tell them: *"Exactly. That is a loop. Instead of writing 'draw a star' 10,000 times, you write it once and tell the computer how many times to repeat it. The result is identical. The code is one line instead of 10,000."*

Now make it more concrete. Write these two versions on the board side by side:

```
WITHOUT A LOOP:          WITH A LOOP:
Check sensor 1           FOR each sensor (1 to 10,000):
Check sensor 2              check the temperature
Check sensor 3           [done]
Check sensor 4
... (9,996 more lines)
```

Ask: *"Beyond saving time — what else is the problem with the left version?"* Push for: if there is a typo or error on line 4,847, it is almost impossible to find. If the number of sensors changes from 10,000 to 12,000, you have to rewrite thousands of lines. The loop version has one line to change. Errors hide in repetition — loops eliminate the repetition.

Phase 3 — Two types of loop · 8–13 min

Write on the board:

```
FOR loop  — use when you know exactly how many times to repeat
WHILE loop — use when you repeat until something changes
```

Explain each with a concrete example students will immediately recognise:

FOR loop: *"FOR each of the 30 students in this class, check whether they have submitted their homework."* You know exactly how many times to run it — 30. It stops automatically when it reaches the end.

Write: `FOR (each student in the class): check homework submitted → yes or no`

WHILE loop: *"WHILE the room temperature is below 18 degrees, keep the heating on."* You do not know how many times it will run. It depends on something changing in the real world — the temperature rising. It keeps going until the condition is no longer true.

Write: `WHILE (temperature < 18): keep heater running`

Ask: *"What happens if the temperature sensor breaks and always reads 10 degrees? What does the WHILE loop do?"* It runs forever — the heater never turns off. That is an infinite loop. In a home that is annoying. In a nuclear power station it is catastrophic. This is why the condition that stops a WHILE loop is one of the most important things a programmer writes.

Ask the class: *"Which loop would you use for each of these?"*

- Sending a notification to every user of an app when a new feature launches — FOR (you know exactly how many users there are)
- Keeping a self-driving car's brakes applied until the speed reaches zero — WHILE (you do not know how many brake pulses are needed)
- Checking every photo uploaded to a platform for policy violations — FOR (you have a fixed list of photos to check)
- Running a hospital ventilator until the patient can breathe independently — WHILE (you do not know how long that will take)

The ventilator example usually lands hard. That is intentional.

Phase 4 — Climate data challenge · 13–24 min

Write these 10 numbers on the board before the lesson starts, labelled as ocean temperature anomalies in degrees Celsius above the seasonal average:

-2, 4, 1, -5, 8, 3.2, 0, 6, -1, 5

Tell students: *"These are 10 days of ocean temperature anomaly readings from one sensor in the North Atlantic. Climate scientists flag any reading above 3 degrees as a potential early warning sign for extreme weather. Your job is to write a loop that reads through all 10 values and identifies the ones that need flagging."*

Round 1 — Write the loop together (3 minutes)

Build the solution on the board as a class. Ask:

- *"What type of loop should we use — FOR or WHILE?"* FOR — we know exactly how many readings there are.
- *"What does the loop do with each number?"* Check whether it is above 3.
- *"What does it do when it finds one above 3?"* Flag it — print it, highlight it, send an alert.

Write the result on the board:

```
FOR (each temperature reading in the list):
   IF (reading > 3)
```

THEN flag this reading as a warning
ELSE do nothing

Run through the 10 numbers manually as a class. Students call out WARNING or SAFE for each one. Flagged readings: 4, 8, 3.2 (just over the threshold — worth discussing), 6, 5.

Ask: *"3.2 is only slightly above 3. Should it be flagged with the same urgency as 8? What would you add to the loop to distinguish severity?"* Students will suggest adding more conditions — flag anything above 3 differently from anything above 6. This is the beginning of nested conditionals inside a loop, and it emerges naturally from the question.

Round 2 — Extend it in groups (6 minutes)

Groups of three extend the loop to handle two new requirements:

First: the scientists want the loop to stop as soon as it finds three warnings — because at that point they trigger a formal review process and do not need to keep reading. Ask: *"Does this change whether we use a FOR loop or a WHILE loop?"* Yes — if the loop stops based on a condition (three warnings found), it becomes a WHILE loop. Let groups wrestle with this transition.

Second: the scientists want the loop to calculate the average of all flagged temperatures — not just list them. Groups must add a step inside the loop that keeps a running total, then divides by the count at the end.

After 6 minutes, ask one group to read their solution aloud while the class traces it through the 10 numbers on the board. Check whether it correctly stops at three warnings and calculates the right average (4 + 8 + 3.2 = 15.2, divided by 3 = approximately 5.07 degrees).

Teacher tip: Groups will often forget to count how many warnings they have found — they write the loop that flags readings but not the variable that tracks the count. When their loop fails to stop at three, ask: *"How does the loop know it has found three warnings? Where is that information stored?"* This connects directly to Lesson 4's concept of variables — plant the word without defining it yet: *"You need somewhere to remember that count. We will build that properly next lesson."*

Phase 5 — Discussion · 24–28 min

Pick one question:

If the infinite loop example from Phase 3 generated strong reactions: *"We said a broken sensor could cause a WHILE loop to run forever. In software that controls physical infrastructure — heating systems, hospital equipment, aircraft — who is responsible for making sure the stop condition is always reachable? The programmer who wrote the loop? The engineer who installed the sensor? The company that sold the system? Or the regulator who approved it?"*

Follow-up: *"In 2018 and 2019, a feedback loop in Boeing's 737 MAX autopilot system repeatedly pushed the nose of the plane down in response to sensor data. Two crashes*

killed 346 people. The loop was working exactly as designed. What does that tell you about the difference between code that functions correctly and code that is safe?"

If the climate framing engaged the class: *"Your loop flagged five readings out of ten as warnings. But the threshold — anything above 3 degrees — was set by a scientist based on historical data. What if climate change means the historical baseline is no longer valid? What if 3 degrees used to be dangerous but is now normal — and the real danger starts at 5 degrees? Who updates the loop, how often, and what happens in the gap between the threshold being wrong and someone noticing?"*

Follow-up: *"Is this a technology problem, a science problem, or a governance problem? Can you separate them?"*

Exit reflection · 28–30 min

Students write one sentence:

> *"A FOR loop is better than a WHILE loop when _____, and a WHILE loop is better when _____."*

Take 2–3 answers. The best will include: FOR when you know the count in advance, WHILE when you stop based on a condition. Write the clearest one on the board next to the sentences from Lessons 1 and 2.

Close: *"You have now written loops that process data. Next lesson we add memory — the ability for a program to remember information as it runs, update it, and use it to make decisions. Without that, your climate loop can flag readings but cannot count them, compare them, or remember what it found. That memory is called a variable — and it is what turns a loop into a system."*

Fast finisher question: *"Your loop processes 10 readings in sequence — one at a time. But modern computers have multiple processors that can run loops simultaneously. If you split the 10 readings across 5 processors running in parallel, you could theoretically finish in one fifth of the time. What could go wrong if two processors try to update the same warning count at exactly the same moment?"* This introduces the concept of race conditions in parallel computing — a genuinely hard problem that students do not need to solve, just sit with.

Teacher note: Did students independently ask how the loop knows when it has found three warnings — identifying the need for a counter variable before you introduced it? Did the ventilator or Boeing example shift the discussion from technical to ethical without prompting? And did any student question whether the threshold of 3 degrees is itself a variable that should be updated — recognising that the rules a loop follows are a human choice, not a natural fact? That last observation is the most sophisticated signal from this lesson.

HS L4

Lesson 4

How do large teams build software? Breaking problems into black boxes

Age group: 15–17 · **Duration:** 30 minutes · **Builds on:** Lessons 1–3 **SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can explain how abstraction and functions allow large teams to build complex software by breaking it into independent, reusable components — and apply this thinking to design a real-world app architecture.

What you need: Whiteboard and marker. Paper and pencils. No other materials needed.

Vocabulary on the board: Function · Abstraction · Module · Input · Output · Black box

Role model: Adele Goldberg (born 1945) — computer scientist who co-developed Smalltalk, one of the first programming languages built entirely around modular, reusable components. Her work is the direct ancestor of how almost every modern app is built. Before her contributions, software was written as one enormous block of instructions. She helped establish the idea that complex systems should be built from independent, interchangeable parts — the same way physical engineering had worked for centuries.

Phase 1 — Spark question · 0–2 min

Write on the board:

> *"When you press 'Share' on Instagram, how many lines of code do you think run? And how many engineers do you think wrote them?"*

45 seconds in pairs, then 2 answers from the class. Students will guess anywhere from hundreds to millions of lines. Tell them: *"A modern social media app has tens of millions of lines of code, written by thousands of engineers across multiple countries over many years. No single person understands all of it. So how does it work without falling apart? That question is what this lesson is about."*

Real world hook: The Apollo Guidance Computer that landed humans on the Moon in 1969 had 145,000 lines of code — considered enormous at the time. A modern iPhone has roughly 12 million. WhatsApp, at its peak, had a backend maintained by just 50 engineers serving 450 million users. The reason a small team could run a system that large is the same reason the Apollo code worked despite its complexity: it was broken into independent components that could be built, tested, and fixed separately. That principle is what students will design with today.

Phase 2 — The human machine · 2–8 min

Assign three students simple, memorable tasks. Tell them clearly what their task is and that they should do it immediately when called by name:

- Student A: clap twice
- Student B: say "processing complete"
- Student C: stand up, turn around, sit back down

Tell the class: *"These three students are now functions. A function is a named set of instructions that does one specific job. I am the main program."*

Now call them in different orders:

"Run Student B." → Student B responds. *"Run Student A."* → Student A responds. *"Run Student C, then run Student B."* → Both respond in order. *"Run Student A three times."* → Student A claps twice, three times in a row.

Then ask the class: *"Did I need to know how to clap in order to make Student A clap?"* No. *"Did I need to understand what Student C was doing in order to use them?"* No. *"All I needed to know was the name of the function and what it would produce. The internal details were hidden from me."*

Write on the board:

```
FUNCTION name → you call it by name
INPUT       → what you give it
OUTPUT      → what it gives back
BLACK BOX   → you do not need to know what happens inside
```

Tell students: *"This hiding of internal complexity behind a simple name is called abstraction. It is one of the most important ideas in all of software engineering — and in engineering generally. You use abstraction every time you press a button without knowing the circuitry inside it."*

Ask: *"What abstractions are you using right now in this classroom without thinking about them?"* Students will identify: the lights, the door handle, their phones, the projector. Every one of those is a complex system with a simple interface. That is abstraction.

Phase 3 — Concept introduction · 8–13 min

Tell students: *"A real app is too large and too complex for one person to build. The way large teams solve this is by agreeing on what each function's input and output will be — then letting different people build the inside independently."*

Draw this on the board:

```
        ┌──────────────────────────┐
INPUT → │   [black box]            │ → OUTPUT
        │   (internal logic        │
        │    hidden inside)        │
        └──────────────────────────┘
```

FUNCTION NAME

Give a concrete example — the food waste app students will design in the challenge. Write on the board:

FUNCTION: findNearestShelter()
INPUT: user's location
OUTPUT: name and address of nearest shelter with capacity
INSIDE: searches a database, calculates distances,
filters by availability — nobody else needs to know this

Tell students: *"Team 1 builds findNearestShelter(). Team 2 builds claimFood(). Team 3 builds sendNotification(). Each team only needs to know the name, input, and output of the other teams' functions — not what is inside them. If findNearestShelter() breaks, the claimFood() function still works. If Team 2 improves their function, Team 1 does not need to change anything. They are independent."*

Ask: *"What goes wrong if two teams build functions that are supposed to work together but never agreed on what the input and output format would be?"* Students will identify: the functions cannot connect — one produces data in a format the other cannot read. This is one of the most common real causes of software failures in large organisations. It is not a coding problem. It is a communication problem.

Tell students: *"Most large software failures are not caused by bad code. They are caused by teams that did not communicate clearly about the interfaces between their components. Abstraction only works if the agreement about inputs and outputs is precise."*

Phase 4 — App architecture challenge · 13–24 min

Tell students: *"You are going to design the architecture — not the code, not the design — of a real app. The app connects supermarkets and restaurants that have surplus food at the end of the day with nearby homeless shelters that need it. Your job is to identify the core functions this app needs and map how they connect."*

Groups of three. Paper and pencils.

Step 1 — Identify the functions (4 minutes)

Groups brainstorm and write down the core functions the app needs. Each function should be named, have a clear input and output, and do one specific job. Push groups to write at least five.

Expect groups to identify functions like:

- loginUser() — input: email and password → output: verified user identity
- listAvailableFood() — input: location and time → output: list of nearby surplus food items
- findNearestShelter() — input: food location → output: nearest shelter with available capacity

- claimFood() — input: shelter ID and food item ID → output: confirmation and pickup time
- sendNotification() — input: recipient and message → output: push notification delivered

Walk around and push groups to be specific about inputs and outputs — not just "it gets the food" but what information exactly goes in and what comes back. If a group is struggling, ask: *"What does this function need to know to do its job? And what does the function that calls it need to receive back?"*

Step 2 — Map the connections (5 minutes)

Groups draw arrows between their functions showing which function calls which and in what order. This is the architecture of the app — how the components fit together.

Ask groups: *"If your findNearestShelter() function breaks tonight — a database error, a server going down — which other functions break with it? Which ones still work independently?"* Groups should trace the dependencies in their map and identify which functions are critical path — if they fail, the whole system fails — and which are optional.

Step 3 — Find the gap (2 minutes)

Tell groups: *"Every architecture has at least one function that nobody thought of until something went wrong. Look at your map — what happens when a shelter's capacity changes between when the food is claimed and when it arrives? What function handles that? Did you build it in?"*

Most groups will not have included a real-time capacity update function. Ask them to add it and show where it fits in their map. This is the exercise of finding missing components — one of the most important skills in system design.

Teacher tip: The goal is not a perfect app design. It is the experience of thinking systematically about how complex systems are structured. Groups who identify that their loginUser() function connects to nearly every other function — making it a single point of failure — are thinking at a genuinely sophisticated level. If a group reaches that insight, ask them: *"How would you redesign the architecture to reduce that risk?"*

Phase 5 — Discussion · 24–28 min

Pick one question:

If the communication point landed strongly in Phase 3: *"We said most large software failures are caused by teams not agreeing on interfaces — not by bad code. Can you think of a real example where a system failed not because the individual parts were broken but because they were not designed to work together?"*

Follow-up: *"In 1999 NASA lost a $327 million Mars orbiter because one engineering team used metric units and another used imperial units. The code in both teams worked perfectly.*

The interface between them was never specified precisely enough. What does that tell you about where the hardest problems in large-scale engineering actually are?"

If the food waste app generated strong engagement: *"Your app connects surplus food with shelters. But it also creates a dataset — which supermarkets have surplus, how often, which shelters collect most, which areas have the most food insecurity. That data is a byproduct of the system you designed. Who should own it? Who should be allowed to access it? And what could go wrong if a company bought your app and decided to sell that data?"*

Follow-up: *"Did you build a function to handle data privacy? Did you build one to delete user data on request? If not — is that a missing function, or a deliberate choice?"*

Exit reflection · 28–30 min

Students write one sentence:

> *"Abstraction is useful because _____, but it becomes dangerous when _____."*

Take 2–3 answers. Strong answers will include: useful because it allows teams to work independently and hide complexity; dangerous when the agreement about inputs and outputs is imprecise, or when nobody understands the system as a whole and something breaks. Write the sharpest one on the board.

Close: *"Next lesson we go inside the black box — and look at what happens when the data going in is the problem. Not broken code, not a missing function, but data that is incomplete, outdated, or systematically biased. And what that means for the people on the receiving end of decisions made by systems built on that data."*

Fast finisher question: *"Your app has a loginUser() function that nearly every other function depends on. In security terms this is called a single point of failure — if it breaks, everything breaks. Large companies handle this by running multiple identical copies of critical functions simultaneously, so if one fails another takes over instantly. This is called redundancy. But redundancy costs money. How would you decide which functions in your architecture deserve redundancy and which do not — and who should make that decision?"*

Teacher note: Did groups independently identify that loginUser() is a single point of failure without being prompted? Did any group ask who owns the data the app generates — before you raised it in the discussion? And did students stay genuinely abstracted in their architecture — describing functions by their inputs and outputs without trying to write the internal logic — or did they keep slipping into describing how the function works internally? The ability to reason about a system at the interface level without needing to know the internal details is the core skill of this lesson. Students who maintain that abstraction consistently are demonstrating exactly the right kind of systems thinking.

HS L5

Lesson 5

How do systems prioritise? Queues, stacks, and emergency triage

Age group: 15–17 · **Duration:** 30 minutes · **Builds on:** Lessons 1–4 **SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can explain the difference between a Stack and a Queue, describe when each is appropriate, and design a Priority Queue algorithm for a real-world scenario where order of arrival is not the same as order of importance.

What you need: Whiteboard and marker. Four physical books or notebooks. Paper and pencils.

Vocabulary on the board: Stack · Queue · LIFO · FIFO · Priority Queue · Severity score

Role model: Annie Easley (1933–2011) — NASA computer scientist and mathematician who developed algorithms for energy distribution systems, including code for the Centaur rocket stage. Her work required deciding in real time which systems received power first when resources were limited. She was one of the first African American scientists at NASA, and spent her career solving problems where the order of operations was not just a technical question but a question of what survives and what does not.

Phase 1 — Spark question · 0–2 min

Write on the board:

"A self-driving car's sensors detect three things at once: a stop sign 20 metres ahead, a pedestrian stepping off the kerb 5 metres away, and a text notification from the passenger's phone. The car's processor cannot handle all three simultaneously. Which does it process first — and who decided that?"

45 seconds in pairs, then 2 answers from the class. Most students will say the pedestrian — which is correct. Push further: *"How does the system know that? Did a programmer write 'pedestrians are more important than stop signs'? Or does the system figure that out on its own?"* Do not answer yet. Tell students: *"The answer is that someone did write that. Not in those words — but in a data structure that assigns weights to different inputs and processes them in order of importance. That structure is what today is about."*

Real world hook: Every computer processor, hospital server, and air traffic control system receives more requests than it can handle simultaneously. The structure that decides what

gets processed first is not neutral — it encodes a set of values about what matters more. Understanding how that structure works is understanding how life-or-death decisions get made at machine speed.

Phase 2 — Books and queues · 2–10 min

The stack — physical demonstration (3 minutes)

Ask one student to come to the front. Hand them a book and say: *"Hold this."* Stack a second book on top, then a third, then a fourth. Ask the class: *"If I need to get the very first book I handed them — the one at the bottom — what has to happen first?"*

The answer is obvious: everything on top has to come off first. The last book added is the first one that can be removed.

Write on the board: **STACK — Last In, First Out (LIFO)**

Give two real examples students use daily without realising it:

- The undo button in any app — it reverses the most recent action first, not the oldest
- The back button in a browser — it takes you to the last page you visited, not the first

Ask: *"Why does undo have to be LIFO? What would happen if undo reversed your oldest action first?"* Students will immediately see the problem — you would lose all your recent work every time you pressed undo. The structure is not arbitrary. It matches the logic of the problem it solves.

The queue — contrast (2 minutes)

Now ask students to imagine a printer connected to 30 laptops. Everyone sends a document to print at the same moment.

Ask: *"In what order should the documents print?"* Most will say first come, first served — whichever was sent first prints first.

Write on the board: **QUEUE — First In, First Out (FIFO)**

Give real examples:

- A printer spool — documents print in the order they were sent
- A customer service call queue — first caller gets answered first
- Packets of data travelling across the internet — processed in arrival order

Ask: *"Is FIFO always fair? Is 'first to arrive' always the right priority?"* Students will immediately think of counterexamples. Let them surface the problem themselves before naming it.

The problem with FIFO in high-stakes contexts (2 minutes)

Write this scenario on the board:

1. 9:00 AM — Patient A arrives: sprained ankle
2. 9:05 AM — Patient B arrives: chest pain
3. 9:10 AM — Patient C arrives: broken arm

9:15 AM — Patient D arrives: cannot breathe

Ask: *"If a hospital emergency room used a standard FIFO queue — treating patients strictly in arrival order — what happens to Patient D?"* They wait 15 minutes while three less urgent patients are seen first. In a real cardiac or respiratory emergency, that is potentially fatal.

Tell students: *"FIFO is fair in the sense that everyone follows the same rule. But fair does not always mean right. Sometimes the order that saves the most lives is not the order that respects the queue. The data structure that handles this is called a Priority Queue — and designing one requires making explicit decisions about what matters more than what."*

Phase 3 — Priority queues · 10–15 min

Write on the board:

4. PRIORITY QUEUE:
5. - Each item has a severity score as well as an arrival time
6. - The queue is constantly re-sorted by severity score, not arrival time
7. - When a new item arrives with a high severity score,

   it moves to the front — ahead of items that arrived earlier

Tell students: *"This sounds simple. But writing the algorithm that re-sorts the queue every time a new item arrives — while the system is still processing the current front of the queue — is one of the genuinely hard problems in computer science. Today you are going to design the logic, not the code."*

Ask: *"What information does a priority queue algorithm need to do its job?"* Walk students through identifying:

- The arrival time of each item (so ties can be broken — two patients with severity 8 are treated in arrival order)
- The severity score of each item (the primary sort key)
- A rule for what happens when scores are equal
- A rule for what happens when a new high-severity item arrives while a lower-severity item is already being processed — do you interrupt mid-treatment or finish the current patient first?

That last question is the most interesting one and has no single correct answer. In computing it is called preemption — whether a high-priority task can interrupt a lower-priority one mid-execution. Write the word on the board without requiring students to memorise it.

Phase 4 — Emergency triage challenge · 15–24 min

The patient table from the spark question is the dataset. Rewrite it clearly on the board:

8. Patient A: arrived 9:00, severity 2 — sprained ankle
9. Patient B: arrived 9:05, severity 9 — chest pain
10. Patient C: arrived 9:10, severity 5 — broken arm

Patient D: arrived 9:15, severity 10 — cannot breathe

Tell groups of three: *"Write the algorithm that a hospital triage system uses to maintain and update this queue. Your algorithm must handle three situations: the initial ordering when patients first arrive, what happens when a new patient arrives with a higher severity than the current front of the queue, and what happens when two patients have the same severity score."*

Round 1 — initial ordering (3 minutes)

Ask groups to write the correctly ordered queue based on severity alone. Expected answer: D (10), B (9), C (5), A (2). This is straightforward — it establishes the baseline.

Ask: *"Patient D arrived last but is treated first. Patient A arrived first and is treated last. Is this system fair to Patient A?"* Take 2–3 quick answers. There is no single correct response — this is a genuine ethical question embedded in a data structure design. Tell students: *"Every priority queue encodes a value judgement about what matters more. In a hospital, most people agree that clinical urgency should override arrival time. But who decided that? And are there contexts where you might disagree?"*

Round 2 — a new patient arrives (5 minutes)

Tell groups: *"At 9:20 AM, a fifth patient arrives: Patient E, severity 7 — deep laceration, significant blood loss. Write the updated algorithm that re-sorts the queue. Then write what happens if Patient E's condition deteriorates and their severity score is updated from 7 to 10 while Patient B is already being treated."*

Groups work through both scenarios. The first is a straightforward insertion into the correct position. The second requires them to address preemption — do you interrupt Patient B's treatment to start Patient E's?

Walk around and push groups with: *"Does your algorithm know that a treatment is in progress? Does it have access to that information, or only to the queue? If it cannot see what is happening in the treatment room, how does it handle the preemption decision?"*

This question introduces the concept of system state — the algorithm does not just need to know the queue, it needs to know the current state of the whole system. That is a

significantly more complex problem and students should feel the complexity rather than resolve it neatly.

Round 3 — the edge case (2 minutes)

Tell groups: *"At 9:25 AM, six more patients arrive simultaneously, all with severity 8. Write the tie-breaking rule your algorithm uses. Then tell me: is there a scenario where your tie-breaking rule produces an outcome that feels wrong?"*

Most groups will use arrival time as a tie-breaker — first to arrive among equals gets treated first. Ask whether that is always right: what if two patients both have severity 8 but one is a child and one is an adult? What if one is a healthcare worker who needs to return to treating other patients? These are real debates in medical ethics, and the point is not to resolve them but to recognise that the tie-breaking rule in the algorithm is a policy decision, not a technical one.

Phase 5 — Discussion · 24–28 min

Pick one question:

If the preemption question generated strong debate: *"We asked whether a high-severity new arrival should interrupt a treatment already in progress. In computing, this is called preemption — a high-priority task interrupting a lower-priority one. Can you think of examples in everyday life where preemption is clearly right, and examples where it is clearly wrong — and what principle separates the two cases?"*

Follow-up: *"An operating system on your laptop uses preemption constantly — high-priority processes interrupt low-priority ones thousands of times per second. You never notice because each interruption lasts microseconds. But in a surgical robot performing a procedure, preemption could be fatal. How should the designer of a safety-critical system decide whether to allow preemption?"*

If the fairness question engaged the class: *"Priority queues encode value judgements — clinical urgency matters more than arrival time. That seems obvious in a hospital. But the same data structure is used in content recommendation algorithms — some posts get shown first because they score higher on an engagement metric. Who set those scores? What values do they encode? And who is disadvantaged by a system that prioritises engagement over accuracy or wellbeing?"*

Follow-up: *"Is there a meaningful difference between a priority queue that saves lives by deprioritising a sprained ankle, and a priority queue that makes money by deprioritising accurate news in favour of outrage? Both are just data structures following their assigned weights. The difference is entirely in who set the weights and why."*

Exit reflection · 28–30 min

Students write one sentence:

> *"A priority queue is more powerful than a standard queue, but that power comes with the responsibility to _____."*

Take 2–3 answers. Strong answers will include: decide who sets the weights and why, acknowledge that the scoring system encodes values, test for cases where the priority ordering produces an unjust outcome. Write the clearest one on the board.

Close: *"Next lesson we look at what happens when the data going into these systems is the problem — not the structure, not the algorithm, but the information itself. Incomplete data, biased data, data that reflects historical inequalities. And what that means for people whose lives are shaped by systems built on that data."*

Fast finisher question: *"A hospital's triage algorithm has been running for 10 years. Researchers analysing its outputs notice that patients from certain postcodes consistently receive lower severity scores for the same presenting symptoms — meaning they wait longer and receive treatment later. The algorithm has no variable for postcode. How could this pattern have emerged? And once you discover it, what are your options — fix the algorithm, replace it, or something else?"* This is a real documented phenomenon in healthcare AI. No answer required — it is a question to carry into Lesson 6.

Teacher note: Did groups identify that their algorithm needs access to system state — not just the queue — in order to handle preemption? Did any student connect the priority queue's weight-setting to a policy decision rather than a technical one without being prompted? And did the fast finisher question about postcode bias produce a genuine reaction — did students immediately try to explain the mechanism, or did they dismiss it as impossible since postcode is not a variable? Students who attempt to trace how bias could emerge from a system that does not explicitly encode it are doing exactly the thinking that Lesson 6 builds on directly.

HS L6

Lesson 6

How do computers learn? Training the fraud detector

Age group: 15–17 · **Duration:** 30 minutes · **Builds on:** Lessons 1–5 **SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can explain the difference between rule-based programming and machine learning, identify features and assign weights to train a simple fraud detection model, and recognise at least one way a model built on limited data produces unfair outcomes.

What you need: Whiteboard and marker. Paper and pencils. No other materials needed.

Vocabulary on the board: Machine learning · Feature · Weight · Threshold · Training data · False positive · False negative

Role model: Fei-Fei Li (born 1976) — computer scientist and professor at Stanford who built ImageNet, a database of over 14 million labelled images that became the foundation for modern AI image recognition. Before ImageNet, AI could not reliably identify a cat in a photo. After training on her data, it could. Her key insight was not a better algorithm — it was better, more diverse training data. She showed that what a machine learns depends entirely on what you show it. Today students will experience that dependency directly.

Phase 1 — Spark question · 0–2 min

Write on the board:

> *"Your bank blocks your card when someone tries to use it fraudulently — even when the transaction looks normal and the thief has your PIN. How does it know?"*

45 seconds in pairs, then 2 answers from the class. Students usually guess: a programmer watches transactions, or there are strict rules like "block all overseas purchases." Tell them: *"If banks used strict IF/THEN rules from Lesson 2, they would block your card every time you travelled abroad or bought something at an unusual time. They do not — because they use something more sophisticated. Today you are going to build it."*

Real world hook: Visa processes around 65,000 transactions per second globally. No human reviews them. No static rule set could distinguish the thousands of legitimate unusual transactions from the fraudulent ones without blocking enormous numbers of real customers. The system that does this uses the same principle you are about to design — and it makes consequential decisions about people's access to their own money millions of times a day.

Phase 2 — Feature extraction · 2–8 min

Hold up any everyday object — a pen, a water bottle, a phone. Ask: *"If I wanted a computer to recognise this object, I could not just show it a photo and say 'this is a pen.' I would have to describe it in terms the computer can measure and compare. What would I tell it to look for?"*

Take answers from the class. Guide toward specific, measurable properties: cylindrical shape, approximately 15cm long, hard plastic surface, small diameter, tapered at one end, contains ink. Write them on the board as a list.

Tell students: *"These specific, measurable properties are called features. Feature extraction — deciding which properties of something are worth measuring — is one of the most important decisions in machine learning. The features you choose determine what the model can and cannot learn."*

Now ask: *"Are all those features equally important for identifying a pen? If you only knew one thing about an object, which feature would be most useful?"* Students will debate this. Some will say shape, some will say it contains ink. Tell them: *"In machine learning, the importance of each feature is represented by a number called a weight. A higher weight means the model pays more attention to that feature when making a decision. Setting those weights correctly is the hard part — and it is what you are going to do today."*

Write on the board:

EXPLICIT RULES (Lessons 1–4):
Human writes the rules → computer follows them

MACHINE LEARNING:
Human provides examples → computer finds the rules itself
→ then uses those rules to make decisions about new cases

Tell students: *"The shift from the left column to the right is one of the most significant changes in the history of computing. It means computers can now handle problems too complex for humans to write rules for — like recognising faces, understanding speech, or detecting fraud. But it also means the computer's decisions are only as good as the examples it learned from. Garbage in, garbage out — but at scale, and invisibly."*

Phase 3 — The fraud detector · 8–24 min

Draw this table on the board:

Transaction	Location	Amount	Time	Outcome
1	Local	€8	9:00 AM	✓ Safe
2	Local	€1,200	2:00 PM	✓ Safe
3	Overseas	€8	3:00 AM	✗ Fraud
4	Overseas	€420	3:15 AM	✗ Fraud

Tell students: *"These four transactions are your training data — the examples the model learns from. Your job is to build a scoring system that would correctly identify transactions 3*

and 4 as fraud without incorrectly blocking transactions 1 and 2. You have three features to work with: location, amount, and time."

Round 1 — Identify the signal (3 minutes)

Ask groups of three to look at the data and identify which feature appears to be the strongest indicator of fraud. They should write their reasoning — not just the answer.

Most groups will identify location or time. The answer is actually that the combination matters more than any single feature — transaction 1 is overseas AND at 3am AND low value, which matches the fraud pattern, while transaction 2 is local AND large amount, which does not match. Push groups to notice that amount alone is not a good predictor (transactions 2 and 4 are both large).

Round 2 — Assign weights and set a threshold (8 minutes)

Groups design a scoring system. For each feature condition, they assign a weight from 1 to 10 representing how strongly that condition suggests fraud. They then set a threshold — a total score above which the transaction is blocked.

Write the framework on the board to structure their work:

FEATURE	CONDITION	WEIGHT (1–10)
Location	Overseas	?
Time	Between 11pm–5am	?
Amount	Over €500	?

THRESHOLD: block transaction if total score exceeds ?

Groups fill in the weights and threshold, then test their model against all four training transactions. It must correctly classify all four — blocking 3 and 4, allowing 1 and 2 — before they move on.

Walk around and ask each group: *"How did you decide on that weight? What is your reasoning for giving location a higher weight than time — or vice versa?"* Push them to articulate the logic, not just produce numbers.

Round 3 — Test against new cases (5 minutes)

Once groups have a working model, present three new transactions they have not seen before:

Transaction 5: Local / €650 / 3:30 AM
Transaction 6: Overseas / €25 / 2:00 PM
Transaction 7: Overseas / €800 / 4:00 AM

Groups apply their scoring system to each new transaction and decide: block or allow?

Then reveal the actual outcomes: Transaction 5 is safe (a night shift worker buying groceries). Transaction 6 is safe (a tourist buying lunch). Transaction 7 is fraud.

Ask groups to check their model's performance:

- Did it correctly allow 5 and 6?
- Did it correctly block 7?
- If it blocked 5 or 6 incorrectly — that is a false positive. The real customer was denied access to their own money.
- If it allowed 7 — that is a false negative. The fraud went through.

Ask: *"Which error is worse — and does the answer depend on who you are? If you are the bank, false negatives cost money. If you are the customer blocked at 3am needing emergency cash, the false positive could be far more serious."*

Teacher tip: Most groups will struggle to get all three new cases right. That is exactly the point. Their model was trained on four examples. Real fraud detection models train on millions. The lesson is not that students designed a bad model — it is that any model trained on limited data will make mistakes on cases it has not seen. The question is not whether the model makes mistakes, but who bears the cost of those mistakes.

Phase 5 — Discussion · 24–28 min

Pick one question:

If the false positive/negative trade-off generated strong reactions: *"Your model blocked a real customer at 3am who needed emergency cash. The bank might call this an acceptable false positive rate — statistically rare, easily resolved by calling the bank. The customer might call it an unacceptable failure. Who should get to define what an acceptable error rate is — the company that built the system, the regulator, or the people affected by it?"*

Follow-up: *"Research has shown that fraud detection models trained primarily on transaction data from certain demographics perform less accurately for other demographics — they generate higher false positive rates for some groups than others. The model has no variable for ethnicity or nationality. How could that pattern emerge — and once you know it exists, what are your obligations?"*

If the training data limitation was the most striking insight: *"Your model learned from four transactions. A real model learns from millions. Does more data always mean a better model — or can a model trained on millions of examples still be systematically wrong? What would have to be true about those millions of examples for the model to be reliably fair?"*

Follow-up: *"If the millions of historical transactions your model trained on reflect a period when certain groups were disproportionately flagged by human fraud analysts — because of bias in those analysts' decisions — what does your model learn? And how would you even detect that the model had learned the bias rather than the actual signal?"*

Exit reflection · 28–30 min

Students write one sentence:

> *"Machine learning is more powerful than explicit rules when _____, but more dangerous when _____."*

Take 2–3 answers. Strong answers: more powerful when the problem is too complex for humans to write rules for, or when patterns exist in data that humans cannot see; more dangerous when the training data reflects historical biases, when the model makes decisions affecting people's lives, or when nobody can explain why it made a specific decision. Write the sharpest one on the board.

Close: *"Next lesson we look at the infrastructure that makes all of these systems possible — and what happens when it fails. Not a bug in the code, not bias in the data, but the physical and organisational systems that the software depends on. And what the consequences look like when those systems go down at scale."*

Fast finisher question: *"Your fraud detection model produces a score for each transaction and blocks it if the score exceeds the threshold. But it cannot explain why a specific transaction was blocked — it can only say the score was too high. This is called the black box problem in machine learning. Should a bank be allowed to deny you access to your money using a decision it cannot explain to you? In 2021, the EU introduced the right to an explanation for automated decisions affecting individuals. How would you build a fraud detection system that is both accurate and explainable — and is there a trade-off between the two?"*

Teacher note: Did groups articulate the reasoning behind their weight assignments — connecting the numbers to a real argument about which features matter more — or did they assign weights arbitrarily until the model worked? Did any group notice unprompted that their training data only contained four examples and question whether that was enough to generalise? And in the discussion, did any student connect the demographic bias question to the postcode bias question from Lesson 5's fast finisher — recognising that the same pattern of emergent bias appears across different systems built on historical data? That connection across lessons is one of the strongest signals of genuine understanding in this program.

HS L7

Lesson 7

Can data be unfair? Auditing algorithms for bias

Age group: 15–17 · **Duration:** 30 minutes · **Builds on:** Lessons 1–6 **SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students can identify hidden bias in a dataset, explain how proxy variables carry discriminatory patterns into algorithms, and write a data policy that addresses specific sources of unfairness — while recognising why fixing a biased AI is harder than fixing a biased rule.

What you need: Whiteboard and marker. Paper and pencils.

Vocabulary on the board: AI bias · Training data · Proxy variable · Algorithmic fairness · Audit · Filter bubble

Role model: Joy Buolamwini (born 1989) — computer scientist and researcher at MIT who founded the Algorithmic Justice League after discovering that major commercial facial recognition systems from IBM, Microsoft, and Amazon had significantly higher error rates for darker-skinned women than for lighter-skinned men — up to 34 percentage points worse. The systems had not been programmed to discriminate. They had simply been trained predominantly on lighter-skinned male faces, so that is what they learned to recognise accurately. Her research forced three major technology companies to change their products and sparked legislation in several US cities banning facial recognition in law enforcement. She demonstrated that auditing AI for bias is not a political act — it is an engineering responsibility.

Phase 1 — Spark question · 0–2 min

Write on the board:

> *"If an AI is trained on historical data, does it inevitably repeat historical mistakes — or can it do better than the humans who created that data?"*

45 seconds in pairs, then 2 answers from the class. Students are often split: some say yes, it is stuck with history; others say the AI could find fairer patterns than humans did. Both positions have merit. Tell students: *"You have both identified something real. Today we are going to look at exactly how historical patterns get encoded into algorithms — and how hard it is to get them out once they are in."*

Real world hook: In 2018 Amazon scrapped an AI hiring tool they had spent four years developing after discovering it systematically ranked women lower than men for technical roles. The system had not been programmed with any rule about gender. It had been trained on 10 years of successful hire data — from a period when the tech industry hired

predominantly men. The AI learned that pattern and applied it. The code was not sexist. The data was. And the data reflected a world that was.

Phase 2 — Proxy variables · 2–8 min

Write these three criteria on the board:

Plays polo
Subscribes to a sailing magazine
Has a second home

Tell students: *"An AI hiring tool has been given these three features as positive indicators of a high-performing employee. The dataset showed that top performers had these traits. What kind of person is this AI actually selecting for?"*

Take answers. Students will quickly identify: wealthy people. Then ask: *"Is wealth listed anywhere in the dataset? Is social class a variable the AI is using?"* No. *"So how is it filtering for wealth?"*

Name it: *"Those three traits are proxy variables. They do not directly measure what they appear to measure — job performance — but they strongly correlate with a characteristic the system is not supposed to use. The AI is not explicitly filtering for wealth or class. It is filtering for things that are statistically associated with wealth and class. The bias is invisible in the code. It is only visible when you look at the real-world outcomes."*

Now give a more serious example. Tell students: *"Zip codes — postcodes — are one of the most common proxy variables in algorithmic decision-making. Postcode strongly correlates with income, which correlates with race and ethnicity due to historical patterns of housing segregation and economic inequality. An algorithm that uses postcode as a variable — to predict credit risk, recidivism, or school performance — may have no variable for race anywhere in its code, and still produce racially discriminatory outcomes."*

Ask: *"How would you even detect that this was happening — if the discriminatory variable is never mentioned in the algorithm?"* This is the core challenge of algorithmic auditing, and it is the skill students will practice in the challenge.

Phase 3 — How bias enters and why it is hard to remove · 8–13 min

Write on the board:

TRADITIONAL PROGRAM:
Bias lives in the rules → find the rule → delete it → fixed

MACHINE LEARNING MODEL:
Bias lives in the weights → weights are distributed across
thousands of parameters → there is no single rule to delete
→ fixing it usually means discarding the model entirely
and retraining from new data

Explain the distinction clearly. In a traditional rule-based system — like the firewall from Lesson 2 — if someone writes a discriminatory rule, you can find it, read it, and remove it. The bias is explicit and locatable.

In a machine learning model, the bias is encoded across thousands of numerical weights — the importance scores the model learned from the data. There is no single line of code that says "penalise women." The bias is distributed throughout the model's mathematical structure in a way that is often impossible to separate from the legitimate patterns it also learned. Removing one without removing the other is like trying to remove salt from seawater.

Tell students: *"This is why Joy Buolamwini's work matters. She did not just identify that bias existed — that had been suspected for years. She quantified it precisely, across multiple commercial systems, in a way that could not be dismissed. And she showed that the only real fix was not a patch to the existing model but a fundamental change to the data it was trained on."*

Ask: *"If you could not fix a biased AI without scrapping it entirely and starting over — and scrapping it would cost millions of euros and years of work — what pressure does that create on organisations to keep using a system they know is biased? Who in that organisation would raise the concern, and who would have the incentive to ignore it?"*

Phase 4 — The audit · 13–24 min

Write this dataset on the board. Tell students it represents the training data for a new AI hiring tool — the historical records of high-performing employees at a large company:

Employee A: Postcode 1071 / Commutes by car / Graduated 2021 / Hobbies: golf, sailing
Employee B: Postcode 1072 / Commutes by car / Graduated 2020 / Hobbies: tennis, travel
Employee C: Postcode 1071 / Commutes by car / Graduated 2022 / Hobbies: skiing, sailing
Employee D: Postcode 1071 / Commutes by car / Graduated 2021 / Hobbies: cycling, golf
Employee E: Postcode 1073 / Commutes by car / Graduated 2019 / Hobbies: golf, reading

Tell groups of three: *"You are algorithmic auditors. Your job is to identify every potential source of bias in this dataset — then write a data policy specifying which features should be excluded from the AI's training data and what new data should be added."*

Round 1 — Feature audit (4 minutes)

Groups examine every feature in the dataset and flag any that could act as a proxy variable for a protected characteristic — factors like race, gender, class, disability, or family status that should not influence a hiring decision.

Expected findings:

- Postcode — proxy for socioeconomic status, potentially race
- Commute method — proxy for income and geography (everyone drives; anyone who takes public transport would be implicitly disadvantaged)

- Hobbies — proxy for wealth and social class (golf, sailing, skiing are expensive; nobody in the dataset has inexpensive hobbies)
- The dataset contains only five people — far too small to be representative of any meaningful population

Walk around and push groups: *"Is graduation year a bias risk? Could it correlate with age, which is a protected characteristic? What about the fact that everyone in this dataset commutes by car — what does the AI learn about people who take the bus or cycle?"*

Round 2 — Write the data policy (5 minutes)

Groups write a formal data policy with two sections:

First section — excluded features: list every variable that should be removed from the training data and give a specific reason for each exclusion. Reasons must go beyond "it could be biased" to specify the mechanism — "postcode is excluded because it correlates with socioeconomic status and historic residential segregation patterns, making it a proxy for race and income."

Second section — required additions: specify what new data must be added to make the training dataset more representative. This is harder than exclusion. Groups must think about who is missing from the dataset entirely — not just which variables are problematic within the existing data.

Round 3 — The hard question (3 minutes)

Present groups with this scenario: *"After removing all the proxy variables from the dataset, the AI's performance at predicting job success drops significantly — it is now only marginally better than random selection. The company argues that the proxy variables, while correlated with class, were genuinely predictive of performance — perhaps because the company culture itself favours certain backgrounds. Should you use a less accurate but fairer model? Or a more accurate model that perpetuates the company culture's existing biases?"*

There is no correct answer. Groups should write their position and one sentence of reasoning. Take 2–3 responses from the class. The disagreement between groups is the point.

Teacher tip: The third question is the most important. Students who say "use the fairer model always" are making a values argument. Students who say "use the more accurate model" are implicitly accepting that existing institutional bias is a legitimate selection criterion. Both positions have defenders in real organisations. The discussion should surface that tension without resolving it — because it is genuinely unresolved in practice.

Phase 5 — Discussion · 24–28 min

Pick one question:

If the proxy variable concept was the most striking: *"We said postcode is a proxy for race because of historical housing patterns. But postcode also genuinely predicts some*

things — school quality, local crime rates, access to resources — that may be relevant to certain decisions. Is there a principled way to distinguish between a proxy that should be excluded because it encodes discrimination, and a proxy that should be included because it captures a real causal factor? Or is that distinction always going to be a political judgement?"

Follow-up: *"A university's admissions algorithm uses postcode as a positive factor — giving bonus points to applicants from disadvantaged areas to increase access. The same data, used in the opposite direction, with the explicit goal of reducing inequality. Is that still proxy bias? Does the intent behind the use of a variable change its ethical status?"*

If the "fixing a biased AI" problem resonated: *"We said fixing bias in a machine learning model usually means scrapping it entirely. But these systems make decisions about real people's lives right now — loan approvals, parole decisions, job applications. If you discovered that a system currently in use was producing biased outcomes, what would be the responsible course of action? Shut it down immediately while you build a replacement? Keep it running while you work on the fix? Tell the people affected? And who has the authority — and the obligation — to make that call?"*

Exit reflection · 28–30 min

Students write one sentence:

> *"An algorithm can be biased without anyone intending it to be, because _____."*

Take 2–3 answers. Strong answers will include: the training data reflects historical inequalities, proxy variables carry discrimination without naming it, the people who built the system did not notice what patterns the data contained. Write the clearest one on the board next to the previous sentences.

Close: *"Final lesson next — you are going to apply everything from this program to design something real and defend it. Not just the technical architecture, but the ethical choices embedded in the design. You will need to explain what your system optimises for, whose interests it serves, what data it uses, and what you would do if you discovered it was producing unfair outcomes. That is what responsible technology looks like."*

Fast finisher question: *"Filter bubbles — the tendency of recommendation algorithms to show you more of what you already agree with — are sometimes described as a threat to democracy. The argument is that people living in algorithmically curated information environments never encounter views that challenge their own, making political polarisation worse. But the same algorithms are also described as giving users a better experience by showing them content they actually want to see. Is there a technical solution to the filter bubble problem — or is it a problem that cannot be solved without making the product worse for individual users? And if improving individual experience comes at a cost to collective democratic health, who should make that trade-off?"*

Teacher note: Did groups go beyond obvious proxy variables — postcode, hobbies — to flag subtler ones like commute method and graduation year? Did any group identify in Round 2 that the problem is not just which variables to exclude but who is entirely absent

from the dataset — recognising that a dataset of five people who all share similar characteristics cannot represent a diverse population no matter how carefully you filter the variables? And did the Round 3 question — whether to use a less accurate but fairer model — produce genuine disagreement within groups, not just between them? Students who disagree with each other within the same group and can articulate why are demonstrating exactly the kind of ethical reasoning this program has been building toward since Lesson 1.

HS L8

Lesson 8

How do we solve real problems? The Tech for Good pitch

Age group: 15–17 · **Duration:** 30 minutes · **Builds on:** All previous lessons **SLO:** Domain A (21A), Domain B (22B)

Learning goal: Students synthesise their knowledge from the full program — algorithms, conditionals, loops, data structures, machine learning, and bias — to design and pitch a technological system that addresses a real-world problem, including its ethical risks and limitations.

What you need: Whiteboard and marker. Paper and pencils. A timer for the pitches.

Vocabulary on the board — full program recap: Algorithm · Efficiency · Boolean logic · Loop · Function · Priority queue · Machine learning · Proxy bias · Audit · Systems design

Having all eight weeks of vocabulary visible is intentional. It signals to students how much ground they have covered and gives them the language for their pitch.

Role model: Vanessa Nakate (born 1996) — Ugandan climate activist who highlights that the people most affected by technology's consequences — and by the problems technology claims to solve — are often absent from the rooms where those technologies are designed. Her work is a reminder that empathy in design is not a soft skill. It is the difference between a system that works for the people it claims to serve and one that works for the people who built it. Today students will design for someone other than themselves.

Phase 1 — Spark question · 0–3 min

Write on the board:

> *"Technology is only as good as the problems it solves. What is the single biggest problem in the world that technology could meaningfully help with — and what is a problem that technology will make worse no matter how well it is designed?"*

Students write both answers — one sentence each — on paper. Take 3–4 responses from the class, deliberately picking contrasting ones. Write the most interesting pair on the board.

Tell students: *"Every serious technology project starts here — with a clear understanding of what you are trying to improve and an honest acknowledgment of what you might make worse. The projects that cause the most harm are usually the ones that skipped the second question entirely."*

Real world hook: The most transformative technologies of the last 20 years — smartphones, social media, search engines, streaming platforms — all started with a genuine problem worth solving. They also all created problems their designers did not anticipate or chose not to address. Today students design with both questions in mind from the start.

Phase 2 — Systems thinking warm-up · 3–8 min

Write one word on the board: **SMARTPHONE**

Hand a marker to a student and ask them to draw a line from "smartphone" to one system it depends on to function — something that has to exist and work for the phone to work. They write it and pass the marker to another student, who adds another dependency.

Keep going for 2 minutes. The board should quickly become a web: satellites, undersea cables, data centres, electricity grids, rare earth mining, semiconductor factories, software engineers, regulatory frameworks, app stores, payment systems, repair shops, recycling infrastructure.

Then ask: *"Which of these dependencies could a teenager in rural Mozambique not access? Which of these dependencies has an environmental cost? Which of them is controlled by a single company or government?"*

Tell students: *"This is systems thinking — understanding that nothing in technology works in isolation, and that every design choice connects to other systems, other people, and other consequences. Your pitch today needs to show this thinking. Not just what your system does, but what it depends on, who it serves, and what happens when one of those dependencies fails."*

Phase 3 — Design brief · 8–13 min

Write on the board:

A COMPLETE SYSTEM DESIGN ANSWERS FIVE QUESTIONS:

1. WHAT PROBLEM are you solving — and for whom specifically?
2. WHAT LOGIC does the system use?
   (Which concepts from this program does it need?)
3. WHAT DATA does it use — and what bias risks does that create?
4. WHAT COULD GO WRONG — technically, ethically, at scale?
5. WHO IS LEFT OUT if this system works exactly as designed?

Tell students: *"Question 5 is the one most technology pitches never answer. Every system optimises for certain users and implicitly deprioritises others. Being explicit about who is excluded is not a weakness in your pitch — it is evidence that you have thought seriously about the design."*

Give a brief example to make all five questions concrete. Use a system students will recognise:

"A navigation app. (1) Problem: people waste time and fuel on inefficient routes. (2) Logic: shortest-path algorithm, real-time traffic loop, conditional rerouting. (3) Data: historical traffic patterns — bias risk: rural areas and lower-income neighbourhoods are underrepresented in the training data, so routing quality is worse there. (4) What could go wrong: privacy risk from storing location data, routing failures during network outages, the algorithm routing all traffic through residential streets to save 2 minutes on the main road. (5) Who is left out: people without smartphones, people who cannot afford data, people with disabilities for whom the fastest route is not the most accessible."

Tell students: *"A real engineering team would spend weeks on those five questions before writing a single line of code. You have 7 minutes. Go."*

Phase 4 — Design and pitch · 13–28 min

Groups of three. Each group chooses a real-world problem — it can come from the UN Sustainable Development Goals, from their own community, or from any genuine problem they care about. The only constraint is that it must be something technology could meaningfully help with, and something where the risks are real enough to be worth discussing.

Design phase (7 minutes)

Groups work on paper or the whiteboard to answer all five questions. Walk around and play what the original lesson document calls "Ethical Hacker" — challenge every group with at least one question that pushes them beyond their initial design:

- *"Your system collects location data. Who stores it, for how long, and what happens if that organisation is acquired by a company with different privacy values?"*
- *"Your priority queue assigns urgency scores. Who decides the scoring criteria — and what happens to people whose situations the scoring system was not designed for?"*
- *"Your training data comes from existing records. What period does it cover, and what historical inequalities does that period contain?"*
- *"Your system requires reliable internet. What is your fallback for the people who need it most but have the least reliable connectivity?"*

Push every group to revise their design at least once based on the challenge question. A design that changes in response to a challenge is stronger than one that was never tested.

Teacher tip: Groups often try to solve too large a problem — "fix climate change" or "end poverty." Redirect them toward a specific, bounded problem: not "improve education" but "help students who miss school due to chronic illness keep pace with their class." The more specific the problem, the more honest the design. Vague problems produce vague designs that cannot be properly audited.

Pitch phase (8 minutes)

Each group has 60 seconds to pitch their system to the class. Strict timing — use a visible timer. The pitch must cover all five questions. Assign one student in each group to speak, but the other two must be ready to answer questions.

After each pitch, the class has 60 seconds to ask one question — not to critique, but to probe. The best questions will come from the vocabulary on the board: *"What data structure does that use?" "What is the bias risk in your training data?" "What is the false positive cost if your conditional fires incorrectly?"*

Teacher tip on the pitch format: 60 seconds is deliberately tight. It forces students to identify what matters most and communicate it clearly — exactly the skill that matters in real technology organisations where decisions get made in brief, high-stakes conversations. Students who try to explain everything will run out of time. Students who identify the one thing that makes their design interesting and lead with that will succeed. After the pitches, it is worth noting which groups led with the problem versus which led with the technology — the former tend to give more compelling pitches, and that pattern is worth naming.

Exit reflection · 28–30 min

Students write two sentences — not one:

> *"The most important thing I understand about technology now that I did not understand eight weeks ago is _____."*
>
> *"The question I am still not sure how to answer is _____."*

Take 3–4 responses to each sentence. The second sentence is as important as the first. Write one from each on the board.

Tell students: *"The second sentence is not a sign of failure. It is the most honest thing you can say at the end of a program like this. The questions that stay open are the ones worth carrying. Anyone who tells you they have all the answers about AI, bias, and ethics is not paying close enough attention."*

Close: *"You have spent eight weeks learning how computers think — how they sort, decide, repeat, remember, and learn. More importantly, you have spent eight weeks asking who benefits from those systems, who bears the risks, and who gets to decide. Those are not extra questions. They are the questions. Every engineer, every product manager, every policy maker working with technology faces them — and most of them have had far less practice asking them than you have now."*

Teacher note: Did any pitch answer Question 5 — who is left out — without being explicitly prompted to by you during the design phase? Did any group revise their design in response to your challenge question rather than defending the original? And did the 60-second pitch format produce a meaningful difference in quality between groups who led with the problem and groups who led with the technology? Those three observations together are the most useful feedback data from the entire program. They tell you whether students have internalised the frame — technology as a set of choices with consequences for real people — or whether they are still thinking about it as a set of technical skills. Pass both observations and the final reflection cards to whoever is responsible for the next iteration of this program. The questions students cannot yet answer are the starting points for the next cohort.

App

Sections and layout

Section 1: Learn

The main lesson track. Eight lessons per age group tied to lesson topic.

Each lesson has five exercises worth 20 XP each, 100 XP total. Completing all five earns one Logic Chip for the pet. A perfect run (no mistakes) earns a bonus 25 XP and a flame badge on that lesson.

Every exercise is one of four types:

Sort it — drag and drop items into the correct order. Used for sequences, algorithms, and sorting challenges.

Build the rule — construct a conditional, loop, or algorithm from colour-coded word blocks. Conditions in blue, actions in teal, connectors in amber. Students drag blocks into a build area and hit check.

Spot the bug — a short algorithm or ruleset with one deliberate error. Student identifies which part is wrong and selects or types the fix.

Real world connect — a short real-world scenario (hospital, logistics, social media) with one multiple choice question connecting the concept to something that actually matters.

Lessons unlock in sequence. Students can replay any completed lesson. Progress within a lesson saves automatically so it can be picked up later.

Section 2: Practice

Three standalone game modes available any time, independent of the lesson track. These reinforce concepts through repetition without requiring a full lesson session.

Algorithm Arcade — a timed sequencing game. A short scenario appears and students tap instruction blocks in the correct order before the timer runs out. Three lives per round, speed increases with level. Earns XP.

Bug Hunt — a continuous feed of short broken algorithms. Student taps the line containing the bug. Correct answers chain into a streak multiplier that increases XP per correct answer. Wrong answers reset the multiplier.

Logic Duel — asynchronous one-on-one challenge. Student is matched with another user at a similar level or challenges a classmate directly by username. Both receive the same puzzle. Fastest correct answer wins the round, best of five. Results shown when both players finish.

Section 3: Classroom

Teacher-facing section, accessed via a teacher login. Students never see this section in their own account.

Live quiz — Kahoot-style. Teacher launches a quiz from a pre-built question bank or builds a custom one. Quiz runs in real time — teacher controls the pace from their device, students answer on theirs. Questions are multiple choice or true/false. After each question, the correct answer is revealed and a live bar chart shows how the class answered. A class leaderboard updates after each question. Points are awarded for both accuracy and speed.

Pre-built question banks cover every lesson in both tracks, 10 questions per lesson. Teacher selects how many to use and whether to randomise the order.

FOR TEACHER IF WE WANT TO CONNECT STRONGLY:

dashboard — teacher sees completion rates per lesson per student, shown as a traffic light (green for complete, amber for in progress, red for not started). Clicking a student shows their exercise history. Teacher can assign specific exercises to individuals or the whole class.

Section 4: Code-A-Pet

Every user has one persistent pet that stays with them across the whole program. The pet is simple — its purpose is to make Variables and Conditionals feel real through ownership rather than explanation.

The pet has three stat bars: Energy and Mood. These are displayed visually on screen, not as numbers. Energy depletes slowly over time and is restored by feeding Logic Chips earned from lessons. Mood increases when the student interacts with the pet and completes exercises.

Evolution changes the pet's appearance — it gets more expressive and complex. Nothing else changes mechanically. Students choose their pet type at signup from four options: Bot, Sprite, Critter, or Byte.

Programs — students write simple IF/THEN rules that control the pet's automatic behaviour. Rules use the same block builder as the Learn section so the mechanic is already familiar. Triggers are limited to three to keep it simple: time of day (phone clock), whether a lesson was completed today, and current streak length. Actions are limited to: pet gains or loses Energy, pet gains or loses Mood, pet plays an animation (sleep, dance, celebrate).

Section 5: Profile and Rank

Streak — counts consecutive days with at least one completed exercise. A streak shield is earned at 14 days and protects against one missed day. Streak is shown prominently on the home screen.

XP and levels — XP accumulates across all activities. Levels 1–10 are named after computing pioneers — Grace Hopper, Margaret Hamilton, Alan Turing, Ada Lovelace, and so on. A one-line description of each person appears when the level is reached. This is the only place role models appear in the profile section — brief, contextual, not a separate feature.

Leaderboard — three tabs: this week, all time, my class. Class leaderboard requires a class code given by the teacher. Global leaderboard has an anonymous option — student appears as "Coder #[number]" if they prefer. Weekly leaderboard resets Monday midnight.

Badges — earned for completing each lesson, achieving a perfect score, hitting streak milestones (7, 14, 30 days), winning a Logic Duel, and reaching each pet evolution stage. Locked badges shown as greyed-out silhouettes so students can see what is possible. No badge descriptions until earned — discovery is part of the appeal.

Progress map — a visual row of eight lesson nodes. Completed lessons are filled, current lesson pulses, locked lessons are dimmed. Tapping a completed lesson shows the score, time taken, and badges earned. This is the student's primary view of how far through the program they are.

Design

HOME

9:41

Good morning, Noel

High School track · Lesson 3 of 8

12 day streak | 1,240 XP | Rank #4

CONTINUE LEARNING

Climate loops

Lesson 3 · 2 exercises left

+100 XP

QUICK STATS

3 Logic chips	85% Avg score
12 Day streak	8 Badges

Home Learn Pet Rank Class

EXERCISE — BUILD THE RULE

9:43

← Lesson 3 · Exercise 4

BUILD THE RULE

Write a FOR loop that reads all temperature readings and flags anything above 3°C as a warning.

FOR each reading in list

IF reading > 3

THEN flag as warning

AVAILABLE BLOCKS

ELSE | do nothing | IF

reading > 6 | flag critical

Check answer

Home Learn Pet Rank Class

CODE-A-PET

9:45

Byte — Stage 1

Logic chips: 3 Feed pet →

Byte

Stage 1 · 3/25 chips to evolve

Energy	65
Mood	80
Power	38

ACTIVE PROGRAMS (2/5)

IF time is 22:00–07:00 THEN pet sleeps + gains 10 energy

IF lesson completed today THEN pet gains 15 mood

+ Add a new program

Home Learn Pet Rank Class

CLASSROOM — LIVE QUIZ

9:47

Classroom

Teacher mode · Lesson 3 recap

LIVE — QUESTION 2 OF 5

What type of loop would you use to process exactly 1,000 temperature readings?

A — FOR loop | B — WHILE loop

C — Either works | D — No loop needed

24 Answered	18 Correct	75% Accuracy

QUIZ LIBRARY

LESSON 1 RECAP

Efficiency and algorithms

10 questions · ~8 min · HS track

Launch quiz

LESSON 2 RECAP

Boolean logic and security

10 questions · ~8 min · HS track

Launch quiz

Home Learn Pet Rank Cl

Content

1 Learn

The model

Each coding topic has one dedicated exercise set. Every exercise set contains 15 levels. Levels 1–5 are entry level — zero prior knowledge needed. Levels 6–10 build on the first five. Levels 11–15 are genuinely challenging. Every level is one short exercise, completable in under 2 minutes. Same four exercise types used throughout — Sort It, Build the Rule, Spot the Bug, Real World Connect — rotating so no two consecutive levels use the same type.

Ten topics in total, one per lesson concept. Each topic is a self-contained unit on the progress map. Students unlock topics in sequence but can replay any completed topic at any time.

The ten topics

1. Algorithms and sequences
2. Loops
3. Conditionals
4. Variables
5. Boolean logic
6. Data structures
7. Functions and abstraction
8. Machine learning and AI
9. AI bias and fairness
10. Systems design and ethics

Topic 1: Algorithms and sequences

The concept running through all 15 levels: An algorithm is a precise set of instructions. Order matters. A computer follows instructions exactly — no common sense, no guessing.

Level 1 — Sort It Put these 4 steps in order to make toast: put bread in toaster / take bread out of bag / press down the lever / take toast out when it pops up. One correct order. Immediate feedback.

Level 2 — Real World Connect A GPS app takes you down a side street instead of the main road. Why? A — The GPS has a broken screen B — The algorithm followed its instructions correctly but the instructions did not account for road conditions C — The GPS needed to be updated D — The driver made a wrong turn Correct: B.

Level 3 — Spot the Bug Find the error in this sandwich-making algorithm:

1. Open the jar of peanut butter
2. Spread peanut butter on the bread
3. Take out two slices of bread ← bug
4. Put the slices together Student taps the buggy step. One tap, done.

Level 4 — Sort It Put these 5 steps in order to send an email: write the message / open your email app / press send / type the recipient's address / add a subject line.

Level 5 — Build the Rule Complete this algorithm by dragging the missing block into the gap: Start at warehouse → [MISSING] → fly to address → land → deliver package → return Missing block: scan destination address.

Level 6 — Sort It A hospital lab processes a blood sample. Order these 6 steps correctly: run chemical analysis / receive sample from ward / centrifuge sample / log patient ID / send result to doctor / generate result report.

Level 7 — Spot the Bug A delivery robot's algorithm:

1. Pick up package
2. Walk to door
3. Ring doorbell
4. Place package on doorstep
5. Confirm delivery in system ← bug: this should happen after step 4 but before leaving, not after ringing the doorbell Student taps the misplaced step and drags it to the correct position.

Level 8 — Real World Connect Amazon's warehouse robots perform over 300 sequential steps per package. One step in the wrong position does not slow the process — it shuts down the entire floor. Why? A — Robots are fragile and break easily B — Each step depends on the output of the step before it — a sequence error breaks the dependency chain C — Amazon's robots are old technology D — The steps are timed to the second and any delay causes a timeout Correct: B.

Level 9 — Build the Rule A drone must visit warehouses A, B, C, D and return to A. Complete the algorithm by placing 4 missing blocks in the correct gaps: Start at A → [gap] → [gap] → [gap] → [gap] → return to A Available blocks: fly to B / fly to C / fly to D / confirm all visited.

Level 10 — Sort It A software team is deploying a banking app update. Order these 8 steps: write the code / get code reviewed / run security tests / deploy to test environment / run user testing / get security sign-off / deploy to production / monitor for errors.

Level 11 — Spot the Bug A self-checkout machine algorithm:

1. Scan item
2. Add price to total
3. Ask: more items?
4. If yes → go to step 1
5. Ask customer to pay total

6. Print receipt
7. Open bag area ← bug: should open bag area before asking customer to pay, not after Student identifies and moves the step.

Level 12 — Build the Rule A food delivery app must follow this rule: if the restaurant is open AND the delivery zone covers the customer's address, show the restaurant. Build this rule from blocks. Blocks available: IF / restaurant is open / AND / delivery zone covers address / THEN / show restaurant / ELSE / hide restaurant.

Level 13 — Real World Connect A 1962 NASA rocket was destroyed because of one wrong character in the flight control algorithm. The character caused the algorithm to amplify course deviations instead of smoothing them. What does this tell you about algorithms? A — Space software should not use algorithms B — One small error in the wrong place can produce catastrophic results because computers follow instructions exactly C — NASA should have tested the rocket more D — The algorithm was too complex for the technology available Correct: B.

Level 14 — Sort It A courtroom uses software to schedule hearings. Order these steps correctly: receive case filing / check judge availability / assign courtroom / notify defendant and lawyer / add to public schedule / send reminder 24 hours before. Then identify: which step would be most dangerous to place last?

Level 15 — Build the Rule A drone delivery system needs a complete algorithm. Build it from 10 blocks covering: checking package security, scanning address, taking off, flying, landing, delivering, confirming delivery, and returning. Three of the 10 blocks are distractors that should not be included. Student builds the algorithm and removes the distractors.

Topic 2: Loops

The concept running through all 15 levels: A loop repeats a set of instructions. A FOR loop runs a known number of times. A WHILE loop runs until a condition changes. A loop without a valid stop condition runs forever.

Level 1 — Real World Connect A teacher asks a student to write "I will be on time" 100 times. A smarter instruction would be: A — Write it faster B — Write it once and repeat it 100 times — that is what a loop does C — Type it instead of writing it D — Ask someone else to write it Correct: B.

Level 2 — Sort It Put these 4 parts of a FOR loop in the correct order: Set counter to 0 / check if counter has reached 10 / run the instruction / add 1 to the counter.

Level 3 — Spot the Bug

```
WHILE temperature < 20:
   turn heating ON
```

What is missing? A — A starting temperature B — An ELSE branch to turn the heating OFF when temperature reaches 20 C — The word REPEAT instead of WHILE D — Nothing — this loop is correct Correct: B.

Level 4 — Build the Rule Complete this FOR loop that sends a reminder to every student in a class of 30: FOR [MISSING] in class list: send reminder Missing block: each student.

Level 5 — Real World Connect A washing machine spin cycle runs 800 times. The programmer did not write "spin" 800 times. What did they write? A — A shortcut command B — A FOR loop: FOR 800 times: spin C — A WHILE loop: WHILE dirty: spin D — A variable set to 800 Correct: B.

Level 6 — Spot the Bug

```
FOR each temperature in the list:
   warning count = 0
   IF temperature > 3:
      add 1 to warning count
```

The warning count always ends at 0 or 1. Why? A — The list is empty B — warning count = 0 is inside the loop so it resets on every iteration C — The condition should be > 30 not > 3 D — FOR loops cannot contain IF statements Correct: B. Student must drag warning count = 0 to before the loop.

Level 7 — Build the Rule Build a WHILE loop that keeps a hospital ventilator running until a patient can breathe independently: WHILE [MISSING]: run ventilator Missing block: patient cannot breathe independently.

Level 8 — Sort It Order these 5 parts of a WHILE loop for a self-driving car braking system: Check current speed / is speed above 0 / apply brake / check speed again / stop when speed = 0.

Level 9 — Real World Connect Netflix processes 500 billion events per day — every play, pause, and scroll. No human reads them. What makes this possible? A — A very large team of data analysts B — Loops that automatically process each event one by one C — AI that understands what users are doing D — Events are deleted if nobody reads them within an hour Correct: B.

Level 10 — Spot the Bug

```
warning count = 0
FOR each reading in list:
   IF reading > 3:
      flag as warning
      add 1 to warning count
WHILE warning count < 3:
   continue reading
```

The WHILE loop is in the wrong place. Where should it replace the FOR loop? Student identifies that the WHILE loop should wrap the whole reading process, not follow the FOR loop.

Level 11 — Build the Rule Build a loop that reads ocean temperature data and stops as soon as 3 warnings are found. Use 8 blocks: warning count = 0 / WHILE / warning count < 3 / read next temperature / IF / temperature > 3 / THEN add 1 to warning count / flag reading.

Level 12 — Real World Connect In 1962 a NASA rocket was destroyed because a loop condition was wrong by one character — it amplified errors instead of correcting them. The loop ran correctly. The condition was wrong. What is the lesson? A — Loops are too dangerous to use in safety-critical systems B — A loop that runs correctly with the wrong condition produces the wrong result reliably and at full speed C — NASA should have used a different programming language D — Manual control is always safer than automated loops Correct: B.

Level 13 — Spot the Bug A social media app loops through all posts and scores them for engagement. The loop runs once when the app opens and never again. Users see the same feed all day even as new posts arrive. Find the bug: the loop should be a WHILE loop that runs continuously, not a FOR loop that runs once at startup. Student identifies the loop type as the bug and selects the correct replacement.

Level 14 — Build the Rule Build a nested structure: a loop that processes 7 days of climate data, and inside that loop, another loop that processes 24 hourly readings per day. 6 blocks available: FOR each day in week / FOR each hour in day / read temperature / IF temperature > threshold / THEN flag / end loops.

Level 15 — Real World Connect A hospital ventilator runs on a WHILE loop. A power outage interrupts the loop mid-cycle. The patient's breathing support stops. Three engineers propose fixes: A — Add a battery backup so the loop can complete even during a power outage B — Change it to a FOR loop with a fixed number of cycles C — Add a condition that resumes the loop from where it stopped when power returns Which fix addresses the actual problem — and which one introduces a new risk? Student selects the best fix and identifies the risk in one of the others. No single correct answer — the app shows what real medical device engineers do (all three in combination) and why.

Topic 3: Conditionals

The concept running through all 15 levels: A conditional checks whether something is true or false and takes a different action based on the result. IF something is true THEN do this ELSE do that.

Level 1 — Build the Rule Complete this conditional: IF it is raining THEN [MISSING] ELSE leave umbrella at home. Missing block: take an umbrella.

Level 2 — Real World Connect Automatic street lights turn on at night without anyone flipping a switch. What makes this possible? A — A timer that guesses when it is dark B — A conditional: IF sky is dark THEN turn lights on ELSE turn lights off C — A sensor that counts passing cars D — The lights are always on but dim during the day Correct: B.

Level 3 — Spot the Bug

```
IF player health = 0:
   play game over sound
```

A player reaches 0 health. The sound plays. But the game keeps running — the player can still move. What is missing? A — A louder sound B — An action to stop the game, not just play a sound C — The condition should be health < 0 not health = 0 D — Nothing is wrong Correct: B. Student adds: AND end the game.

Level 4 — Sort It Order these parts of a conditional correctly: ELSE display sold out / IF stock > 0 / THEN display as available.

Level 5 — Build the Rule A traffic light needs a conditional. Build it: IF [MISSING] THEN show green light ELSE show red light Missing block: no cars approaching from cross direction.

Level 6 — Spot the Bug

```
IF door sensor detects motion:
   unlock door
```

At 3am the sensor detects a raccoon outside. The door unlocks. What condition is missing? Student writes or selects: AND time is between 7am and 10pm / OR recognised face detected. Any answer that adds a second condition that would have prevented the raccoon unlocking the door is accepted.

Level 7 — Build the Rule A smart fridge should send a notification when milk is low. Build the complete conditional including the ELSE branch. Blocks: IF / milk level < 20% / THEN / send notification "buy milk" / ELSE / do nothing.

Level 8 — Real World Connect A spam filter blocks an email containing the word "Winner" even though it is from your employer announcing a staff raffle. What type of error is this? A — A false negative — a threat got through B — A false positive — a safe item was blocked C — A loop error D — The filter needs more training data Correct: B.

Level 9 — Sort It Order these parts of a multi-condition rule for a hospital door access system: ELSE deny access / IF staff ID is valid / AND time is within shift hours / AND department matches ID clearance / THEN unlock door.

Level 10 — Spot the Bug

```
IF transaction is overseas AND amount > €500:
   block transaction
```

A hacker in the same city as the account holder spends €600. The transaction goes through. Fix the rule to also catch large local suspicious transactions. Student adds a third condition using OR.

Level 11 — Build the Rule An emergency room triage system. Build a conditional chain for four severity levels: IF severity = 10 THEN treat immediately ELSE IF severity >= 7 THEN treat within 15 minutes ELSE IF severity >= 4 THEN treat within 1 hour ELSE add to standard queue 8 blocks provided including two distractors.

Level 12 — Real World Connect COMPAS, a US court algorithm, produced racially different outcomes despite having no race variable. Which explains this? A — The algorithm contained a hidden race variable that was not disclosed B — Variables like prior arrest history acted as proxies for race because of historical policing patterns C — The algorithm was not sophisticated enough D — The racial difference was statistically insignificant Correct: B.

Level 13 — Spot the Bug A self-driving car's braking conditional:

```
IF object detected AND distance < 5 metres:
   apply brakes
```

A child steps off the kerb 4.9 metres away during rain. The sensor reads the distance as 5.1 metres due to interference. The car does not brake. Two bugs — identify both. Bug 1: threshold too exact, should be <= 6 metres at low speeds. Bug 2: no fallback when sensor data is unreliable.

Level 14 — Build the Rule A content moderation system. Build a rule that flags a post if it contains a banned word OR was posted by an account less than 24 hours old, but NOT if it was posted by a verified journalist account. 9 blocks including two distractors.

Level 15 — Real World Connect A UK healthcare algorithm correctly reduced unnecessary medical interventions by 30%. It also caused deaths by misclassifying recoverable patients as terminal. The algorithm achieved its goal. What was the design failure? A — The algorithm had a bug in the classification logic B — Doctors should not have followed the algorithm C — The algorithm optimised for one metric and never measured harm caused by false positives D — The algorithm needed more training data Correct: C. Student then answers: what second metric should have been tracked alongside intervention reduction?

Topic 4: Variables

The concept running through all 15 levels: A variable is a named container that stores a value. The value can change. Variables let programs remember information while they run.

Level 1 — Real World Connect A game shows your score in the corner. The score changes every time you collect a coin. What stores the score between coins? A — The screen B — A variable — a labelled container that holds the current value and updates when it changes C — The internet D — The game saves it to a file every second Correct: B.

Level 2 — Build the Rule Set the starting value of a score variable: [MISSING] = 0 Missing block: SCORE.

Level 3 — Sort It Order these 4 operations correctly for a game collecting a coin worth 10 points: Check current SCORE value / add 10 to SCORE / display new SCORE / player touches coin.

Level 4 — Spot the Bug

```
LIVES = 3
FOR each level:
   LIVES = 3
   IF player falls: subtract 1 from LIVES
```

LIVES resets to 3 at the start of every level. The player can never run out of lives. Fix: move LIVES = 3 to before the loop.

Level 5 — Build the Rule A virtual pet has a HUNGER variable starting at 5. When the pet eats, hunger decreases by 2. Build this: IF pet eats THEN [MISSING] Missing block: subtract 2 from HUNGER.

Level 6 — Real World Connect Instagram knows how long you have spent watching a video even after you close the app. What makes this possible? A — Instagram is always recording your screen B — A variable stores the watch time, which updates every second and saves when you leave C — Instagram estimates based on your scroll speed D — The video server tracks how long it streams to you Correct: B.

Level 7 — Sort It A fraud detection system processes a transaction. Order these 5 steps: read RISK_SCORE variable / set RISK_SCORE = 0 / add 8 points if transaction is overseas / add 5 points if time is between 11pm and 5am / IF RISK_SCORE > 10 THEN block.

Level 8 — Spot the Bug

```
TEMPERATURE = read sensor
IF TEMPERATURE < 18:
   turn heating ON
```

The variable TEMPERATURE is read once at startup. The heating turns on if it was cold at startup and never turns off. Fix: move TEMPERATURE = read sensor inside a loop so it updates continuously.

Level 9 — Build the Rule A hospital has three variables: BEDS_AVAILABLE, PATIENTS_WAITING, ALERT_SENT. Build a rule: IF PATIENTS_WAITING > BEDS_AVAILABLE AND ALERT_SENT = false THEN send alert AND set ALERT_SENT = true. 8 blocks provided.

Level 10 — Real World Connect A self-driving car has a variable called SPEED updated 100 times per second. What happens if the sensor feeding SPEED stops working and the variable freezes at its last value? A — The car slows down automatically B — The car

continues as if it is still travelling at the frozen speed, making braking decisions based on incorrect data C — The car stops immediately D — The variable resets to 0 Correct: B.

Level 11 — Spot the Bug A canteen app:

```
STOCK = get stock from database
IF STOCK > 0:
   display as available
```

STOCK is retrieved once when the app opens and never refreshed. Students are seeing items as available that sold out an hour ago. Fix: refresh STOCK inside a loop that runs every 60 seconds.

Level 12 — Build the Rule A climate alert system needs a WARNING_COUNT variable. Build a complete loop that reads 10 temperature values, counts how many exceed 3 degrees, and triggers an alert when WARNING_COUNT reaches 3. 10 blocks provided.

Level 13 — Sort It A bank processes a transaction. Order these 8 steps involving variables: Read ACCOUNT_BALANCE / read TRANSACTION_AMOUNT / check if ACCOUNT_BALANCE > TRANSACTION_AMOUNT / IF yes: subtract TRANSACTION_AMOUNT from ACCOUNT_BALANCE / update ACCOUNT_BALANCE in database / send confirmation / IF no: send declined message / log transaction in HISTORY.

Level 14 — Spot the Bug A vending machine:

```
COINS_INSERTED = 0
[customer inserts €2]
COINS_INSERTED = COINS_INSERTED + 2
[machine dispenses item costing €1.50]
[next customer approaches]
```

COINS_INSERTED is never reset to 0 after a purchase. The next customer's transaction starts with €2 already counted. Fix: add COINS_INSERTED = 0 after dispensing the item.

Level 15 — Real World Connect A UK breast cancer screening system had a bug where a variable storing a patient's date of birth was used in an age calculation that failed for specific birth dates. Approximately 450,000 women were not invited for screening. The variable was correct — the calculation using it was wrong. What does this tell you? A — Variables are the most dangerous part of any program B — A correct variable used in a wrong calculation produces wrong results — the bug is in how the variable is used, not in the variable itself C — Date of birth should not be stored digitally D — The screening system should have been manual Correct: B.

Topic 5: Boolean Logic

The concept running through all 15 levels: Boolean logic uses AND, OR, and NOT to combine conditions. AND requires both conditions to be true. OR requires at least one. NOT flips the result.

Level 1 — Real World Connect A school door opens only for students who have a valid ID card AND are on the approved entry list. A student has a valid ID but is not on the list. Does the door open? A — Yes — they have a valid ID B — No — AND means both conditions must be true C — It depends on the time of day D — Yes — the system gives the benefit of the doubt Correct: B.

Level 2 — Build the Rule A firewall blocks a packet if it is from an unknown origin AND is an executable file. Build: IF [MISSING] AND [MISSING] THEN block Missing blocks: origin = unknown / file type = .exe.

Level 3 — Spot the Bug

```
IF transaction is overseas OR amount > €500:
   block transaction
```

A customer on holiday abroad buys a €10 coffee. Their card is blocked. Fix: change OR to AND.

Level 4 — Sort It Order the parts of this access rule: THEN unlock / AND has valid shift hours / IF staff ID is valid / ELSE deny access / AND clearance matches department.

Level 5 — Build the Rule A content filter blocks posts containing banned words OR posted by accounts less than 1 day old. Build this: IF [MISSING] OR [MISSING] THEN flag for review Missing blocks: post contains banned word / account age < 1 day.

Level 6 — Real World Connect An email spam filter blocks an email containing "Free prize" OR "Click here" OR "Urgent response needed". An email containing all three words is: A — Blocked — all three conditions are true and OR means any one is enough B — Not blocked — all three must be true C — Blocked only if the sender is unknown D — Sent to the promotions folder Correct: A.

Level 7 — Spot the Bug A hiring algorithm:

```
IF technical score > 80 NOT communication score > 80:
   flag for interview
```

This flags people with high technical scores and low communication scores — the opposite of what was intended. Fix: change NOT to AND.

Level 8 — Build the Rule A smart home locks the front door at 11pm unless someone is still outside. Build: IF [MISSING] AND NOT [MISSING] THEN lock door Missing blocks: time = 11pm / someone outside = true.

Level 9 — Sort It Order the parts of a firewall rule that blocks .exe files from unknown sources but not from the company's own IP range: IF / file type = .exe / AND / origin = unknown / AND NOT / source = internal IP range / THEN block / ELSE allow.

Level 10 — Real World Connect In 2003, one Boolean condition in a power grid alarm system in Ohio was wrong — it suppressed a critical alert instead of triggering it. 55 million people lost power for up to two days. The rest of the code was correct. What does this show? A — Boolean logic is too complex for safety-critical systems B — One wrong Boolean operator in the right place can silently suppress critical information at full system speed C — The power grid should have had a manual backup D — The software was outdated Correct: B.

Level 11 — Build the Rule A zero-trust network rule: grant access only if the user has valid credentials AND their device is registered AND their location is not flagged as suspicious. Build using 9 blocks including one distractor.

Level 12 — Spot the Bug

```
IF severity > 5 AND NOT patient is stable:
   trigger emergency alert
```

A patient with severity 7 is classified as stable by the monitoring system due to a sensor error. The alert does not trigger. Two bugs: the NOT condition blocks the alert when it should not, and the system has no fallback for sensor failure. Student identifies both.

Level 13 — Sort It Order 8 parts of a complete access control rule for a hospital covering staff, visitors, and emergency access. Rule must use AND, OR, and NOT.

Level 14 — Build the Rule A social platform moderates content using: block if post contains violence AND account is not verified, OR if account was created today, OR if post has been reported more than 5 times AND content score is below threshold. Build from 12 blocks including 3 distractors.

Level 15 — Real World Connect A facial recognition system uses: block access IF face match confidence < 95% AND NOT user has registered a backup ID method. A researcher shows the system fails more often for darker-skinned faces — not because the Boolean logic is wrong but because the confidence scores the logic depends on are less accurate for that group. What is the root problem? A — The Boolean logic needs to be rewritten B — The threshold of 95% is set too high C — The confidence scores are produced by a model trained on unrepresentative data — the Boolean logic is correct but its inputs are biased D — The system should use passwords instead Correct: C.

Topic 6: Data Structures

The concept running through all 15 levels: Data structures organise information so computers can store and retrieve it efficiently. A stack is last in, first out. A queue is first in, first out. A priority queue reorders based on importance, not arrival time.

Level 1 — Real World Connect A printer has 5 documents sent to it at the same time. Which order does it print them? A — The longest document first B — The most recently sent document first C — The first document sent first — queues are first in, first out D — Random order Correct: C.

Level 2 — Sort It Four books are stacked in this order (bottom to top): Biology, History, Maths, English. Which book comes off the top first? Drag the books in the order they would be removed from the stack. Answer: English, Maths, History, Biology. Last in, first out.

Level 3 — Build the Rule A printer queue receives three jobs: Report (sent first), Spreadsheet (sent second), Photo (sent third). Complete the rule: The [MISSING] job prints first. Missing block: first sent / Report / most recent. Correct: first sent.

Level 4 — Real World Connect The undo button in any app reverses your most recent action first, not your oldest. What data structure makes this possible? A — A queue — first action is undone first B — A stack — last action is undone first C — A variable storing all past actions D — A loop that replays actions in reverse Correct: B.

Level 5 — Spot the Bug A hospital waiting room app uses a standard queue — patients are seen in arrival order. Patient D arrives last with a cardiac arrest. Patient A arrived first with a paper cut. The app sends Patient A in first. What is wrong? A — The queue is too slow B — A standard queue uses arrival time — this system needs a priority queue that orders by urgency C — The app has a calculation error D — Patient D should have arrived earlier Correct: B.

Level 6 — Sort It Five patients arrive at different times with different severity scores. Order them as a priority queue would — highest severity first, with arrival time as tiebreaker:

- Patient A: arrived 9:00, severity 3
- Patient B: arrived 9:05, severity 9
- Patient C: arrived 9:10, severity 6
- Patient D: arrived 9:15, severity 9
- Patient E: arrived 9:20, severity 2 Correct order: B, D, C, A, E. B and D are tied on severity — B wins on arrival time.

Level 7 — Build the Rule Complete this priority queue rule for an emergency room: IF [MISSING] THEN move patient to front of queue Missing block: new patient severity > current front of queue severity.

Level 8 — Real World Connect Your phone's operating system decides which app gets processor time next. It uses a priority queue. A phone call comes in while a game is running in the background. What happens? A — The game crashes to make room for the call B — The call is added to the queue and waits its turn C — The call gets highest priority and interrupts the game immediately D — Both run at the same time with equal resources Correct: C.

Level 9 — Spot the Bug A stack is used to track a user's navigation history in an app. When the user taps Back, the app removes the most recent screen. But the app always returns to the home screen, not the previous screen. What is the bug? A — The stack is too small B — The Back button is clearing the entire stack instead of removing just the top item C — Navigation history should use a queue not a stack D — The stack is storing screen names incorrectly Correct: B.

Level 10 — Sort It A stack of browser tabs opened in order: Gmail, YouTube, Twitter, Reddit, BBC News. The user presses Back five times. Order the tabs as they close: BBC News, Reddit, Twitter, YouTube, Gmail. Last opened, first closed.

Level 11 — Build the Rule A traffic management system uses a priority queue at a junction. Build the complete rule: Set base priority = 1 for all vehicles IF vehicle is an ambulance THEN set priority = 10 IF vehicle is a bus THEN set priority = 5 Process vehicles in [MISSING] order Missing block: highest priority first. 7 blocks provided including one distractor.

Level 12 — Real World Connect A content moderation team receives 10,000 posts flagged for review per day. They use a priority queue weighted by: number of times reported, account age, and whether the content contains specific keywords. A post reported 200 times is reviewed before a post reported twice. What risk does this create? A — Popular posts get unfair treatment B — A coordinated group could artificially boost a post's priority by mass-reporting it, pushing legitimate content aside C — The queue will never empty D — Newer posts are always deprioritised Correct: B. Student then answers: what rule would you add to prevent this?

Level 13 — Spot the Bug A hospital triage algorithm. Two patients arrive simultaneously with severity 8 each. The algorithm has no tiebreaker — it crashes rather than choosing. Find the missing rule and write it: student selects from options including arrival time as tiebreaker, patient age, random assignment, or staff override. App shows what real triage systems use and asks whether students agree.

Level 14 — Build the Rule A network router handles data packets from four sources: a video call (priority 9), a file download (priority 3), an email (priority 2), and a smart home sensor (priority 6). Build a priority queue rule that also handles the case where two packets have equal priority. 10 blocks including 2 distractors.

Level 15 — Real World Connect A bail decision algorithm in US courts assigns risk scores that determine how long defendants wait in custody before trial. The algorithm uses a priority queue ordered by risk score — higher scores wait longer. Researchers found the queue produced different average wait times for defendants of different racial backgrounds despite race not being a variable. What caused this? A — The algorithm contained hidden racial bias B — The risk score variables acted as proxies for race through historical data patterns, causing the priority ordering to reflect past discriminatory decisions C — The queue was processing too slowly D — The researchers misread the data Correct: B. Student then answers in one sentence: what would you change about the data feeding into the risk score?

Topic 7: Functions and Abstraction

The concept running through all 15 levels: A function is a named, reusable block of instructions. Abstraction means hiding complexity behind a simple interface. Large systems are built from many independent functions that communicate through agreed inputs and outputs.

Level 1 — Real World Connect When you press the volume button on your phone, you do not need to know how the speaker hardware works. You just press the button and the volume changes. What concept does this illustrate? A — A loop running in the background B — Abstraction — complex internal details are hidden behind a simple interface C — A variable storing the volume level D — A conditional checking whether sound is allowed Correct: B.

Level 2 — Build the Rule A function called calculateTotal takes a price and a quantity and returns the total cost. Complete it: calculateTotal(price, quantity): RETURN [MISSING] Missing block: price × quantity.

Level 3 — Sort It Order the steps of calling a function: Write the function definition / give the function a name / call the function by name in the main program / function runs its instructions / function returns the result.

Level 4 — Spot the Bug A developer calls a function called sendEmail but forgets to provide the recipient's address. The function runs but sends the email to nobody. What is missing? A — The function needs a loop B — The function was called without its required input — the recipient address C — The function should be called sendMessage not sendEmail D — Email functions do not need a recipient address Correct: B.

Level 5 — Real World Connect WhatsApp was maintained by a team of 50 engineers serving 450 million users. How was a team that small able to manage a system that large? A — They worked extremely long hours B — Most of the system ran automatically without human intervention C — The system was broken into independent functions that different small teams could build and maintain separately — abstraction meant no one person needed to understand the whole system D — They outsourced most of the work Correct: C.

Level 6 — Build the Rule A food waste app has three functions: loginUser(), findNearestShelter(), claimFood(). Complete the main program flow: Step 1: call [MISSING] Step 2: call [MISSING] Step 3: call [MISSING] Missing blocks in correct order: loginUser() / findNearestShelter() / claimFood().

Level 7 — Spot the Bug Two teams are building a navigation app. Team A builds getLocation() which returns coordinates as text: "52.3, 4.9". Team B builds calculateRoute() which expects coordinates as two separate numbers. When Team B calls Team A's function, the app crashes. What went wrong? A — The functions are named incorrectly B — The teams agreed on what the function would do but not on the format of the output — the interface contract was broken C — getLocation() has a bug in its internal logic D — calculateRoute() should be called before getLocation() Correct: B.

Level 8 — Sort It A software team is deploying a new feature. Order these abstraction layers from lowest level (closest to hardware) to highest level (closest to the user): Operating system / Physical processor / App feature / Programming language / User interface button. Correct: Physical processor / Operating system / Programming language / App feature / User interface button.

Level 9 — Build the Rule A hospital records system has a function called getPatientRecord(patientID). It takes one input and returns the patient's full record. Build the

call that retrieves the record for patient ID 10472: result = MISSING Missing blocks: getPatientRecord / 10472.

Level 10 — Real World Connect In 1999 NASA lost the $327 million Mars Climate Orbiter because one team calculated thrust in metric units and another expected imperial units. Both functions worked correctly. The interface between them was wrong. What does this show? A — NASA should have used better hardware B — Functions that work correctly can still fail catastrophically if the agreed interface between them is imprecise C — Space missions should not use software from multiple teams D — The functions should have been written in the same programming language Correct: B.

Level 11 — Spot the Bug A developer builds a large app as one continuous block of 10,000 lines rather than breaking it into functions. A bug is found on line 7,432. The fix on line 7,432 accidentally breaks something on line 2,108. Why does this happen less often with functions? A — Functions are faster to run B — Functions isolate logic — a change inside one function cannot affect another function unless the interface changes, because the internal details are hidden C — Functions automatically fix bugs D — Functions are shorter so there are fewer places for bugs to hide Correct: B.

Level 12 — Build the Rule A social media app has functions: loadFeed(), scorePost(post), rankFeed(scores), displayFeed(ranked). Build the main program that calls them in the correct order. 6 blocks provided.

Level 13 — Sort It A team is building a fraud detection system. Match each function to its correct description:

- collectTransaction() → receives and stores raw transaction data
- extractFeatures(transaction) → pulls relevant data points from the transaction
- calculateRiskScore(features) → assigns a risk value based on the features
- makeDecision(score) → blocks or allows based on the score
- logOutcome(decision) → records what happened for auditing Order them as they would run in sequence.

Level 14 — Real World Connect An app developer refactors their code — they rewrite the internal logic of a function to make it faster, without changing the function's name, inputs, or outputs. Every other part of the app continues to work without any changes. What does this demonstrate? A — The app was poorly designed originally B — Abstraction — as long as the interface contract is honoured, internal changes are invisible to the rest of the system C — The other parts of the app were not connected to this function D — Refactoring always improves performance Correct: B.

Level 15 — Build the Rule Design the function architecture for a school timetabling app. Student receives 8 function names and must: match each to its correct input and output, identify which two functions must communicate directly (their output feeds the next one's input), and identify which function is the single point of failure — if it breaks, everything else breaks. No single correct answer for the last question — the app shows what real system architects consider a single point of failure and why redundancy is built around it.

Topic 8: Machine Learning and AI

The concept running through all 15 levels: AI learns patterns from examples rather than following fixed rules. The examples it learns from are called training data. The patterns it finds are called the model. The model makes predictions about new cases it has never seen.

Level 1 — Real World Connect A child learns to recognise dogs by being shown 100 dogs and told "this is a dog." They then correctly identify a dog they have never seen before. This is how machine learning works. What is the training data? A — The 100 dogs the child was shown B — The child's brain C — The new dog they had never seen D — The word "dog" Correct: A.

Level 2 — Sort It Order the steps of training a machine learning model: Collect labelled examples / identify useful features in the examples / adjust weights based on errors / make predictions on new data / test accuracy on unseen data.

Level 3 — Build the Rule A spam filter learns from examples. For each email in the training data, it checks these features: contains "free money" / sender is unknown / has an attachment / subject line is all caps. Complete: the more features are [MISSING], the higher the spam score. Missing block: present / absent / unusual / marked. Correct: present.

Level 4 — Real World Connect Before Fei-Fei Li built ImageNet (14 million labelled images), AI could not reliably identify a cat in a photo. After training on ImageNet it could. The algorithm did not change significantly. What changed? A — The computers became faster B — The training data became larger and more diverse C — A better algorithm was discovered D — Cameras became higher resolution Correct: B.

Level 5 — Spot the Bug A music recommendation model was trained only on songs listened to on Friday evenings. On Monday mornings it recommends dance music to people trying to focus at work. What is the bug? A — The model is too simple B — The training data was collected from a specific time period — the model learned patterns that only apply in that context C — The model needs more features D — Music recommendation should use rules not machine learning Correct: B.

Level 6 — Build the Rule A fraud detection model assigns weights to features. Assign weights (1–5) to these features by dragging the correct number next to each:

- Transaction is overseas (suggested weight: 4)
- Transaction time is 3am (suggested weight: 3)
- Amount is over €1000 (suggested weight: 3)
- Device is unrecognised (suggested weight: 5)
- Transaction is for a new merchant (suggested weight: 2) Student sets weights. App tests the model against 5 transactions and shows accuracy. Student adjusts weights to improve accuracy — this is the concept of model tuning without using that term.

Level 7 — Sort It Order these steps for a streaming service building a recommendation model: Collect viewing history from users / identify features (genre, length, time watched, completion rate) / assign initial weights to features / test recommendations against a group of users / measure satisfaction / adjust weights / deploy to all users.

Level 8 — Spot the Bug A model trained to identify whether a tumour is malignant achieves 95% accuracy. The doctors are delighted. But researchers notice that 90% of tumours in the training data were benign. The model gets 95% accuracy by predicting "benign" for every single case — it has never learned to identify malignant tumours. What is the bug? A — The model needs more data overall B — 95% accuracy sounds good but the training data was so imbalanced that the model learned to always predict the majority class — it has zero ability to detect what actually matters C — The model should use a different algorithm D — The doctors should not use AI for cancer detection Correct: B.

Level 9 — Build the Rule Complete this description of a model's decision process: IF weighted feature score > [MISSING] THEN predict fraud ELSE predict legitimate The value in the gap is called the threshold. Student drags a number between 5 and 15 into the gap, then tests against 5 labelled transactions. The app shows false positives (legitimate transactions blocked) and false negatives (fraud that got through) for their chosen threshold. Student adjusts the threshold and observes the trade-off.

Level 10 — Real World Connect Amazon's hiring algorithm was trained on 10 years of successful hire data. It taught itself to penalise CVs containing the word "women's" (as in "women's chess club"). Amazon scrapped it. What happened? A — The algorithm had a bug in its text processing B — The algorithm correctly learned from the training data — the training data reflected a decade of hiring that was predominantly male, so the model learned maleness as a feature of success C — The algorithm was designed by biased engineers D — Women's CVs are formatted differently and confused the model Correct: B.

Level 11 — Spot the Bug A model predicts student exam performance based on: previous grades, attendance, homework submission rate, and postcode. The model is more accurate in schools in wealthy areas than in schools in less wealthy areas. No bug is visible in the code. Where is the problem? A — The model needs more variables B — The training data was collected predominantly from schools in wealthy areas — the model learned patterns from one context and is less reliable in a different context it was not trained on C — Postcode should not be included as a variable D — The model is accurate enough for practical use Correct: B. Student then answers: what would you change about the data collection process?

Level 12 — Build the Rule A content recommendation model uses these features: completion rate of last video, genre match, time of day, session length, explicit search query. Assign weights and set a threshold for "recommend this content." Then answer: which single feature, if removed, would most reduce the risk of the model creating a filter bubble?

Level 13 — Sort It Order these steps for auditing a machine learning model for bias: Define what fair means for this context / collect outputs grouped by demographic / compare error rates across groups / identify features that may act as proxies / test model with modified data / decide whether to retrain or discard / document findings.

Level 14 — Real World Connect A hospital uses an AI model to predict which patients need urgent follow-up. The model is 89% accurate overall. Researchers find it is 94% accurate for patients over 50 but only 79% accurate for patients under 30. No bug is found in the code. What explains the gap? A — Younger patients are healthier so there is less data about them B — The training data contained more cases from older patients — the model

learned their patterns better and is less reliable for a group it saw less often C — The model should be recalibrated for age D — 79% is still acceptable accuracy for clinical use Correct: B. Student then answers: at what accuracy gap between groups would you stop using this model?

Level 15 — Build the Rule Design the complete pipeline for a fraud detection model. Student receives 10 components — data collection, feature extraction, weight assignment, threshold setting, testing on labelled data, measuring false positive rate, measuring false negative rate, deploying, monitoring live accuracy, retraining when accuracy drops — and must arrange them in the correct order, identify which two steps must happen simultaneously (testing false positives and false negatives), and identify which step must have a human in the loop before the model is deployed. No fully correct answer — the app shows what responsible ML deployment looks like and asks what the student would add.

Topic 9: AI Bias and Fairness

The concept running through all 15 levels: AI bias occurs when a model produces systematically unfair outcomes for specific groups. Bias enters through training data, through proxy variables, and through what the model is asked to optimise for. Fixing bias requires changing the data and the goal — not just the code.

Level 1 — Real World Connect A facial recognition system is trained on 1 million photos. 80% of the photos are of lighter-skinned faces. The system is significantly less accurate for darker-skinned faces. Why? A — The algorithm is designed to work better for lighter-skinned faces B — The camera hardware performs differently for different skin tones C — The model learned from unrepresentative data and is more accurate for the group it saw most often D — Facial recognition cannot work for all skin tones Correct: C.

Level 2 — Sort It Order these steps for how bias enters a model: Collect training data from existing records / existing records reflect historical decisions / historical decisions were sometimes unfair / model learns the historical patterns / model reproduces those patterns in new decisions / bias is present in outputs despite no intent to discriminate.

Level 3 — Build the Rule A hiring algorithm uses these features: university attended, years of experience, previous job title, postcode, hobbies listed on CV. Identify which features could act as proxies for wealth or class by dragging them into the RISKY column. Student drags university attended, postcode, and hobbies. Years of experience and previous job title go in the ACCEPTABLE column.

Level 4 — Real World Connect COMPAS, a US court algorithm, has no race variable. It still produces racially different outcomes. Which explanation is correct? A — The algorithm was programmed with hidden racial preferences B — Variables like prior arrest history and postcode correlate with race due to historical policing patterns — they act as proxies C — The racial difference is statistically insignificant D — Black defendants commit more crimes — the model is accurate Correct: B.

Level 5 — Spot the Bug A school reading app recommends books at each student's reading level. Students who start the year with low scores receive simpler books. At the end of the year, the gap between high and low scorers is larger than at the start. What is the bug? A — The app is recommending the wrong books B — The personalisation logic is working correctly but creating a feedback loop — lower-scoring students get less challenging material, which limits their development, which keeps their scores low C — Students with low scores should not use the app D — The scoring algorithm is calculating reading levels incorrectly Correct: B.

Level 6 — Build the Rule A data policy for a hiring algorithm. Student must drag each variable into INCLUDE or EXCLUDE:

- Years of experience → Include
- University ranking → Exclude (proxy for wealth)
- Postcode → Exclude (proxy for race and class)
- Previous job titles → Include
- Hobbies → Exclude (proxy for class)
- References from previous employers → Include
- Name → Exclude (proxy for gender and ethnicity)

Level 7 — Sort It Order the steps of an algorithmic audit: Define what fair means in this context / collect model outputs grouped by demographic / compare accuracy and error rates across groups / identify variables acting as proxies / test model after removing proxy variables / measure whether fairness improved / decide whether to redeploy.

Level 8 — Real World Connect A healthcare algorithm reduced unnecessary medical interventions by 30%. It also caused deaths by misclassifying recoverable patients as terminal. The algorithm achieved its stated goal. What was the design failure? A — The algorithm contained a classification bug B — Doctors over-relied on the algorithm C — The goal was measured by one metric — interventions reduced — while the harm caused by false positives was never tracked D — The training data was too small Correct: C.

Level 9 — Spot the Bug A credit scoring model is 93% accurate overall. Researchers find it generates false positives — incorrectly denying credit — at twice the rate for applicants from lower-income postcodes compared to higher-income postcodes. The algorithm contains no income or postcode variable. Where is the problem? A — 93% overall accuracy is sufficient — some groups will always have higher error rates B — The training data reflects historical lending decisions that were less favourable to lower-income applicants — the model learned those patterns C — The researchers used the wrong statistical method D — Credit scoring should use postcode as a variable to improve accuracy Correct: B.

Level 10 — Build the Rule A fairness test for a model. Build the rule that flags the model for review: IF error rate for group A > error rate for group B by more than [MISSING]% THEN flag model for bias review Student drags a threshold percentage (5%, 10%, 20%) into the gap and sees how many real-world models would be flagged at each threshold. At 5% almost all major deployed models are flagged. At 20% almost none are. Student answers: which threshold would you set and why?

Level 11 — Sort It A social media platform's recommendation algorithm optimises for engagement. Research shows it consistently recommends more extreme content after political videos. Order these responses from least to most effective: Add a warning label to extreme content / reduce the weight of engagement as a metric / add user wellbeing as a second metric / retrain the model with a combined objective / redesign the goal from scratch / shut the system down temporarily. No single correct order — the app shows what different platforms have tried and what the research says about effectiveness.

Level 12 — Real World Connect A predictive policing algorithm tells officers where to patrol based on historical crime data. More patrols → more arrests in that area → more data showing crime in that area → algorithm recommends more patrols. What is this pattern called? A — A feedback loop — the algorithm's outputs become its own training data, amplifying the original pattern regardless of whether it reflects actual crime levels B — A sorting error C — A loop condition failure D — A proxy variable problem Correct: A.

Level 13 — Spot the Bug An algorithm recommends job candidates to recruiters. After removing race, gender, and postcode variables, the output still shows gender disparity. Researchers identify that the variable "gap in employment history" strongly predicts gender because women are more likely to have taken career breaks for caregiving. Find the proxy variable and suggest a rule that addresses it without removing employment history entirely.

Level 14 — Build the Rule Design a fairness policy for a hiring algorithm. Student must write three rules using blocks: Rule 1 — which variables are excluded and why Rule 2 — what fairness metric is measured and at what threshold the model is flagged Rule 3 — who must review the model before it is deployed to real candidates 9 blocks provided including 2 distractors. Student then answers: even with all three rules, what bias risk remains?

Level 15 — Real World Connect The UK Ofqual algorithm awarded predicted A-level grades during the COVID pandemic. It was designed to prevent grade inflation by adjusting school-level grades based on historical performance. Students in schools with a history of lower grades had their individual predictions reduced — even when their teachers predicted high grades based on that student's actual work. Students from private schools were less affected because private schools have smaller class sizes, making the statistical adjustment less severe. The algorithm was withdrawn after public outcry. What was the fundamental error? A — The algorithm had a calculation bug B — The algorithm punished individual students for the historical performance of their school — it optimised for statistical consistency at the school level while causing systematic unfairness at the individual level C — Teachers' predictions were unreliable and should have been ignored D — The algorithm needed more training data Correct: B. Student then answers: whose interests should an exam grading algorithm optimise for — the system's statistical consistency or the individual student's opportunity? Is there a way to do both?

Topic 10: Systems Design and Ethics

The concept running through all 15 levels: Technology does not exist in isolation. Every system affects multiple stakeholders, creates unintended consequences, and encodes

values through its design choices. Responsible design asks who benefits, who is harmed, and who decides.

Level 1 — Real World Connect A smartphone requires satellites, undersea cables, data centres, rare earth mining, and a global supply chain to function. If any one of these fails, the phone stops working. What concept does this illustrate? A — Technology is fragile B — Systems thinking — no technology works in isolation, it depends on other systems that each have their own dependencies and failure modes C — Smartphones are too complex D — Technology should be simpler Correct: B.

Level 2 — Sort It Match each technology decision to its stakeholder impact:

- An app stores user location data → affects user privacy
- A recommendation algorithm prioritises engagement → affects user wellbeing
- A hiring algorithm uses proxy variables → affects job applicants from certain backgrounds
- A smart city sensor tracks pedestrian movement → affects anyone who walks past it, whether they consented or not. Student drags each decision to the stakeholder group most directly affected.

Level 3 — Build the Rule Complete this ethical design checklist question: Before deploying this system, have we identified [MISSING] who will be affected by it — including people who did not choose to use it? Missing block: all stakeholders.

Level 4 — Real World Connect Facebook's Like button was designed to spread positivity. It also created a system where users' self-worth became tied to social validation metrics, and where content designed to provoke strong reactions spread faster than calm content. Nobody designed these outcomes. What are they called? A — Design bugs B — Unintended consequences — outcomes that emerge from a design decision that were not planned and often not noticed until significant harm has already occurred C — User errors D — Marketing failures Correct: B.

Level 5 — Spot the Bug A school app tracks student attendance using facial recognition. It was designed to save teachers time. Three problems emerge: students without smartphones cannot be tracked, the system is less accurate for students with certain physical features, and parents were not told their children's biometric data was being collected. Find the design bug in each case. Student matches each problem to its cause: accessibility failure / training data bias / missing consent mechanism.

Level 6 — Build the Rule Complete this stakeholder analysis for a new app that connects food banks with surplus food from supermarkets: User group 1: food banks → benefit: [MISSING] User group 2: supermarkets → benefit: [MISSING] User group 3: people receiving food → potential risk: [MISSING] User group 4: delivery drivers → potential risk: [MISSING] Student selects from option banks for each gap. Multiple valid answers — the app shows the range of responses and asks which risk the design team should prioritise first.

Level 7 — Sort It Order these design thinking stages correctly: Empathise with the users / define the problem clearly / generate possible solutions / build a prototype / test with real users / iterate based on feedback.

Level 8 — Real World Connect A ride-sharing company's surge pricing algorithm automatically raises prices during emergencies — storms, attacks, evacuations. The algorithm was optimising for supply and demand balance. People trying to evacuate during an emergency paid five to ten times normal fares. The algorithm was working exactly as designed. What was the design failure? A — The surge pricing formula was miscalculated B — The algorithm optimised for market efficiency with no exception for situations where price gouging during an emergency causes disproportionate harm to people in crisis C — The drivers should have refused surge pricing voluntarily D — Surge pricing should be banned entirely Correct: B.

Level 9 — Spot the Bug A city deploys smart traffic cameras that also collect data on pedestrian movement patterns. The stated purpose was traffic management. The data is later shared with law enforcement for crowd monitoring during protests. Three design failures — find them all: Student identifies: purpose limitation failure (data collected for one purpose used for another), consent failure (pedestrians did not consent to being monitored), and oversight failure (no governance structure prevented the data being repurposed).

Level 10 — Build the Rule Build an ethical use policy for a hospital AI diagnostic tool. Student must complete three rules: Rule 1: The AI output must be [MISSING] by a qualified clinician before any treatment decision is made. Rule 2: If the AI and the clinician disagree, the [MISSING] decision takes precedence. Rule 3: Every case where the AI was wrong must be [MISSING] and used to retrain the model. Missing blocks: reviewed / clinician's / logged.

Level 11 — Sort It A company discovers its hiring algorithm has a 12% higher false positive rate for female candidates than male candidates. Order these responses from least to most responsible: Continue using the model and monitor for further complaints / add a disclaimer to candidate rejection emails / pause the model immediately / investigate the source of the bias in training data / retrain the model with balanced data / deploy a third-party audit before redeployment / publish the findings regardless of outcome. No single correct order — the app shows what responsible AI governance frameworks recommend and where most companies stop.

Level 12 — Real World Connect In 2021 the EU introduced the right to an explanation for automated decisions affecting individuals. A bank using an AI model to deny a loan application must now explain why. The bank's model cannot explain its decisions — it only outputs a score. What does this mean for the bank? A — The bank must stop using AI for lending decisions B — The bank must either switch to a model that can explain its decisions or provide a human review process alongside the AI — the right to explanation cannot be satisfied by an opaque model C — The bank can explain the decision by describing the model's training data D — The regulation only applies to governments not private companies Correct: B.

Level 13 — Spot the Bug A school uses an AI system to identify students at risk of dropping out. The system is 84% accurate. It flags students to counsellors who offer support. Researchers notice that students from lower-income families are flagged at twice the rate of

students from higher-income families and receive intervention meetings that take them out of class. The support is well-intentioned. Find the two bugs: the proxy variable causing over-flagging of lower-income students, and the unintended consequence that the intervention itself (missing class) may be causing harm.

Level 14 — Build the Rule Design a complete ethical framework for deploying an AI system in a school. Student must write five rules covering: what data is collected and why / who has access / how long data is retained / what happens when the model is wrong / how students and parents are informed. 12 blocks provided including 3 distractors. Student then answers: which rule is most likely to be ignored under time and budget pressure — and how would you make it harder to skip?

Level 15 — Real World Connect You are part of the team that built a content recommendation algorithm. Internal research shows it increases watch time by 40% and increases exposure to politically extreme content by 70% for users who watch political videos. The algorithm is profitable. Fixing it would reduce watch time and revenue. Your manager asks you to archive the research findings rather than share them publicly. What should you do?

A — Archive the findings as instructed — the decision is above your pay grade B — Share the findings with your immediate team but take no further action C — Report the findings through internal channels and document that you did so — if internal channels fail, escalate to relevant regulatory or oversight bodies D — Resign and publish the findings on social media

The app does not mark one answer as fully correct. It shows what happened in real documented cases (Facebook, Google, Twitter internal researchers who faced this exact situation), what protections exist in EU law for whistleblowers, and asks: what would you actually do — and is there a gap between what you said and what you think you would do in practice?

Practice

Practice

The Learn section teaches concepts through structured lessons. The Classroom section is teacher-led. The Pet section creates daily habit. Practice is where students go when they want to use the app but are not in a lesson, not in class, and just want something engaging to do for a few minutes.

It does not teach new concepts. It reinforces what students have already encountered in the Learn section — but in a faster, more game-like format that rewards repetition and speed rather than careful thinking. The exercises are shorter, the feedback is immediate, and the competitive element is more prominent.

The three game modes

Algorithm Arcade A timed reflex game. A short scenario appears — a broken sequence, a misplaced step, a wrong order — and the student must fix it before the timer runs out. Three lives per round. Speed increases with each correct answer. Ends when the student runs out of lives. Score is the number of consecutive correct answers before the first mistake. Global leaderboard shows today's top scores. This is the purest skill test in the app — no reading, no options, just pattern recognition and speed.

Bug Hunt A continuous scrolling feed of short broken algorithms. Student scrolls through and taps the line containing the bug. Correct answers build a streak multiplier — the longer the streak, the more XP per correct answer. Wrong answers reset the multiplier to zero. No lives, no timer, no end state — students play as long as they want. The leaderboard shows today's longest correct streak globally and within the class. This is the most addictive mode — the multiplier mechanic and the absence of a formal end state means students naturally push themselves to beat their own streak.

Logic Duel Asynchronous one-on-one. Student is matched with another user at a similar level or challenges a classmate directly by username. Both receive the same puzzle. Fastest correct answer wins the round. Best of five rounds wins the duel. Results shown when both players have finished — not in real time. Students can have multiple active duels simultaneously. Winning earns 30 XP and counts toward the Duel Winner and Duel Champion badges.

Mode 1: Algorithm Arcade

The format A scenario appears on screen. Student has 15 seconds to answer. Correct — next question immediately, timer resets, speed stays the same for the first 5 correct answers then increases slightly every 3 correct answers after that. Wrong or timeout — one life lost. Three lives total. Game ends when all lives are gone. Score is the total number of correct answers in that run.

Each question is one of three formats, rotating:

Fix the order — four steps shown in the wrong order, student drags the correct step to the highlighted gap. One gap, one fix, done.

Spot the one — five lines of an algorithm shown, one is wrong, student taps it. No explanation needed — just the tap.

Complete it — an algorithm with one block missing, two options shown, student taps the correct one.

All questions are designed to be answerable in under 10 seconds by a student who knows the concept. The challenge is speed and pattern recognition, not working something out from scratch.

Content bank — sequences and algorithms (unlocked from Lesson 1)

Fix the order:

- Make toast: [take out bread] → [put in toaster] → [press lever] → [take out toast] — one step pre-shuffled, student fixes it
- Send a message: open app → type message → add recipient → press send — recipient and type message swapped
- Withdraw cash: insert card → enter PIN → select amount → take cash → take card — take cash and take card swapped
- Board a flight: check in → go through security → find gate → board plane → take seat — find gate and board plane swapped
- Start a car: put on seatbelt → insert key → check mirrors → start engine → release handbrake — start engine and check mirrors swapped

Spot the one:

- Drone algorithm with "return to base" appearing before "deliver package"
- Email algorithm with "press send" appearing before "write the message"
- Shopping algorithm with "pay for items" appearing before "scan items at checkout"
- Login algorithm with "enter password" appearing before "enter username"
- Recipe algorithm with "put cake in oven" appearing before "mix the ingredients"

Complete it:

- Deliver package: take off → fly to address → [?] → lower package — options: land / scan address. Correct: land
- Make tea: boil kettle → [?] → pour water → wait 3 minutes — options: put teabag in cup / add milk. Correct: put teabag in cup
- Send email: open app → [?] → write message → press send — options: type recipient / close app. Correct: type recipient
- Withdraw cash: insert card → enter PIN → [?] → take cash — options: select amount / take card. Correct: select amount
- Log in: go to website → [?] → enter password → access account — options: enter username / click logout. Correct: enter username

Content bank — loops (unlocked from Lesson 2)

Fix the order:

- FOR loop structure: set counter to 0 → check counter < 10 → run instruction → add 1 to counter — add 1 and run instruction swapped
- WHILE loop: check condition → run instruction → update condition → check again — update condition and run instruction swapped
- Climate data: set warning count to 0 → read next temperature → check if above threshold → add 1 to warning count — check and add swapped
- Printing queue: receive document → check printer is ready → send to printer → confirm printed — confirm and send swapped
- Charging check: check battery level → IF below 20% → send low battery alert → wait 60 seconds — wait and send swapped

Spot the one:

- Loop with warning count = 0 inside the loop instead of before it
- WHILE loop with condition WHILE temperature > 20 to heat a room — condition should be < 20
- FOR loop that processes 100 items but counter starts at 1 and checks counter < 100, missing the last item
- Loop that reads temperature data but updates the variable inside the ELSE branch instead of every iteration
- Heating loop with no ELSE branch — turns on but never turns off

Complete it:

- FOR [?] in student list: send reminder — options: each student / all. Correct: each student
- WHILE warning count [?] 3: keep reading — options: < / >. Correct:
- FOR each item in order list: [?] price to total — options: add / subtract. Correct: add
- Set counter = 0 before the [?] — options: loop / condition. Correct: loop
- WHILE temperature [?] 20: turn heating on — options: < / >. Correct:

Content bank — conditionals (unlocked from Lesson 3)

Fix the order:

- IF/THEN/ELSE: IF stock > 0 → THEN display available → ELSE display sold out — THEN and ELSE swapped
- Access rule: IF ID is valid → AND shift hours match → THEN unlock door → ELSE deny access — THEN and ELSE swapped
- Fraud check: read transaction → check risk score → IF score > threshold → THEN block — IF and check risk score swapped
- Alarm: detect motion → IF time is night → THEN trigger alarm → log event — log event and trigger alarm swapped
- Triage: check severity → IF severity > 8 → THEN treat immediately → ELSE add to queue — IF and check severity swapped

Spot the one:

- IF transaction overseas OR amount > 500 — should be AND
- IF player health = 0: play sound — missing: end the game
- IF door detects motion: unlock door — missing: AND time is daytime
- IF file = .exe: block — missing: AND origin = unknown
- IF temperature < 20: turn heating on — missing: ELSE turn heating off

Complete it:

- IF stock > 0 THEN display available [?] display sold out — options: ELSE / AND. Correct: ELSE
- IF file = .exe [?] origin = unknown THEN block — options: AND / OR. Correct: AND
- IF severity > 8 THEN treat immediately [?] IF severity > 5 THEN treat within 1 hour — options: ELSE / AND. Correct: ELSE
- IF player health = 0 THEN [?] — options: end the game / add 10 health. Correct: end the game
- IF motion detected AND [?] THEN trigger alarm — options: time is night / door is open. Both accepted — app shows both are valid and asks which is more precise

Content bank — variables (unlocked from Lesson 4)

Fix the order:

- Score update: player touches coin → read SCORE → add 10 to SCORE → display new SCORE — read and display swapped
- Fraud score: set RISK = 0 → check location → add points to RISK → check RISK against threshold — set RISK and check location swapped
- Inventory check: read STOCK → IF STOCK > 0 → display available → ELSE display sold out — IF and read STOCK swapped
- Login attempt: read USERNAME → read PASSWORD → check against database → grant or deny access — grant and check swapped
- Pet energy: read current ENERGY → IF ENERGY < 20 → send alert → ELSE do nothing — IF and read ENERGY swapped

Spot the one:

- LIVES = 3 inside the loop instead of before it
- SCORE variable never reset between game rounds
- TEMPERATURE variable read once at startup and never updated inside the loop
- COINS_INSERTED never reset to 0 after a purchase
- WARNING_COUNT incremented but never initialised — starts at a random value

Complete it:

- [?] = 0 before the loop starts — options: warning count / IF. Correct: warning count
- SCORE = SCORE [?] 10 when coin collected — options: + / -. Correct: +
- IF ENERGY [?] 20 THEN send low energy alert — options: < / >. Correct:
- LIVES = LIVES [?] 1 when player falls — options: - / +. Correct: -

- Read [?] from sensor every 60 seconds inside the loop — options: TEMPERATURE / IF. Correct: TEMPERATURE

Content bank — Boolean logic (unlocked from Lesson 5)

Fix the order:

- Access rule: IF ID valid → AND time in shift hours → AND clearance matches → THEN unlock → ELSE deny — THEN and ELSE swapped
- Firewall: read packet → check origin → check file type → IF unknown AND .exe → THEN block — IF and check file type swapped
- Spam filter: read email → check for banned words → check sender → IF banned word OR new sender → flag for review — flag and IF swapped
- Fraud check: check location → check amount → check time → IF overseas AND night AND > 500 → block — block and IF swapped
- Content flag: read post → check keywords → check account age → IF keywords OR age < 1 day → flag — IF and check account age swapped

Spot the one:

- IF transaction overseas OR amount > 500 — OR should be AND, too many false positives
- IF file = .exe AND NOT source = internal network — NOT applied to wrong condition
- IF severity > 5 AND NOT patient is stable — NOT means it never triggers for unstable patients above severity 5
- IF banned word OR verified journalist — OR means journalists get flagged too, should be AND NOT
- IF staff ID valid OR shift hours match — OR too permissive, should be AND

Complete it:

- IF origin = unknown [?] file = .exe THEN block — options: AND / OR. Correct: AND
- IF motion detected AND [?] time is night THEN alarm — options: NOT / AND. Correct: NOT is wrong here — student must identify AND is the correct connector
- IF post flagged [?] account age < 1 day THEN review — options: OR / AND. Correct: OR
- IF transaction overseas AND amount > 1000 [?] device unrecognised THEN flag — options: OR / AND. Both defensible — app shows both and asks which produces fewer false positives
- IF staff ID valid AND shift hours match AND [?] clearance matches THEN unlock — options: NOT / missing nothing. Correct: nothing missing — student must identify the rule is already complete

Mode 2: Bug Hunt

The format A feed of short algorithms, one at a time. Each algorithm is 4–6 lines. One line contains the bug. Student scrolls through and taps the line. Correct — streak goes up by 1, XP awarded based on current multiplier, next algorithm loads immediately. Wrong — streak resets to 0, multiplier resets to 1, correct line highlighted briefly, next algorithm loads.

Multiplier increases every 5 correct answers: 1x at 0–4, 2x at 5–9, 3x at 10–14, 4x at 15–19, 5x at 20+. XP per correct answer at 1x is 5. At 5x it is 25. The multiplier is shown clearly on screen — students feel it building and feel the reset.

No lives. No timer. No end state. Students play until they choose to stop.

Bug bank — sequences (unlocked from Lesson 1)

Each entry shows the full algorithm as written and the buggy line in brackets.

1.

1. Open the jar of peanut butter

2. Spread peanut butter on the bread [BUG — step 3 not done yet]

3. Take out two slices of bread

4. Put the slices together

5. Cut in half

Bug on line 2 — spreading before taking out the bread.

2.

1. Type the recipient's address

2. Open your email app [BUG — app not open yet]

3. Write the message

4. Add a subject line

5. Press send

Bug on line 1 — typing recipient before opening the app.

3.

1. Insert card into ATM

2. Take the cash [BUG — cash not dispensed yet]

3. Enter PIN

4. Select amount

5. Take card

Bug on line 2 — taking cash before entering PIN or selecting amount.

4.

1. Check in online

2. Arrive at airport

3. Board the plane [BUG — security not done]

4. Go through security

5. Find the gate

Bug on line 3 — boarding before security.

5.

1. Start the car

2. Insert the key [BUG — key not inserted]

3. Put on seatbelt

4. Check mirrors

5. Release handbrake

Bug on line 1 — starting before inserting key.

Bug bank — loops (unlocked from Lesson 2)

6.

1. FOR each student in class:

2. warning count = 0 [BUG — resets every iteration]

3. read their grade

4. IF grade < 50: add 1 to warning count

5. Send report with warning count

Bug on line 2.

7.

```
1. Set counter = 0
2. WHILE counter < 10:
3.     send reminder
4. Add 1 to counter  [BUG — outside the loop]
5. End
```

Bug on line 4 — counter update outside the loop, infinite loop.

8.

```
1. WHILE temperature > 20:  [BUG — wrong direction]
2.     turn heating ON
3. ELSE:
4.     turn heating OFF
```

Bug on line 1 — should be < 20, heating turns on when already warm.

9.

```
1. warning count = 0
2. FOR each reading in list:
3.     IF reading > 3:
4.         flag reading
5.         add 1 to warning count
6. WHILE warning count < 3: continue  [BUG — WHILE after FOR]
```

Bug on line 6 — the WHILE loop should replace the FOR loop, not follow it.

10.

```
1. WHILE printer has jobs:
2.     print next document
3.     remove from queue
4. End when queue empty  [BUG — end condition outside loop]
```

Bug on line 4 — end condition should be the WHILE condition, not a separate line.

Bug bank — conditionals (unlocked from Lesson 3)

11.

1. IF transaction is overseas OR amount > €500: [BUG — OR too broad]
2. block transaction
3. ELSE:
4. allow transaction

Bug on line 1 — OR blocks all overseas transactions regardless of amount.

12.

1. IF door sensor detects motion:
2. unlock door [BUG — no time check]
3. ELSE:
4. keep door locked

Bug on line 2 — unlocks any time, should check time of day.

13.

1. IF player health = 0:
2. play game over sound [BUG — game keeps running]
3. ELSE:
4. continue game

Bug on line 2 — should end the game, not just play a sound.

14.

1. IF stock > 0:
2. display as available
3. [BUG — no ELSE branch]
4. Continue to next item

Bug on line 3 — missing ELSE: display as sold out.

15.

```
1. IF severity > 8:
2.     treat immediately
3. ELSE IF severity > 5:
4.     treat within 1 hour
5. ELSE IF severity > 8:  [BUG — duplicate condition]
6.     treat within 30 minutes
```

Bug on line 5 — duplicate condition, unreachable code.

Bug bank — variables (unlocked from Lesson 4)

16.

```
1. FOR each level in game:
2.     LIVES = 3  [BUG — resets every level]
3.     IF player falls: LIVES = LIVES - 1
4.     IF LIVES = 0: game over
```

Bug on line 2.

17.

```
1. TEMPERATURE = read sensor
2. WHILE temperature < 18:  [BUG — variable never updates]
3.     turn heating on
4. ELSE: turn heating off
```

Bug on line 1 — TEMPERATURE read once, never updated inside loop.

18.

```
1. COINS_INSERTED = 0
2. Customer inserts €2
3. COINS_INSERTED = COINS_INSERTED + 2
4. Dispense item
```

5. [BUG — COINS_INSERTED never reset]

6. Next customer approaches

Bug on line 5 — missing COINS_INSERTED = 0 after dispensing.

19.

1. FOR each transaction:

2. RISK_SCORE = RISK_SCORE + location_points [BUG — never initialised]

3. RISK_SCORE = RISK_SCORE + time_points

4. IF RISK_SCORE > 10: block

Bug on line 2 — RISK_SCORE never set to 0 before the loop.

20.

1. STOCK = get stock from database

2. IF STOCK > 0: display available

3. ELSE: display sold out

4. Wait 30 minutes [BUG — STOCK never refreshed]

5. Go to line 2

Bug on line 4 — STOCK should be re-read from the database, not just waited out.

Bug bank — Boolean logic (unlocked from Lesson 5)

21.

1. IF file = .exe OR origin = unknown: [BUG — OR too broad]

2. block packet

3. ELSE:

4. allow packet

Bug on line 1.

22.

1. IF technical score > 80 NOT communication score > 80: [BUG — NOT instead of AND]

2. flag for interview

Bug on line 1.

23.

1. IF staff ID valid OR shift hours match: [BUG — OR instead of AND]

2. unlock door

3. ELSE:

4. deny access

Bug on line 1.

24.

1. IF post contains banned word AND account verified: [BUG — AND instead of AND NOT]

2. flag for review

Bug on line 1 — verified accounts should not be flagged.

25.

1. IF severity > 5 AND NOT patient is critical: [BUG — NOT blocks critical patients]

2. trigger alert

Bug on line 1 — NOT means critical patients above severity 5 never trigger the alert.

Mode 3: Logic Duel

The format Student is matched with one opponent at a similar XP level or challenges a classmate by username. Both receive the same puzzle. Each duel is best of five rounds. Fastest correct answer wins the round. Results shown when both players finish — not in real time. Multiple duels can run simultaneously.

Each round is one of three puzzle types:

Which rule is correct — two rules shown, one is right, one has a subtle bug. Student taps the correct one.

What does this output — a short algorithm shown with a specific input. Student selects what the output would be from three options.

Name the concept — a one-sentence description of a behaviour. Student selects the correct concept name from three options.

Duel puzzle bank — sequences

Which rule is correct:

- Rule A: scan address → take off → fly → land → deliver vs Rule B: take off → scan address → fly → land → deliver. Correct: Rule A
- Rule A: write code → test → review → deploy vs Rule B: write code → deploy → test → review. Correct: Rule A
- Rule A: insert card → enter PIN → select amount → take cash vs Rule B: insert card → select amount → enter PIN → take cash. Correct: Rule A

What does this output:

- Algorithm: open bag → take out bread → spread butter → put together. Input: the steps are followed correctly. Output options: a finished sandwich / bread still in the bag / an error. Correct: a finished sandwich
- Algorithm: scan item → add to total → ask for more items. If yes: go to step 1. If no: request payment. Input: three items scanned. Output options: total of three items / payment requested after first item / error. Correct: total of three items

Name the concept:

- "A set of step-by-step instructions for solving a problem" → Algorithm / Loop / Variable. Correct: Algorithm
- "One step depends on the output of the step before it" → Dependency / Condition / Loop. Correct: Dependency
- "The same algorithm produces a catastrophic result when applied to one million items instead of ten" → Scale / Bug / False positive. Correct: Scale

Duel puzzle bank — loops

Which rule is correct:

- Rule A: warning count = 0 then FOR loop vs Rule B: FOR loop with warning count = 0 inside. Correct: Rule A
- Rule A: WHILE temperature < 18: turn heating on ELSE turn heating off vs Rule B: WHILE temperature < 18: turn heating on (no ELSE). Correct: Rule A
- Rule A: FOR each student: send reminder vs Rule B: send reminder FOR each student (action before loop). Correct: Rule A

What does this output:

- Loop: warning count = 0. FOR each of 5 readings (2, 5, 1, 8, 4): IF reading > 3 add 1 to warning count. Output options: warning count = 3 / warning count = 5 / warning count = 2. Correct: 3 (readings 5, 8, 4 are above 3)
- Loop: counter = 0. WHILE counter < 3: add 1 to counter. Output options: counter = 3 / counter = 4 / counter = 2. Correct: 3

Name the concept:

- "A loop that runs forever because the stop condition is never reached" → Infinite loop / False negative / Dependency error. Correct: Infinite loop
- "A loop that runs a known number of times" → FOR loop / WHILE loop / Conditional. Correct: FOR loop

- "The number of times a loop runs through its instructions once" → Iteration / Variable / Threshold. Correct: Iteration

Duel puzzle bank — conditionals

Which rule is correct:

- Rule A: IF stock > 0 THEN available ELSE sold out vs Rule B: IF stock > 0 THEN available (no ELSE). Correct: Rule A
- Rule A: IF overseas AND amount > 500 THEN block vs Rule B: IF overseas OR amount > 500 THEN block. Correct: Rule A
- Rule A: IF health = 0 THEN end game AND play sound vs Rule B: IF health = 0 THEN play sound. Correct: Rule A

What does this output:

- Rule: IF stock > 0 THEN available ELSE sold out. Input: stock = 0. Output options: available / sold out / error. Correct: sold out
- Rule: IF overseas AND amount > 500 THEN block ELSE allow. Input: overseas transaction, amount €300. Output options: blocked / allowed / flagged for review. Correct: allowed
- Rule: IF severity > 8 THEN immediate ELSE IF severity > 5 THEN 1 hour ELSE standard queue. Input: severity 6. Output options: immediate / 1 hour / standard queue. Correct: 1 hour

Name the concept:

- "A legitimate user blocked by a security rule" → False positive / False negative / Infinite loop. Correct: False positive
- "A threat that gets through a security rule" → False negative / False positive / Edge case. Correct: False negative
- "A valid input that a rule was not designed to handle" → Edge case / Bug / Variable. Correct: Edge case

Duel puzzle bank — variables

Which rule is correct:

- Rule A: LIVES = 3 before the loop vs Rule B: LIVES = 3 inside the loop. Correct: Rule A
- Rule A: TEMPERATURE = read sensor inside the loop vs Rule B: TEMPERATURE = read sensor before the loop only. Correct: Rule A
- Rule A: COINS_INSERTED = 0 after dispensing vs Rule B: COINS_INSERTED = 0 never reset. Correct: Rule A

What does this output:

- Code: SCORE = 0. Player collects 3 coins worth 10 each. SCORE = SCORE + 10 three times. Output options: SCORE = 30 / SCORE = 10 / SCORE = 3. Correct: 30

- Code: LIVES = 3. Player falls twice. LIVES = LIVES - 1 twice. Output options: LIVES = 1 / LIVES = 2 / LIVES = 0. Correct: 1

Name the concept:

- "A named container that stores a value which can change" → Variable / Function / Loop. Correct: Variable
- "Setting a variable back to its starting value" → Resetting / Looping / Initialising. Correct: Resetting — app notes both resetting and initialising are acceptable terms
- "A variable that is read once and never updated causes this problem" → Stale data / False positive / Edge case. Correct: Stale data

Duel puzzle bank — Boolean logic

Which rule is correct:

- Rule A: IF .exe AND origin unknown THEN block vs Rule B: IF .exe OR origin unknown THEN block. Correct: Rule A
- Rule A: IF ID valid AND shift hours match THEN unlock vs Rule B: IF ID valid OR shift hours match THEN unlock. Correct: Rule A
- Rule A: IF banned word AND NOT verified journalist THEN flag vs Rule B: IF banned word AND verified journalist THEN flag. Correct: Rule A

What does this output:

- Rule: IF overseas AND amount > 500 THEN block. Input: local transaction, €600. Output options: blocked / allowed / flagged. Correct: allowed — local fails the first condition
- Rule: IF file = .exe AND origin = unknown AND NOT source = internal THEN block. Input: .exe file, unknown origin, source = internal. Output options: blocked / allowed / error. Correct: allowed — NOT internal means internal source passes through

Name the concept:

- "A variable that is not a direct measure of what the algorithm is trying to predict but correlates with it" → Proxy variable / False negative / Edge case. Correct: Proxy variable
- "Both conditions must be true for the rule to fire" → AND / OR / NOT. Correct: AND
- "At least one condition must be true for the rule to fire" → OR / AND / NOT. Correct: OR

Classroom

Classroom Section Content

How the classroom quiz works

Teacher launches a quiz from their device. Students join on their phones, they join via a link or class code displayed on the teacher's screen. Questions appear one at a time on student devices. Teacher controls the pace. After each question the correct answer is revealed with a brief explanation and a live bar chart shows how the class answered. Points are awarded for accuracy and speed. Class leaderboard updates after each question.

Each quiz takes 8–10 minutes. Designed to be used at the start of a lesson as a recap of the previous week, or at the end as an exit check.

Lesson 1 Quiz — How does a computer think?

Quiz title: Algorithms and sequences **Recommended timing:** Start of Lesson 2, recap of Lesson 1 **Number of questions:** 8 **Estimated time:** 8 minutes

Question 1 What is an algorithm?

A — A type of computer hardware B — A set of step-by-step instructions for solving a problem C — A programming language D — A computer that thinks for itself

Correct: B Points: 100 Shown after answer: *"An algorithm is just a set of precise instructions. Recipes, morning routines, and LEGO instructions are all algorithms — the computer just follows them faster and more literally than a human would."*

Question 2 You give a robot these instructions: "Put the peanut butter on the bread." The robot picks up the closed jar and places it on top of the closed bag of bread. What went wrong?

A — The robot is broken B — The instruction was not precise enough — the robot followed it exactly as written C — The robot does not understand English D — The instruction was in the wrong order

Correct: B Points: 200 Shown after answer: *"Computers and robots follow instructions exactly and literally. If the instruction does not say 'open the jar first' or 'take out a slice of bread,' those steps do not happen. Precision is the whole job."*

Question 3 True or false: the order of steps in an algorithm does not matter as long as all the steps are included.

A — True B — False

Correct: B Points: 100 Shown after answer: *"Order is everything. Putting your shoes on before your socks works physically — but the result is wrong. In code, a step in the wrong position can break everything that comes after it."*

Question 4 A GPS app takes you through a tiny side street instead of the main road. The fastest route at each junction was always chosen. What type of algorithm caused this?

A — A sorting algorithm B — A greedy algorithm — it always picks the best local option without considering the overall result C — A broken algorithm D — A loop that ran too many times

Correct: B Points: 300 Shown after answer: *"A greedy algorithm makes the best available decision at each step. That often works — but it can lead to a worse overall outcome when short-term best decisions accumulate into a long-term bad result."*

Question 5 Which of these is NOT an example of an algorithm?

A — A recipe for making pasta B — The steps to tie a shoelace C — The colour blue D — Instructions for assembling flat-pack furniture

Correct: C Points: 100 Shown after answer: *"An algorithm has to be a set of instructions — it has to tell something or someone what to do in what order. A colour is just a value, not a process."*

Question 6 A delivery drone visits warehouses in this order: A → C → B → D → E → A. Using the 'always go to the nearest unvisited warehouse' rule, the drone ends up stranded at E with no direct route home. What does this tell us?

A — The drone's navigation system is broken B — A correct algorithm can still produce a bad result if it only looks one step ahead C — The drone should have started at a different warehouse D — The rule should use a loop instead

Correct: B Points: 300 Shown after answer: *"This is the greedy failure. Every decision was locally correct. The overall result was a trap. Good algorithms need to consider future consequences, not just the next step."*

Question 7 What is the difference between an algorithm that works and an algorithm that is good?

A — A good algorithm uses more steps B — A good algorithm produces the right result with less wasted time and effort — it is efficient C — A working algorithm is only for simple problems D — There is no difference

Correct: B Points: 200 Shown after answer: *"A working algorithm gives you the right answer eventually. A good algorithm gives you the right answer efficiently — using the least time, memory, and effort. At scale, the difference between the two can be enormous."*

Question 8 Amazon's logistics algorithms sort millions of packages per hour. A 1% improvement in the algorithm saves hundreds of millions of euros. A 1% degradation causes cascading delays. What does this show about algorithms at scale?

A — Large companies need larger algorithms B — Small improvements or errors in an algorithm have consequences that grow proportionally with the scale of the system using it C — Amazon's algorithms are more important than other companies' D — Manual sorting would be more reliable

Correct: B Points: 400 Shown after answer: *"Scale amplifies everything. An algorithm that is slightly inefficient for 10 items is catastrophically inefficient for 10 million. This is why efficiency is not a bonus — it is the whole point."*

Lesson 2 Quiz — How do computers make secure decisions?

Quiz title: Boolean logic and cybersecurity **Recommended timing:** Start of Lesson 3, recap of Lesson 2 **Number of questions:** 8 **Estimated time:** 9 minutes

Question 1 What does AND mean in a conditional rule?

A — Either condition can be true B — Both conditions must be true C — The first condition is more important D — The conditions cancel each other out

Correct: B Points: 100 Shown after answer: *"AND is strict. Both sides must be true for the rule to fire. If either side is false, the whole condition is false. This is how computers combine multiple checks — every condition in the AND chain must pass."*

Question 2 A firewall rule says: IF file type = .exe OR origin = unknown THEN block. A legitimate software update from the company's own IT team is blocked. Why?

A — The file type is wrong B — OR means either condition alone is enough to trigger the block — the .exe file type triggered it even though the origin was trusted C — The firewall has a memory error D — The IT team used the wrong file format

Correct: B Points: 200 Shown after answer: *"OR casts a wide net. Anything matching either condition gets caught — including legitimate traffic. This is a false positive. In security, the challenge is writing rules precise enough to catch threats without blocking safe traffic."*

Question 3 What is a false positive in cybersecurity?

A — A threat that gets through the firewall B — A safe user or file that gets blocked by the security rules C — A password that is almost correct D — An alarm that rings too quietly

Correct: B Points: 100 Shown after answer: *"A false positive blocks something safe. A false negative lets something dangerous through. Both are failures — just in opposite directions. Good security design minimises both, but in high-stakes contexts the cost of each failure is different."*

Question 4 True or false: NOT flips the result of a condition. IF NOT user is logged in THEN deny access — this blocks users who are NOT logged in.

A — True B — False

Correct: A Points: 100 Shown after answer: *"NOT inverts the Boolean value. True becomes false, false becomes true. It is useful when it is easier to describe the exception than the rule: allow everything except this specific case."*

Question 5 A hospital firewall blocks all .exe files from unknown origins. A trusted doctor connects from a new personal laptop. The laptop is unrecognised — it counts as an unknown device. Their files are blocked. Which concept does this demonstrate?

A — A loop error B — A false positive caused by an edge case — a situation the rule was not designed to handle C — The doctor used the wrong file type D — The firewall is working incorrectly

Correct: B Points: 300 Shown after answer: *"Rules that work for most cases often fail at the edges. A trusted person on an unrecognised device is an edge case the simple rule cannot handle. Real security systems add layers to handle edge cases without blocking legitimate users."*

Question 6 Which rule correctly blocks .exe files from unknown origins but NOT from the company's own internal network?

A — IF file = .exe THEN block B — IF file = .exe AND origin = unknown THEN block C — IF file = .exe AND origin = unknown AND NOT source = internal network THEN block D — IF file = .exe OR origin = unknown THEN block

Correct: C Points: 400 Shown after answer: *"C adds the NOT condition to carve out the exception. The rule catches the threat while protecting the legitimate use case. This layering of AND and NOT to handle exceptions is how real firewall rules are structured."*

Question 7 In 2017, the NHS WannaCry ransomware attack encrypted files across 80 NHS trusts. The attack spread through the network because the firewall rules were not configured to block a specific file type and port combination. What would have stopped it?

A — Faster computers B — One additional AND condition in the firewall rules C — A larger IT team D — Better passwords

Correct: B Points: 300 Shown after answer: *"Boolean logic at the right place at the right time. The technical fix was simple — the failure was not having the condition in place before the attack. This is why security is a design problem, not just a response problem."*

Question 8 Zero-trust security means no user or device is automatically trusted — everyone must be verified every time, even if they were trusted before. What is the main trade-off of zero-trust?

A — It makes the system slower and more expensive to run B — It creates friction for legitimate users — every access requires verification, which costs time and effort C — It is less secure than traditional security models D — It only works for large organisations

Correct: B Points: 200 Shown after answer: *"Zero-trust maximises security by removing all assumed trust. The cost is usability — legitimate users face more verification steps. This is*

the core tension in security design: more secure systems are usually harder to use. The right balance depends on the stakes."

Lesson 3 Quiz — How do computers handle massive data?

Quiz title: *Loops and data processing* ***Recommended timing:*** *Start of Lesson 4, recap of Lesson 3* ***Number of questions:*** *8* ***Estimated time:*** *9 minutes*

Question 1 *What is a loop in programming?*

A — A type of error that causes a program to crash B — A set of instructions that repeats until a condition is met or a set number of times C — A way to store multiple values in one variable D — A connection between two different programs

Correct: B Points: 100 Shown after answer: "A loop says: do this again and again. Without loops, processing a list of 10,000 items would require writing the same instruction 10,000 times. With a loop, you write it once."

Question 2 *What is the difference between a FOR loop and a WHILE loop?*

A — FOR loops are faster than WHILE loops B — FOR loops run a known number of times. WHILE loops run until a condition changes — you do not know in advance how many times they will repeat C — WHILE loops can only be used with numbers D — There is no practical difference

Correct: B Points: 200 Shown after answer: "FOR: process all 30 students in the register. WHILE: keep the ventilator running until the patient can breathe independently. The first has a fixed count. The second has a condition. Choosing the right type matters."

Question 3 *True or false: a WHILE loop with a condition that is always true will run forever.*

A — True B — False

Correct: A Points: 100 Shown after answer: "This is an infinite loop — one of the most common and consequential bugs in software. The loop never finds a reason to stop. In a minor case, the app freezes. In a safety-critical system, the consequences can be far more serious."

Question 4 *A climate scientist needs to read 10,000 temperature readings and flag any above 3 degrees. Writing "check reading 1, check reading 2..." for all 10,000 would take forever. What is the solution?*

A — Use a conditional instead B — Use a FOR loop: FOR each reading in the list: IF reading > 3 THEN flag it C — Use a variable to store all 10,000 readings at once D — Hire more scientists

Correct: B Points: 200 Shown after answer: "The loop reduces 10,000 individual instructions to one instruction that repeats. The conditional inside the loop makes the decision on each

reading. This combination — loop + conditional — is the foundation of almost every data processing system ever built."

Question 5 *This loop has a bug. Find it:*

```
warning count = 0

FOR each reading in list:

    warning count = 0

    IF reading > 3: add 1 to warning count
```

What is wrong?

A — The FOR loop should be a WHILE loop B — warning count = 0 is inside the loop so it resets on every iteration — the count never accumulates C — The threshold should be > 30 not > 3 D — FOR loops cannot contain IF statements

Correct: B Points: 300 Shown after answer: "Initialising a variable inside a loop that should be initialised before it is one of the most common bugs beginners write. The fix is simple: move warning count = 0 to the line before the loop starts. Small placement, big consequence."

Question 6 *When should you switch from a FOR loop to a WHILE loop?*

A — When the list is very long B — When you want the loop to stop based on a condition that changes during the loop, not after a fixed number of runs C — When the program is running too slowly D — FOR and WHILE loops are interchangeable — use whichever you prefer

Correct: B Points: 300 Shown after answer: "If the climate alert system should stop as soon as 3 warnings are found — regardless of how many readings are left — a WHILE loop is correct. WHILE warning count < 3: keep reading. The stop condition is dynamic, not fixed."

Question 7 *Netflix processes 500 billion events per day — every play, pause, and scroll. No human reads them. What makes this possible?*

A — A very large team of data analysts working continuously B — Loops that automatically process each event according to a set of rules, running continuously without human intervention C — Events are deleted if no action is taken within one second D — Netflix uses a different type of computer that does not need loops

Correct: B Points: 200 Shown after answer: "Every data pipeline at every technology company is built on loops. The scale is different from the climate reading exercise — but the concept is identical. A loop processes one item, then the next, then the next, until the condition says stop."

Question 8 *In 1962 NASA's Mariner 1 rocket was destroyed because one character in a loop condition was wrong — the loop amplified course errors instead of correcting them. What does this tell us about loops in safety-critical systems?*

A — Safety-critical systems should not use loops B — A loop running correctly with the wrong condition produces wrong results reliably, at full speed, with no way to tell from the outside that anything is wrong C — NASA should have used better hardware D — Manual control is always safer than automated loops

Correct: B Points: 400 Shown after answer: "A loop does not know it is wrong. It executes the condition it was given, correctly, every single time. If the condition is wrong, the loop produces the wrong output reliably and at full speed. The computer does not hesitate or doubt — it executes. That is why testing loop conditions carefully is not optional."

Code-A-Pet

Code-A-Pet

What it is

A simple digital pet that lives in the app. It has two stats: Energy and Mood. Students keep it alive by doing lessons. They make it smarter by writing simple IF/THEN rules for it. That is the whole thing.

The two stats

Energy (0–100) Represents how powered-up the pet is. Depletes slowly over time — roughly 10 points every 24 hours when the student is inactive. Restored by completing lessons (each completed lesson set restores 20 Energy) and by feeding the pet Logic Chips earned from exercises. If Energy reaches 0 the pet enters dormant mode — it sits still, greyed out, until the student feeds it a Logic Chip.

Mood (0–100) Represents how happy the pet is. Does not deplete automatically over time — it only changes in response to specific events. Mood increases when the student completes an exercise, wins a Logic Duel, or checks in with the pet daily. Mood decreases when the student misses three or more days in a row. Mood affects the pet's animation — at high Mood it bounces and plays, at low Mood it sits quietly.

Both stats are shown as simple visual bars on the pet screen, labelled Energy and Mood. No numbers displayed — just the bar. Students learn to read the bar, not a number. This is intentional: the bar is more immediately intuitive and avoids making the pet feel like a spreadsheet.

The four pets

Student chooses one at signup. This is the only choice — no customisation beyond the initial selection. Keeping it simple.

Bot — a small boxy robot with expressive LED eyes. Moves by rolling. Celebrations involve spinning and flashing.

Sprite — a floating ghost-like creature with a soft rounded form. Moves by drifting. Celebrations involve glowing and pulsing.

Critter — a small four-legged creature with large ears. Moves by scurrying. Celebrations involve jumping and wagging.

Byte — an abstract geometric shape — a dodecahedron that rotates and shifts. Moves by floating. Celebrations involve colour shifts and geometric transformations.

Each pet has three visual stages tied to Logic Chip count. Stage 1 is the base form, simple and small. Stage 2 at 25 chips adds detail and expressiveness. Stage 3 at 75 chips is the fully evolved form — more complex, more animated, clearly different from Stage 1. Evolution is visual only. The stats and mechanics stay the same across all stages.

Logic Chips

Earned by completing a full lesson set of 5 exercises. One chip per set. Each chip fed to the pet restores 20 Energy. Chips never expire and are never taken away.

Profile

Streaks

A streak counts consecutive days where the student completes at least one exercise. The streak number is shown prominently on the home screen — it is the first thing a returning student sees.

Streak shield: Earned automatically at 7 days. Protects against one missed day without breaking the streak. Used automatically — the student does not need to activate it. One shield maximum at any time. A new shield is earned after the next 7-day run.

Streak milestones shown with a small animation when reached: 3 days, 7 days, 14 days, 30 days, 100 days. No fanfare beyond that — brief, satisfying, done.

XP and levels

XP is earned across everything:

Activity	XP
Exercise correct first attempt	20
Exercise correct after retry	10
Daily pet check-in	5
Completing a full lesson set	25 bonus
Perfect lesson set (no mistakes)	50 bonus
Winning a Logic Duel	30
Streak milestone reached	20

Levels 1–10 named after computing pioneers. Name and one-line description shown when the level is reached — nothing else, no separate screen, just a brief moment of recognition before moving on.

Level	Name	Pioneer
1	Hopper	Grace Hopper — invented the compiler
2	Lovelace	Ada Lovelace — first algorithm
3	Hamilton	Margaret Hamilton — Apollo software

4	Turing	Alan Turing — foundations of computing
5	Easley	Annie Easley — NASA algorithms
6	Johnson	Katherine Johnson — Moon landing calculations
7	Perlman	Radia Perlman — internet architecture
8	Goldberg	Adele Goldberg — modular programming
9	Li	Fei-Fei Li — ImageNet and AI vision
10	Buolamwini	Joy Buolamwini — algorithmic justice

Levels beyond 10 continue with increasing XP thresholds but no new names. The pioneer list is complete at 10.

Leaderboard

Three tabs: This week / All time / My class.

This week resets every Monday at midnight. Shows top 20 students globally.

All time never resets. Shows top 20 students globally across the full program.

My class shows all students in the same class code. No global ranking here — just the class. This is the most used tab in classroom settings.

Each row shows: rank, username, level name, XP for the relevant period. No other personal information visible.

Anonymous option: Student can switch their display name to "Coder #[number]" for the global tabs at any time in settings. Their own rank is always visible to them regardless. Classmates in My class always see each other's usernames — anonymity only applies to the global leaderboard.

Badges

Earned silently — no interrupt, just a quiet indicator that a new badge has been collected. Student sees the full collection on their profile page.

Locked badges shown as grey silhouettes. No descriptions until earned — discovery is part of the appeal.

Badge list:

Badge	How to earn

First step	Complete your first exercise
On a roll	3-day streak
Week one	7-day streak
Fortnight	14-day streak
Monthly	30-day streak
Century	100-day streak
No mistakes	Perfect score on any lesson set
Clean sweep	Perfect score on 5 lesson sets
Bug hunter	Complete all Spot the Bug exercises in one topic
Duel winner	Win your first Logic Duel
Duel champion	Win 10 Logic Duels
Logic chip	Earn your first Logic Chip
Well fed	Feed your pet 10 Logic Chips total
Evolved	Pet reaches Stage 2
Fully evolved	Pet reaches Stage 3
Halfway	Complete lessons 1–4
Finished	Complete all 8 lessons
Top of class	Reach number 1 on the class leaderboard for a week

17 badges total. Enough to be motivating without being overwhelming.

Progress map

A row of 8 lesson nodes on the profile page. Completed lessons filled and labelled. Current lesson pulsing. Locked lessons dimmed.

Tapping a completed lesson shows three things: score achieved, badges earned from it, date completed. Nothing else. Clean and fast.